

GR24 Regulations

III B.Tech I Semester

Micro Controller and Internet of Things Lab

(GR24A3095)

Department of Computer Science and Engineering

GOKARAJU RANGARAJU
INSTITUTE OF ENGINEERING AND TECHNOLOGY
(Autonomous)

SYLLABUS

Gokaraju Rangaraju Institute of Engineering and Technology

Micro Controller and Internet of Things Lab

Course code:GR24A3095

L:0 T:0 P:4 C:2

Course Outcomes:

1. Analyze the different blocks involved in an IoT eco-system.
2. Summarize interface techniques to connect different sensors to a micro controller.
3. Examine different communication protocols used in IoT.
4. Apply cloud environment for IoT.
5. Apply the concepts learnt to implement IoT projects.

Task 1

Write an ARDUINO Program for Smart street light management system.

Task 2

Write an ARDUINO Program for Smart reserve break monitoring system.

Task 3

Write an ARDUINO Program for Smart soil state monitoring system.

Task 4

Write an ARDUINO Program for Microcontroller-Activator Interface using DC Motor.

Task 5

Write an ARDUINO Program for Moving curtail display using LCD.

Task 6

Write an ARDUINO Program for sound Sensor.

Task 7

Write an ARDUINO Program to Develop a Mobile App for simple interface.

a) Location Sensor

b) Text to speech converter

Task 8

Write an ARDUINO program for sensor/Activator Interfacing using ESP32 using MPU 6050.

Task 9

Implement Internet enabled remote range indicator.

Task 10

Implement Internet enabled IR Sensor.

Task 11

Implement Internet enabled smart garden maintenance.

Task 12

Implement Internet Enabled Display of Ambient parameter.

Task 13

Implement Internet Enabled Home Safety and security system.

Task 14

Implement Internet Enabled Ambient light proximity digital and guest gesture sensor (APDS-9960).

Task 15

Implement Internet Enabled PIR Sensor.

TEXT BOOKS:

1. Embedded Controllers using C and Arduino/2E by Jmes M.Fiore
2. Arshdeep Bahga, Vijay Madisetti "Internet of Things(A hands on approach)" 1ST edition, VPIpublications, 2014
3. Embedded Controllers using C and Arduino/2E by Jmes M.Fiore

REFERENCES:

1. Simon Monk, "Programming the Raspberry Pi™™ Getting Started with Python", McGraw-Hill Publications.
2.Muhammad Ali Mazidi, Sepher Naimi, Sarmad Naimi, "The AVR Microcontroller and Embedded Systems using assembly and C", Second Edition, Pearson Education.
3.Website:https://medium.com/firebase-developers/getting-started-with-esp32-and-firebase-le7f19f63401

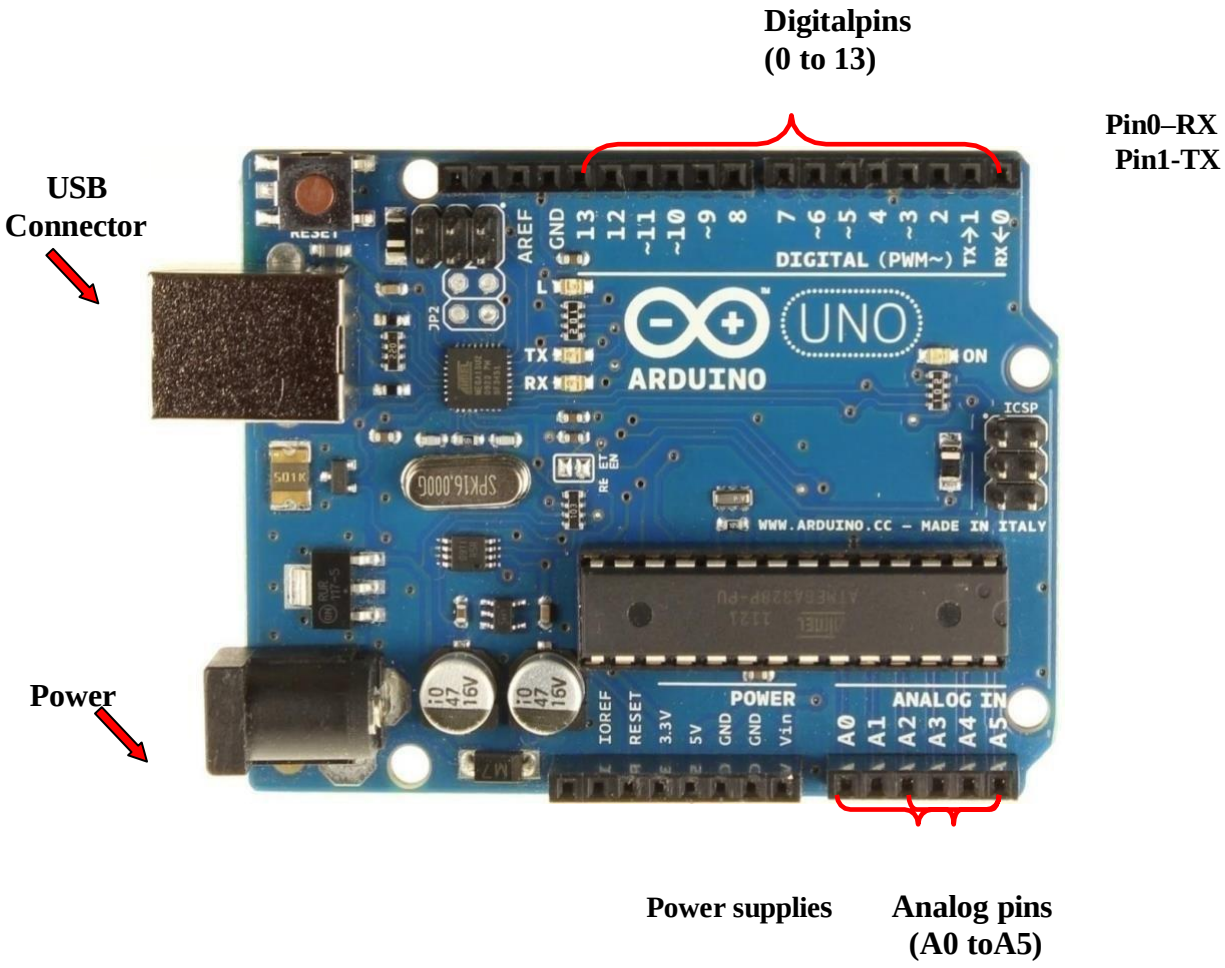
INDEX

S.No	Tasks	Page No.
1	Write an ARDUINO Program for Smart street light management system.	10
2	Write an ARDUINO Program for Smart reserve break monitoring system.	13
3	Write an ARDUINO Program for Smart soil state monitoring system.	15
4	Write an ARDUINO Program for Microcontroller-Activator Interface using DC Motor.	18
5	Write an ARDUINO Program for Moving curtail display using LCD.	21
6	Write an ARDUINO Program for sound Sensor.	25
7	Write an ARDUINO Program to Develop a Mobile App for simple interface. c) Location Sensor d) Text to speech converter	27
8	Write an ARDUINO program for sensor/Activator Interfacing using ESP32 using MPU 6050.	52
9	Implement Internet enabled remote range indicator.	55
10	Implement Internet enabled IR Sensor.	59
11	Implement Internet enabled smart garden maintenance.	63
12	Implement Internet Enabled Display of Ambient parameter.	68

13	Implement Internet Enabled Home Safety and security system.	71
14	Implement Internet Enabled Ambient light proximity digital and guest gesture sensor (APDS-9960).	74
15	Implement Internet Enabled PIR Sensor.	78

Introduction to Arduino

The Arduino Uno is a microcontroller board based on the ATmega328. It has 14 digital input/output pins (of which 6 can be used as PWM outputs), 6 analog inputs, a 16 MHz ceramic resonator, a USB connection, a power jack, an ICSP header, and a reset button. It contains everything needed to support the microcontroller. Simply connect it to a computer with a USB cable or power it with a AC-to-DC adapter or battery to get started.



Features of Uno board

Microcontroller	ATmega328
Operating Voltage	5V
Input Voltage(recommended)	7-12V
Input Voltage(limits)	6-20V
Digital I/O Pins	14(ofwhich6providePWM output)
Analog Input Pins	6
DC Current per I/O Pin	40 Ma
DC Current for 3.3VPin	50 mA
Flash Memory	32KB(ATmega328)ofwhich0.5KBusedby boot loader
SRAM	2 KB(ATmega328)
EEPROM	1 KB(ATmega328)
Clock Speed	16 MHz

Power

- The Arduino Uno can be powered via the USB connection or with an external power supply. The power source is selected automatically.
- External (non-USB) power can come either from an AC-to-DC adapter (wall-wart) or battery. The adapter can be connected by plugging a 2.1mm center-positive plug into the board's power jack. Leads from a battery can be inserted in the Gnd and Vin pin headers of the POWER connector.
- The board can operate on an external supply of 6 to 20 volts. If supplied with less than 7V, however, the 5V pin may supply less than five volts and the board may be unstable.

If using more than 12V, the voltage regulator may overheat and damage the board. The recommended range is 7 to 12 volts.

The power pins areas follows:

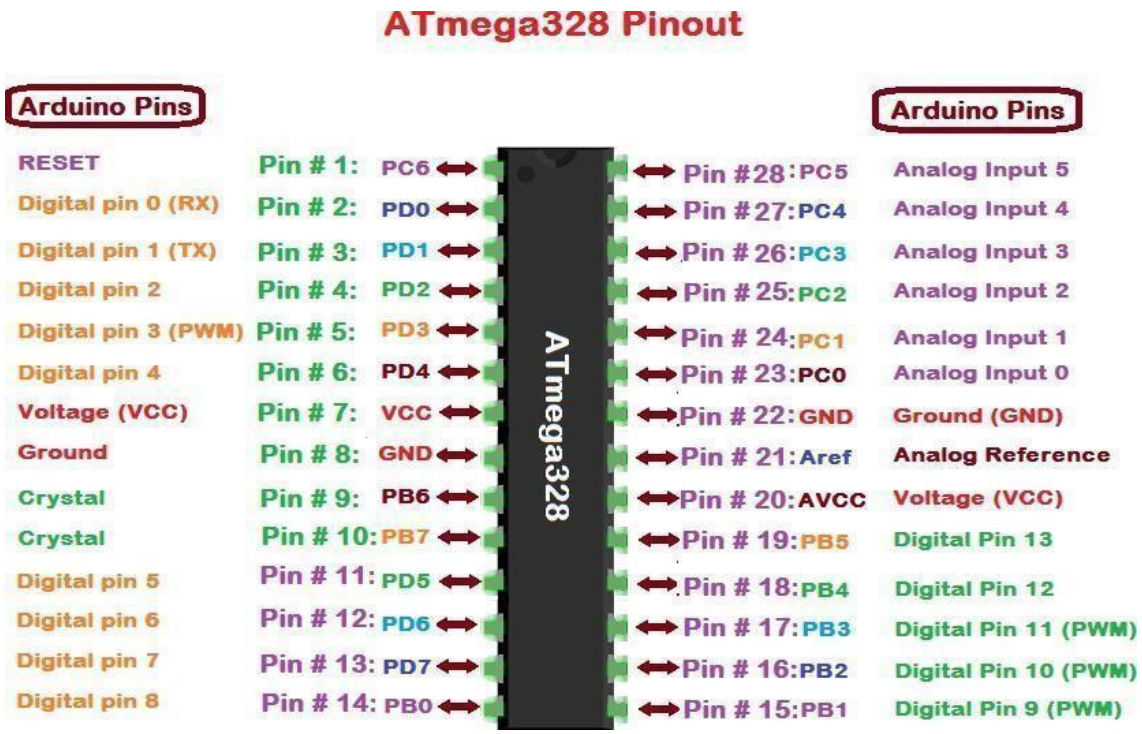
- **VIN.** The input voltage to the Arduino board when it's using an external power source (as opposed to 5 volts from the USB connection or other regulated power source). You can supply voltage through this pin, or, if supplying voltage via the power jack, access it through this pin.
- **5V.**This pin outputs a regulated 5V from the regulator on the board. The board can be supplied with power either from the DC power jack (7 - 12V), the USB connector (5V), or the VIN pin of the board (7-12V).
- **3V3.**A3.3voltsupply generated by the on-board regulator. Maximum current drawis50 mA.
- **GND.** Ground pins.

Each of the 14 digital pins (pins 0 to 13) on the Uno can be used as an input or output, using pinMode(), digitalWrite(), and digitalRead() functions. They operate at 5 volts. Each pin can provide or receive a maximum of 40 mA and has an internal pull-up resistor (disconnected by default) . In addition, some pins have specialized functions:

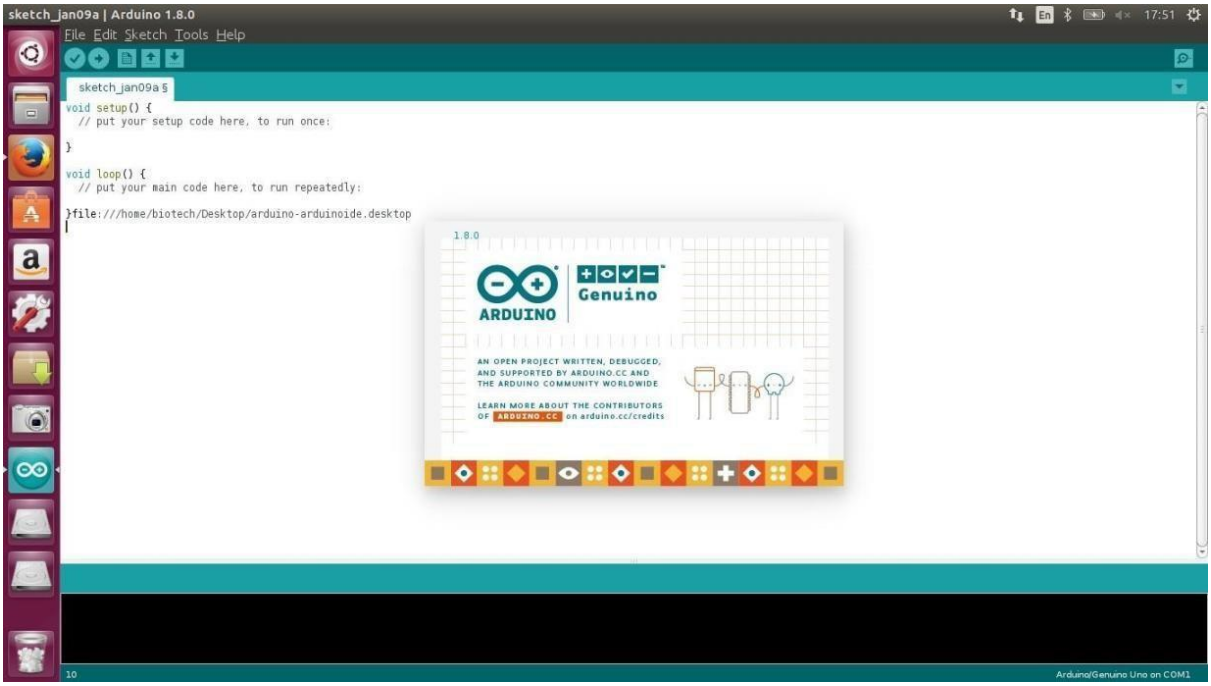
- **Serial:0 (RX) and 1 (TX).**Used to receive (RX) and transmit (TX) TTL serial data. These pins are connected to the corresponding pins of the ATmega8U2USB-to-TTL Serial chip.
- **External Interrupts: 2 and 3.** These pins can be configured to trigger an interrupt on a low value, a rising or falling edge, or a change in value.
- **PWM:3, 5, 6, 9, 10 and 11.**Provide 8-bit PWM output with the analogWrite() function.
- **SPI:10(SS), 11(MOSI), 12(MISO), 13(SCK).**These pins support SPI communication using the SPI library.
- **LED:13.**There is a built-in LED connected to digital pin 13.When the pin is HIGH value, the LED is on, when the pin is LOW, it's off.

The Uno has 6 analog inputs, labeled A0 through A5, each of which provide 10 bits of resolution (i.e. 1024 different values). By default they measure from ground to 5 volts, though it is possible to change the upper end of their range using the AREF pin and the analogReference() function.

ATmega328 - Arduino Pin Mapping:



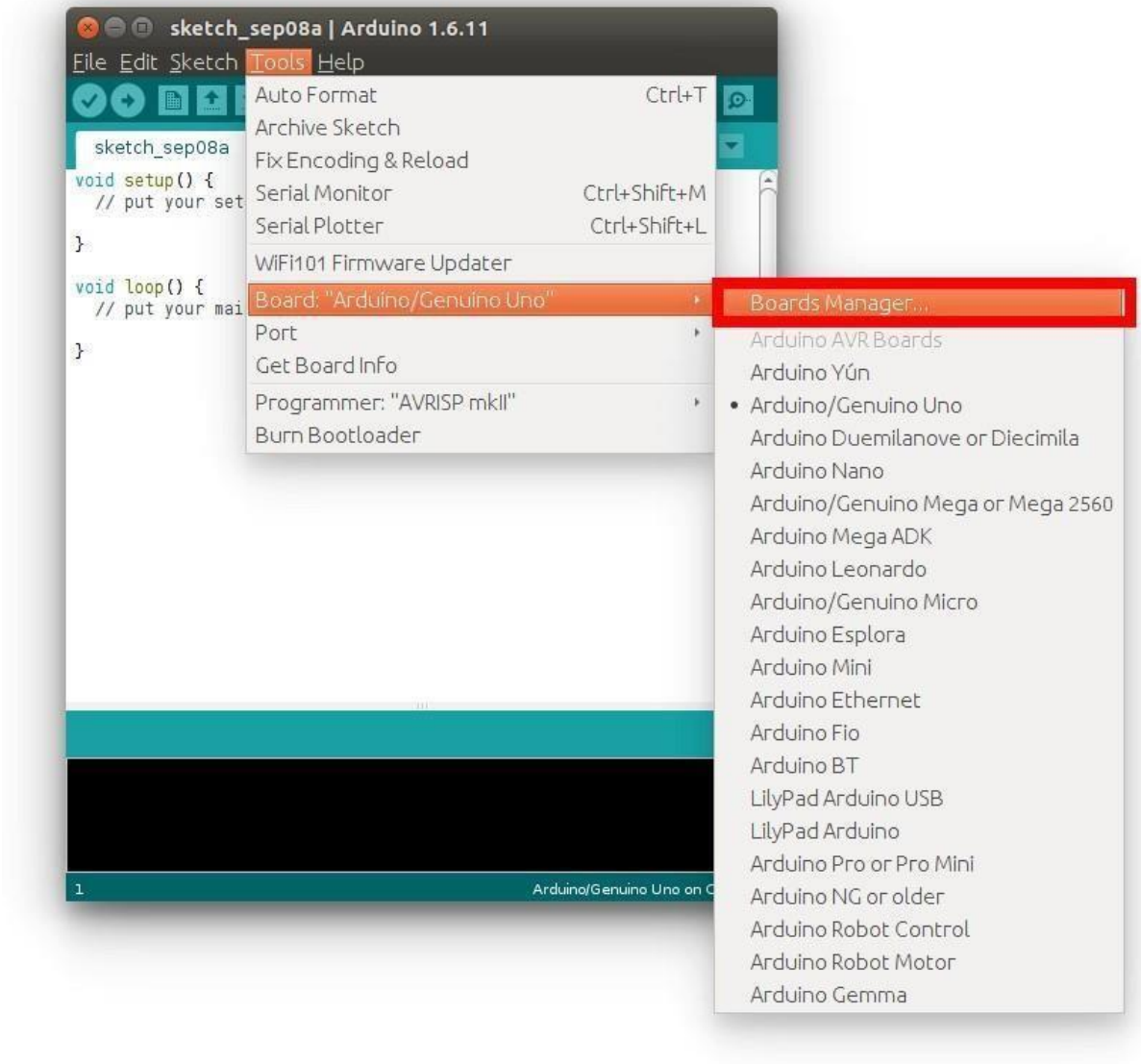
Arduino IDE



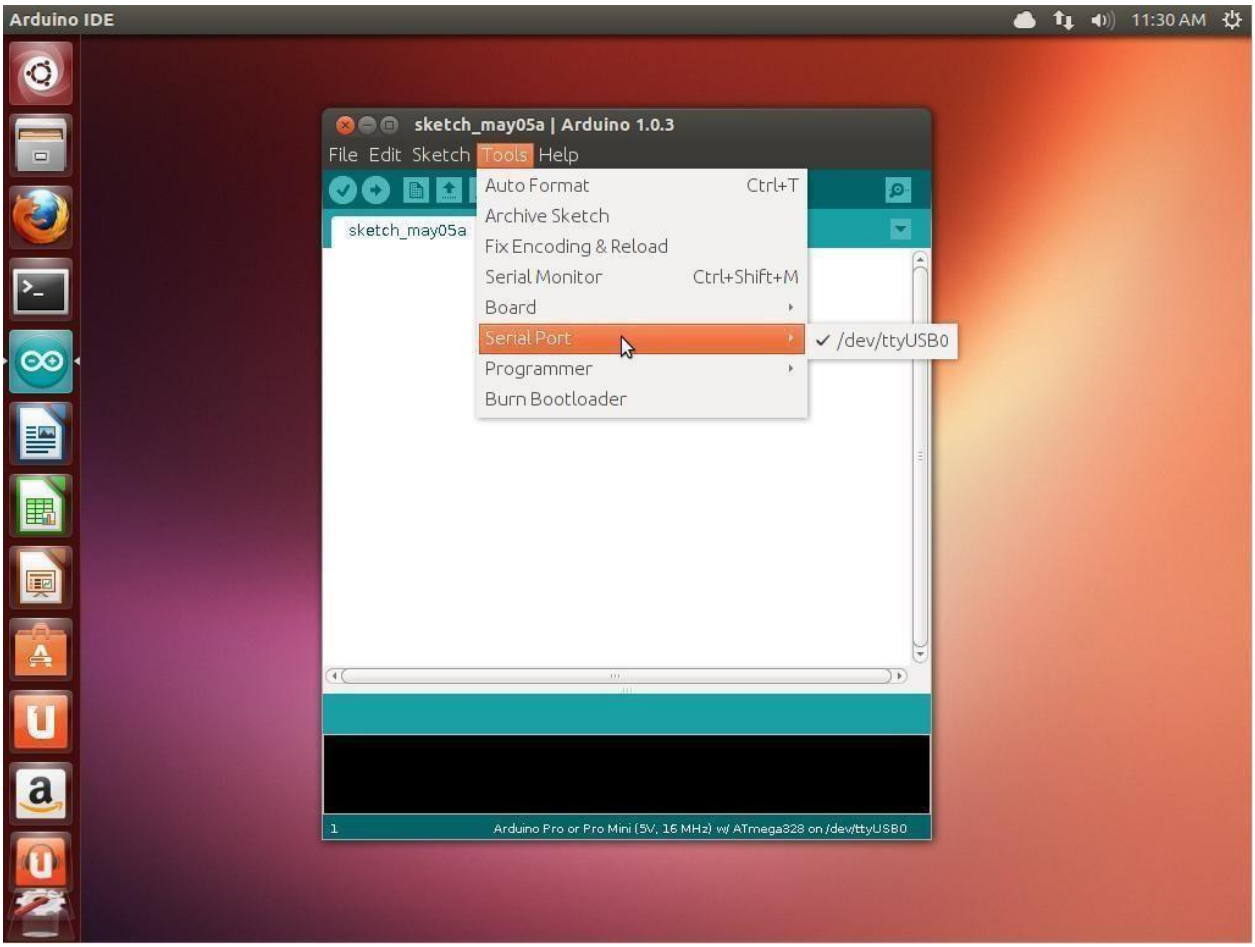
The programs written for Arduino are called sketches. For the sketch to work on the Arduino Uno, there are two hardware related settings need to be done in the Arduino IDE

- Board
- Serial Port

For selecting the board, go to the Tools tab and select Board. From the menu select Uno.



- When you connect your Arduino Uno to the USB port of your laptop, it will be mapped as a serial port.
- In the Arduino IDE, select the Serial Port as the port to which the Arduino is mapped.



- The basic structure of the Arduino sketch is fairly simple and has two required functions:
- void setup()

  {

  Statements;

  }

  void loop()

  {

  Statements;

  }
- **setup()** is the preparation, **loop()** is the execution. Both functions are required for the program to work.
- The set up function should follow the declaration of any variables at the very beginning

of the program.

- It is the first function to run in the program, is run only once, and is used to set pinMode or initialize serial communication.
- The loop function follows next and includes the code to be executed continuously – reading inputs, triggering outputs, etc. This function is the core of all Arduino programs and does the bulk of the work.
- **setup():** The setup() function is called once when your program starts. Use it to initialize pin modes, or begin serial. It must be included in a program even if there are no statements to run.
- void setup()

  {

  pinMode(pin,OUTPUT);//sets the 'pin' as output

  }
- **loop():**After calling the setup() function, the loop() function does precisely what its name suggests, and loops consecutively, allowing the program to change, respond, and control the Arduino board.
- void loop()

  {

  digitalWrite(pin, HIGH); // turns 'pin' on delay(1000); //pauses for one second
  digitalWrite(pin, LOW); // turns 'pin' off delay(1000); // pauses for one second

  }
- **pinMode(pin, mode):** Used in void setup() to configure a specified pin to behave either as an INPUT or an OUTPUT.
- pinMode(pin, OUTPUT);//sets ‘pin’ to output
- pinMode(pin, INPUT);//set ‘pin’ to input
- Pins configured as OUTPUT can provide 40 mA (milliamps) of current to other devices/circuits. This is enough current to brightly light up an LED (don't forget the

series resistor),but not enough current to run most relays, solenoids, or motors.

- Short circuits on Arduino pins and excessive current can damage or destroy the output pin, or damage the entire Atmega chip. It is often a good idea to connect an OUTPUT pin to an external device in series with a 470Ω or 1KΩ resistor.
- **digitalRead(pin):**Reads the value from a specified digital pin with the result either HIGH or LOW. The pin can be specified as either a variable or constant (0-13).
- value = digitalRead(Pin); //sets 'value' equal to the input pin.
- **digitalWrite(pin, value):** Outputs either logic level HIGH or LOW at(turns on or off) a specified digital pin. The pin can be specified as either a variable or constant (0-13).
- digitalWrite(pin, HIGH); //sets 'pin' to high.
- **analogRead(pin):** Reads the value from a specified analog pin with a 10-bit resolution. This function only works on the analog in pins (0-5). The resulting integer values range from 0 to 1023.
- value = analogRead(pin); //sets 'value' equal to 'pin'.
- **analogWrite(pin, value):** Writes a pseudo-analog value using hardware enabled pulse width modulation (PWM) to an output pin marked PWM. On Uno, this function works on pins 3, 5, 6, 9, 10, and 11. The value can be specified as a variable or constant with a value from 0-255.
- analogWrite(pin, value); //writes 'value' to analog 'pin'.
- **delay(ms):** Pauses a program for the amount of time as specified in milliseconds, where 1000 equals 1 second.
- delay(1000);//waits for one second.
- **Serial.begin(baud rate):** Opens serial port and sets the baud rate for serial data transmission. The typical baud rate for communicating with the computer is 9600 although other speeds are supported.
- void setup()

  {

Serial.begin(9600);//opens serial port

} //sets data rate to 9600 bps.

- **Serial.println(data):** Prints data to the serial port, followed by an automatic carriage return and line feed. This command takes the same form as Serial.print(), but is easier for reading data on the Serial Monitor.
- Serial.println(analogValue); //sends the analog Value'

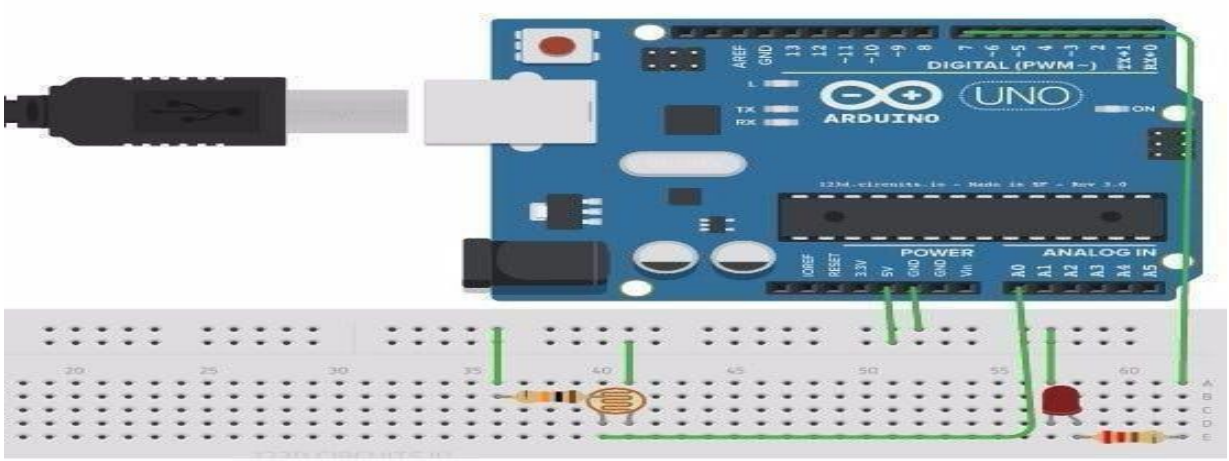
Task 1

Light Dependent Resistor

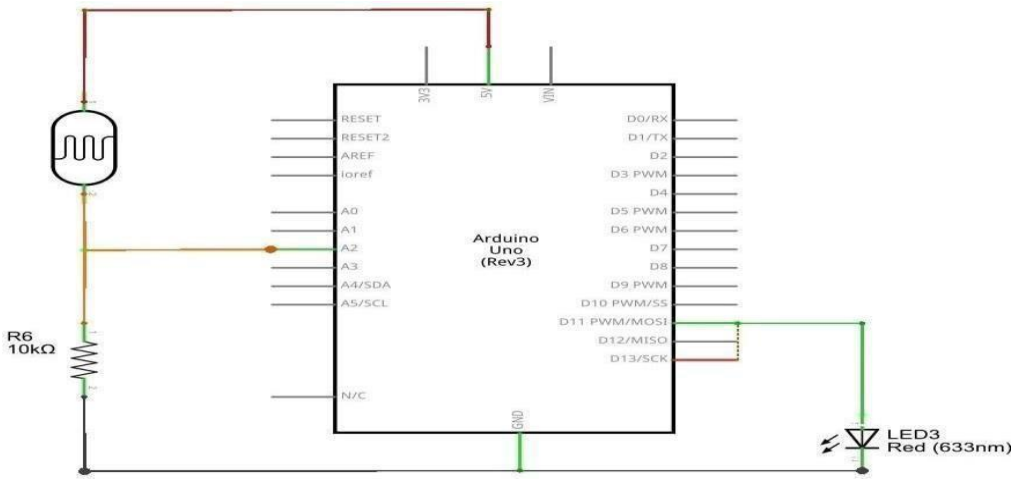
Aim: Write an ARDUINO Program for Smart street light management system.

Components Required: Arduino UNO, LED , LDR, connecting wires, breadboard, Computer system with Arduino IDE.

Circuit:



Schematic Diagram:



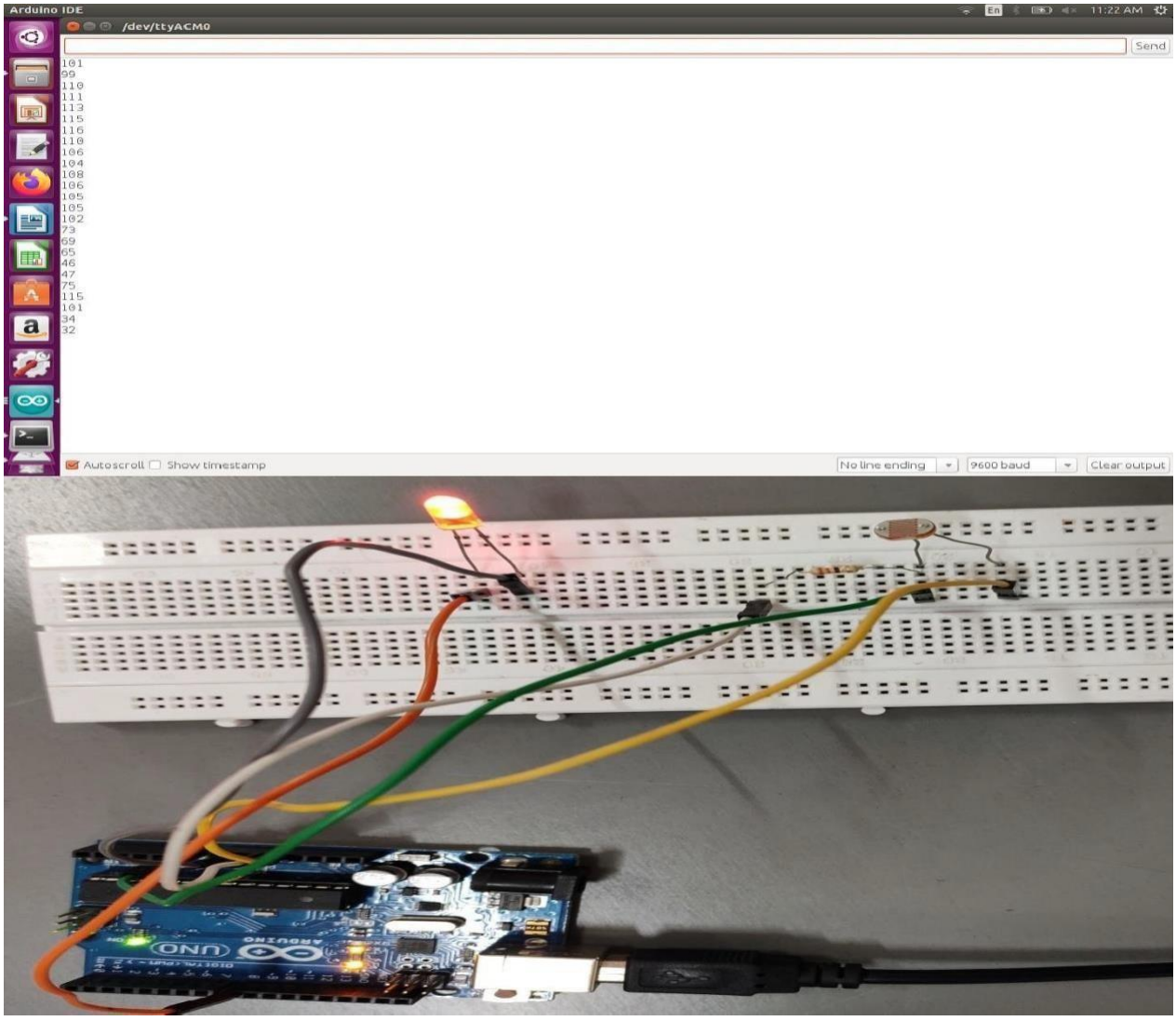
LDR is Light Dependent Resistor. LDRs are made from semiconductor materials to enable them to have their light-sensitive properties. These LDRs or PHOTO RESISTORS works on the principle of “Photo Conductivity”. Now what this principle says is, whenever light falls on the surface of the LDR (in this case) the conductance of the element increases or in other words, the resistance of the LDR falls when the light falls on the surface of the LDR. This property of the decrease in resistance for the LDR is achieved because it is a property of semiconductor material used on the surface.

Program:

```
void setup() {
pinMode(A0,INPUT);
Serial.begin(9600);
pinMode(8,OUTPUT);
}
void loop(){
//Serial.print("LDR=");

int i=analogRead(A0);
Serial.println(i);
if(i>60)
digitalWrite(8,1);
else
digitalWrite(8,0);
delay(500);
}
```

Output:


Arduino IDE
11:22 AM
/dev/ttyACM0
Send
101
99
110
111
113
115
116
110
106
104
108
106
105
105
102
73
69
65
46
47
75
115
101
34
32
Autoscroll
Show timestamp
No line ending
9600 baud
Clear output
UNO
ARDUINO

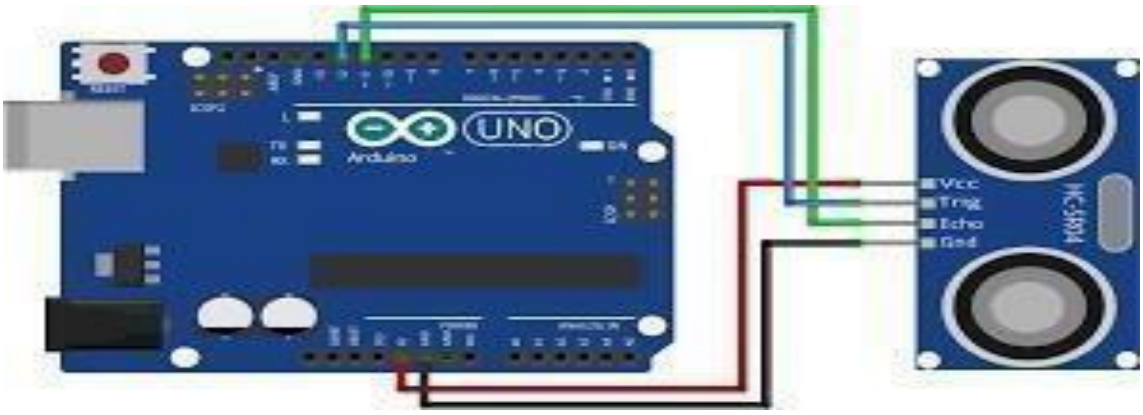
Task 2

Ultrasonic Sensor

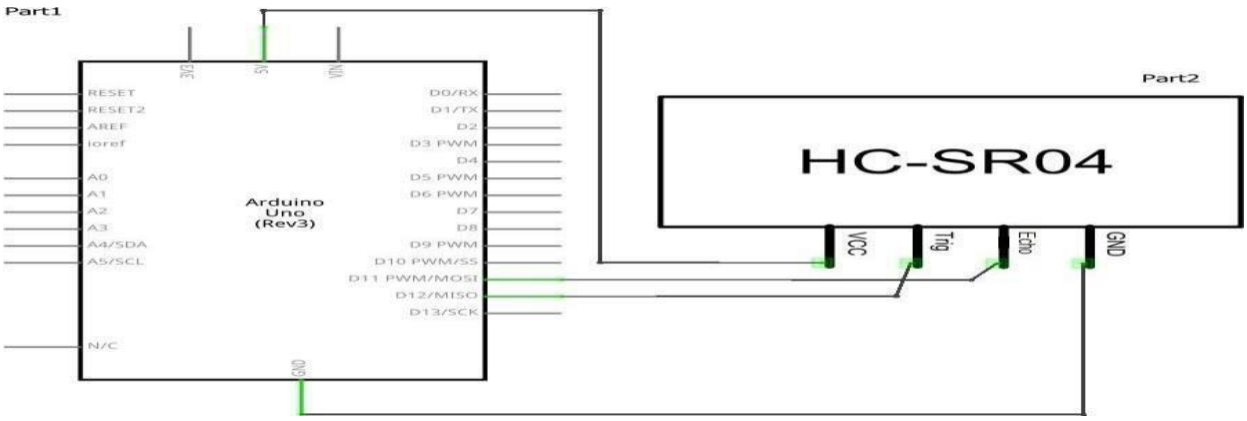
Aim: To Write an Arduino program for measuring distance using ultrasonic sensor.

Components Required: Arduino UNO, Ultrasonic sensor, connecting wires, breadboard, Computer system with Arduino IDE.

Circuit:



Schematic Diagram:



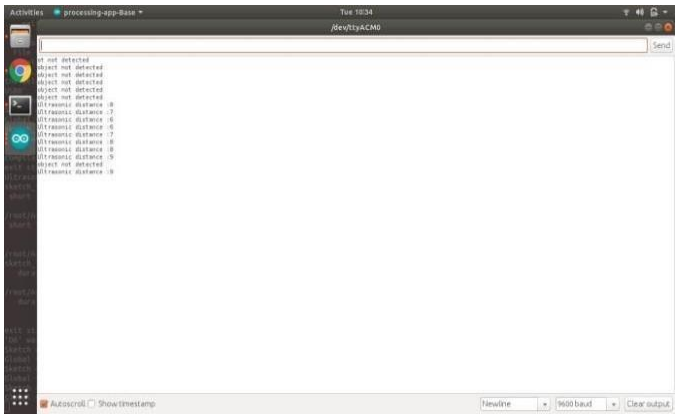
The HC-SR04 ultrasonic sensor uses SONAR to determine the distance of an object just like the bats do. It offers excellent non-contact range detection with high accuracy and stable readings in an easy-to-use package from 2 cm to 400 cm or 1” to 13 feet.

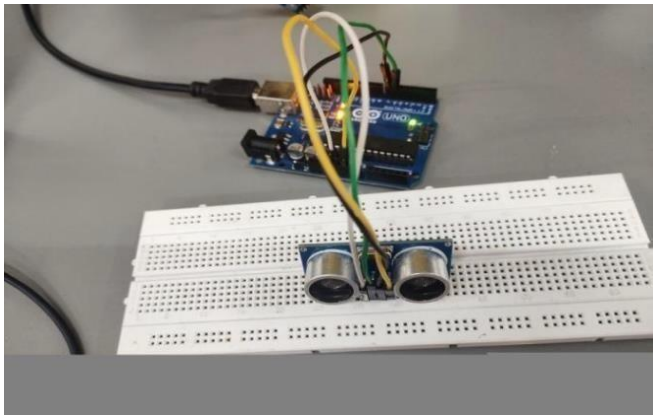
The operation is not affected by sunlight or black material, although acoustically, soft materials like cloth can be difficult to detect. It comes complete with ultrasonic transmitter and receiver module.

Program:

```
#include<Ultrasonic.h>
 Ultrasonic ultrasonic(11,12);//Init an Ultrasonic object
int Distance;
void setup() {
Serial.begin(9600);
}
void loop()
{
 Distance=ultrasonic.read();//getthecurrentresult;
 delay(100);
 Serial.print("the distance is");
 Serial.println(Distance);
 delay(1000);
}
```

Output:





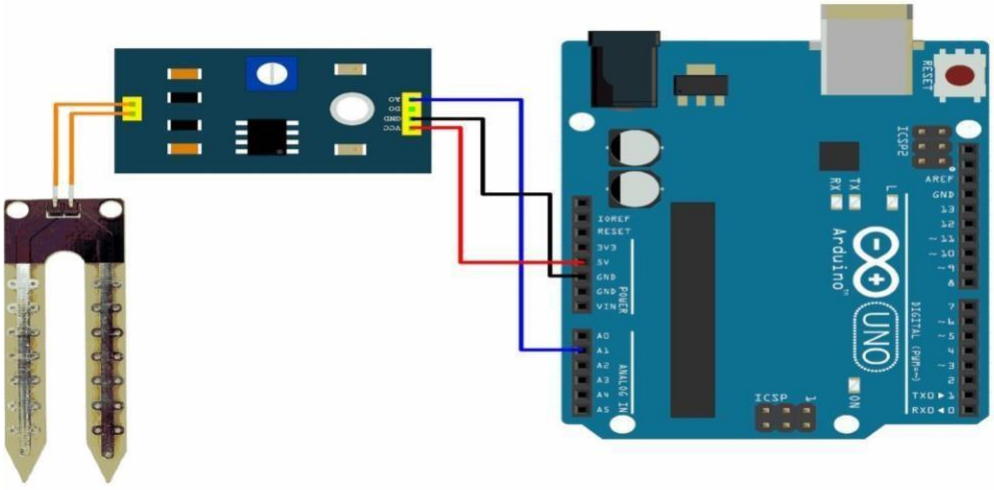
Task 3

Soil Moisture Sensor

Aim: To write an Arduino program for measuring wetness of soil using soil moisture sensor

Components Required: Arduino UNO, soil moisture sensor, connecting wires, breadboard, Computer system with Arduino IDE.

Circuit:



Soil moisture is basically the content of water present in the soil. This can be measured using a soil moisture sensor which consists of two conducting probes that act as a probe. It can measure the moisture content in the soil based on the change in resistance between the two conducting plates.

The resistance between the two conducting plates varies in an inverse manner with the amount of moisture present in the soil.

Here, the analog output of soil moisture sensor is processed using ADC. The moisture content in terms of percentage is displayed on the serial monitor.

The output of the soil moisture sensor changes in the range of ADC value from 0 to 1023. This can be represented as moisture value in terms of percentage using formula given below.

$$AnalogOutput = \frac{ADCValue}{1023}$$

Moisture in percentage=100 – (Analog output * 100)

For zero moisture, we get maximum value of 10-bit ADC, i.e.1023.This, in turn, gives 0% moisture.

Program:

```
#include<SoftwareSerial.h>

void setup() {

  //put your setup code here, to run once:

pinMode(A4,INPUT);
pinMode(13,OUTPUT);

Serial.begin(9600);

}

void loop(){

int sensorvalue = analogRead(A4);
Serial.println(sensorvalue);
if(sensorvalue<500)
{

  Serial.println("ledison");

  digitalWrite(13,HIGH);

  delay(1000);

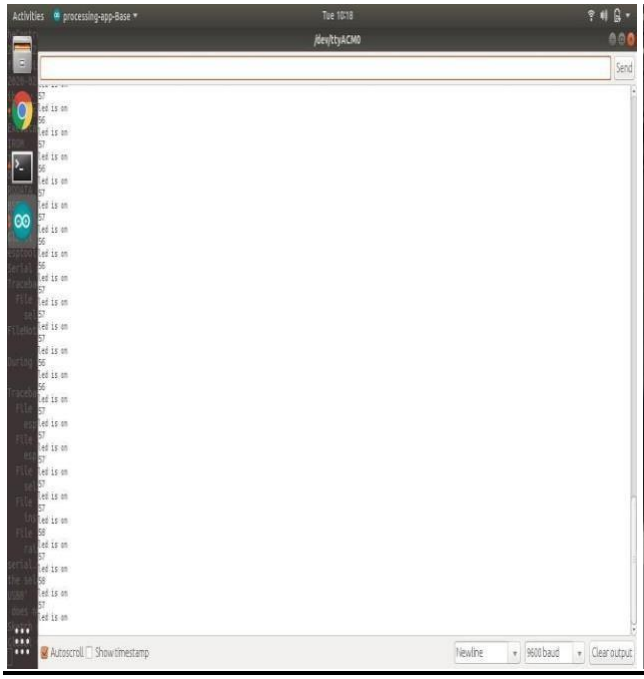
}

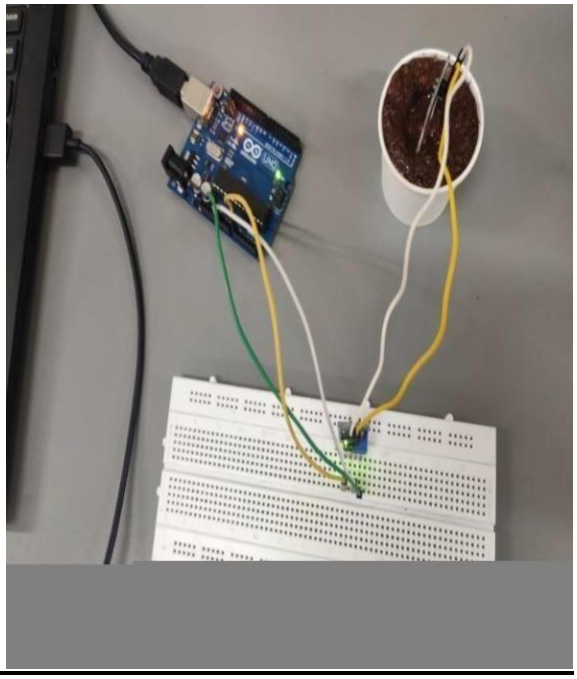
else
{
  digitalWrite(13,LOW);

  delay(1000);

}
```

Output:


Activities
processing-app-Base
Tue 10:18
/dev/ttyACM0
Send
Autoscroll
Show timestamp
Newline
9600 baud
Clear output



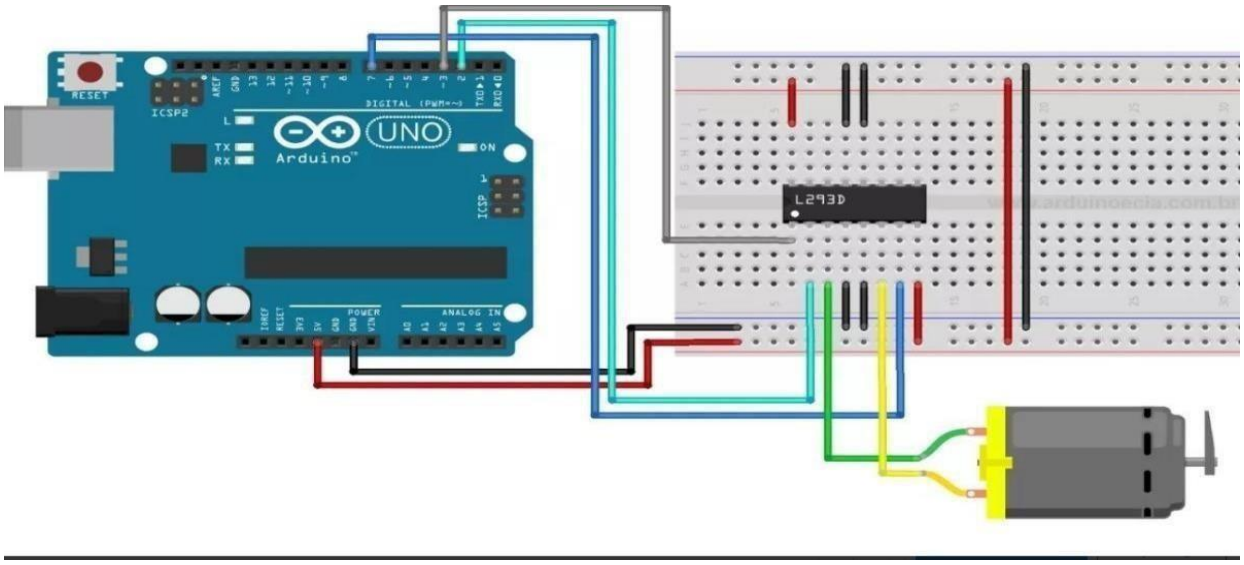
Task 4

DC Motor

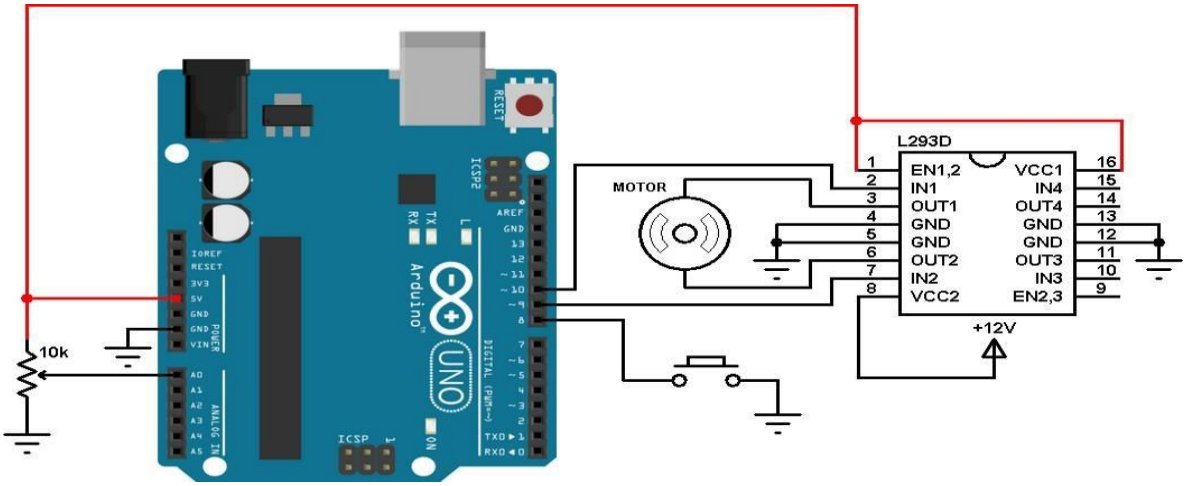
Aim: To write an Arduino program for rotating motor in clockwise and anticlockwise directions.

Components Required: Arduino UNO, DC Motor, L293DIC, connecting wires, breadboard, and Computer system with Arduino IDE.

Circuit:



Schematic Diagram:



Truth Table for Motor:

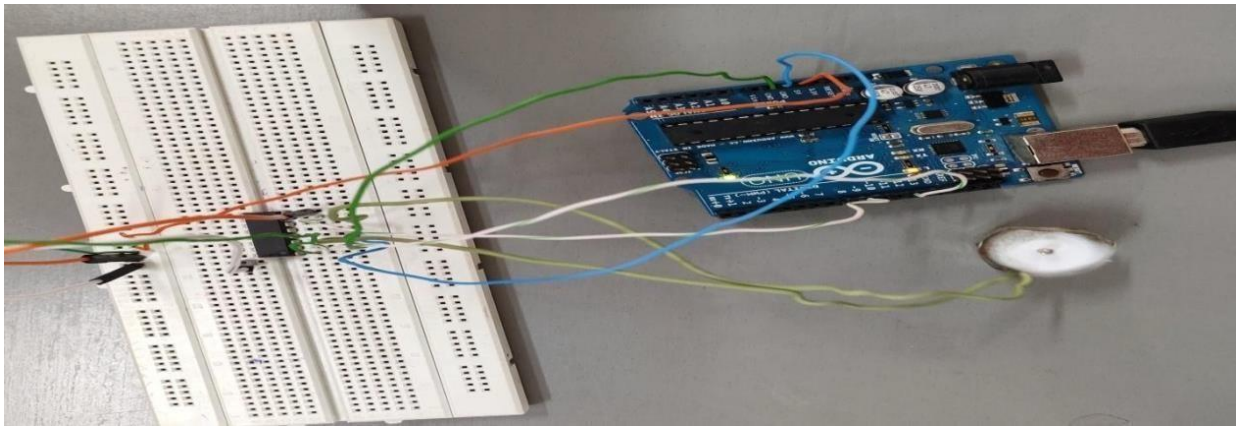
Enable	Input1	Input2	Motor action
H	H	L	Clock wise rotation
H	L	H	Counter-clock wise rotation
H	L	L	Stop
H	H	H	Stop

Program:

```
void setup(){
pinMode(10,OUTPUT);
pinMode(9, OUTPUT);
}
void loop(){
digitalWrite(10, LOW);
digitalWrite(9, HIGH);
delay(3000);
digitalWrite(9,LOW);
digitalWrite(10,HIGH);
delay(3000);
digitalWrite(9, LOW);
digitalWrite(10, LOW);
delay(3000);
}
```

Output:

Motor rotates in clockwise and anticlockwise directions and finally stops



Task 5
Liquid Crystal Display(LCD)

Aim: To write an Arduino program for moving curtail display using LCD.

Components Required: Arduino UNO, LCD, connecting wires, breadboard, and Computer system with Arduino IDE.

Introduction: LCD Shield mainly constitutes 16*2 LCD.A Liquid Crystal Display(LCD) is a flat panel display, electronic visual display or video display that uses the light modulating properties of liquid crystal



Pin-out:

LCD Pin	Arduino Pin
RS	8
EN	9
D4	4
D5	5
D6	6
D7	7

Header File:

#include<LiquidCrystal.h>

Functions

1. **LiquidCrystallcd(rs, enable, d4, d5, d6, d7):**Creates a variable of type LiquidCrystal. The display can be controlled using data lines.
2. **rs:** The number of Arduino pin that is connected to RS (register Select) pin on LCD.
3. **enable:** The number of Arduino pin that is connected to enable pin on LCD.
4. **d4,d5,d6,d7:**The number of Arduino pin that is connected to corresponding data pins on LCD.
5. **lcd.begin(cols, rows):**Specifies the dimensions of the display (usually cols = 16, rows = 2).
6. **lcd.clear():**Clears the LCD screen and positions the cursor in the upper left corner.
7. **lcd.setCursor(col, row) :**Positions the LCD cursor at the given row and column of the LCD screen.
8. **lcd.write(data):** Writes a character on to the LCD.
9. **lcd.print(data):** Prints text on the LCD.
10. **lcd.display():**Turns on the LCD display.
11. **lcd.nodisplay():** Turns off the LCD display.
12. **lcd.scrollDisplayLeft():** Scrolls the contents of the display ones pace to the left.
13. **lcd.scrollDisplayRight():**Scrolls the contents of the display one space to the right.
14. **lcd.autoscroll():**Turns on automatic scrolling of the LCD. If current text direction is left-to-right, the display scrolls to the left..If current text direction is right-to-left, the display scrolls to the right.
15. **lcd.noAutoscroll():**Turns off automatic scrolling of LCD.
16. **lcd.leftToRight() :** Sets the direction for text written to left-to-right, the default. This means that subsequent characters written to the display will go from left to right.
17. **lcd.rightToLeft() :** Sets the direction for text written to right-to-left. This means that subsequent characters written to the display will go from right to left.
18. **lcd.createChar(num, data) :**Creates a custom character for use on LCD. Num specifies the character to be overridden. data specifies the custom character array name.

Program:

```
#include<LiquidCrystal.h>
LiquidCrystallcd(8,9,4,5,6,7);
int i=0;
char ch;
chars[]="Welcome to MCLAB!";
void setup()
{
  lcd.begin(16,2);
}
void loop()
{
  lcd.setCursor(0,0);
  ch='!';
  lcd.print(ch);
  delay(1000);
  lcd.clear();
  lcd.setCursor(3,0);
  for(i=0;s[i]!='\0';i++)
  {
  lcd.print(s[i]);
  if(i==10)
  lcd.setCursor(5,1);
  }
  delay(2000);
  lcd.clear();

  }
```

Output:

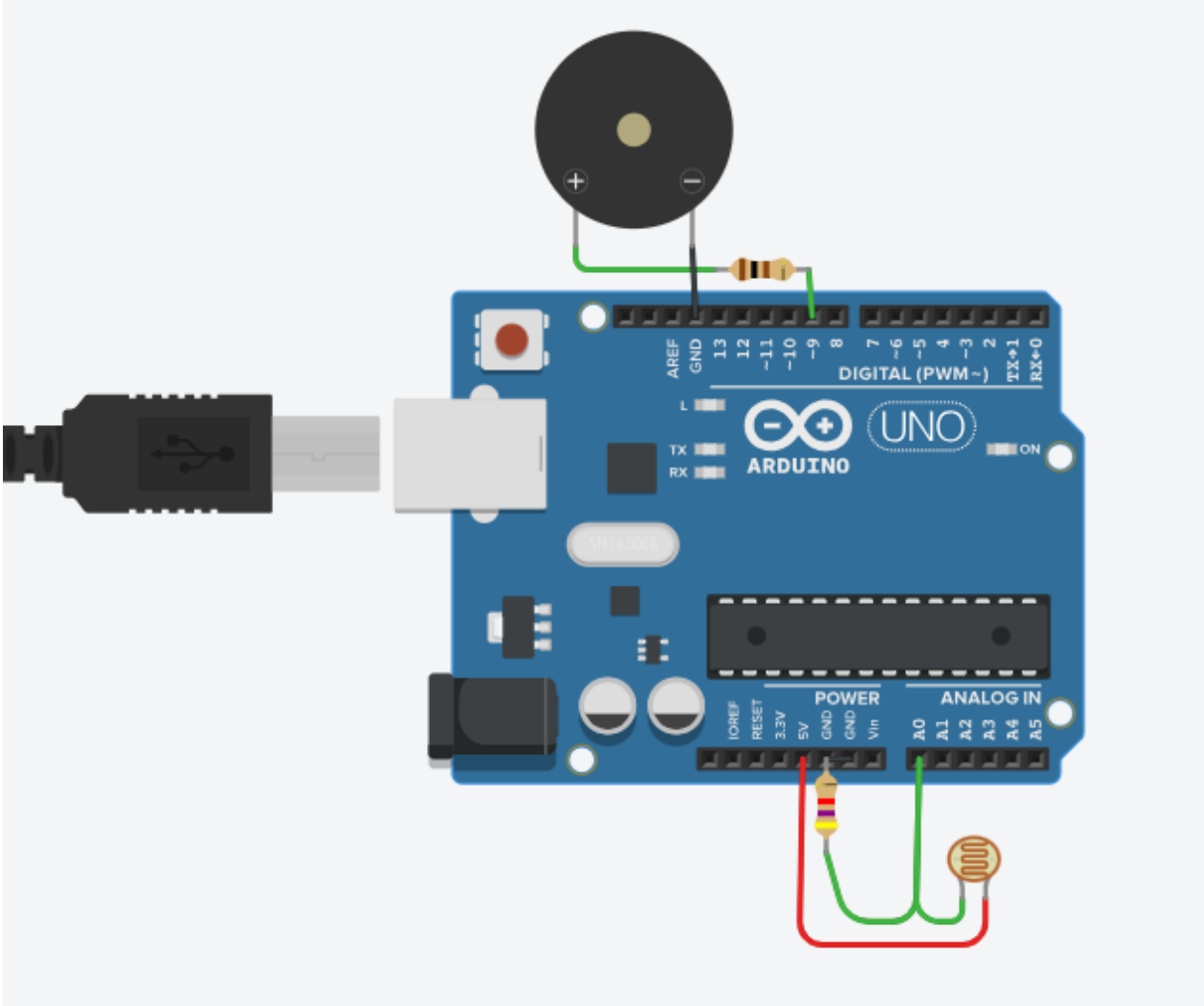

LCD KeyPad Shield
Welcome to
MC LAB!
SELECT
LEFT
RIGHT
RST
ROBOT
FUTURE

Task 6

Sound Sensor

Aim: Write an Arduino Program for Sound Sensor using ESP32.

Components Required: Arduino Board, Sound Sensor, Cable (Tinker cad Online Simulator).




```
// C++ code
//
/*
  Pitch follower

  Plays a pitch that changes based on a changing
  analog input

  circuit:
  * 8-ohm speaker on digital pin 9
  * photoresistor on analog 0 to 5V
```

```
  * 4.7K resistor on analog 0 to ground

  created 21 Jan 2010
  modified 31 May 2012  by Tom Igoe, with
  suggestion from Michael Flynn

  This example code is in the public domain.
  http://www.arduino.cc/en/Tutorial/Tone2
*/
```

Program:

```
int sensorReading = 0;

void setup()
{
  pinMode(A0, INPUT);
  Serial.begin(9600);
  pinMode(9, OUTPUT);
}
void loop()
{
  // read the sensor
  sensorReading = analogRead(A0);
  // print the sensor reading so you know its range
  Serial.println(sensorReading);
  // map the sensor reading to a range for the
  // speaker
  tone(9, 440 * pow(2.0, (constrain(int (map(sensorReading, 0, 1023, 36, 84)), 35, 127) - 57) / 12.0), 1000);
  delay(10); // Delay a little bit to improve simulation performance
}
```

Output:



Task 7

Mobile App

Aim: Write an ARDUINO Program to Develop a Mobile App for simple interface.
a) Location Sensor
b) Text to Speech Converter.

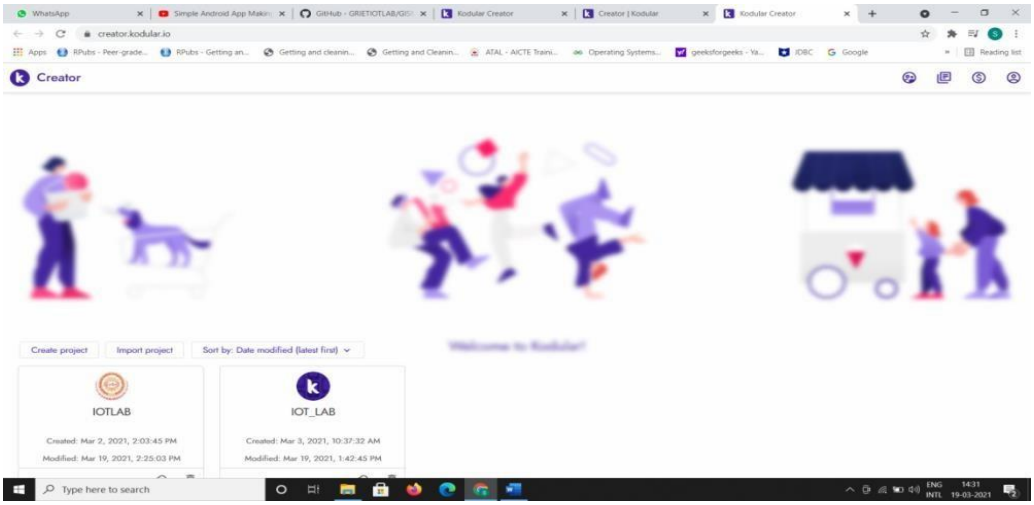
Steps to Develop a Mobile app for simple User Interface :

There are several steps to develop a mobile app. These are the following Steps:

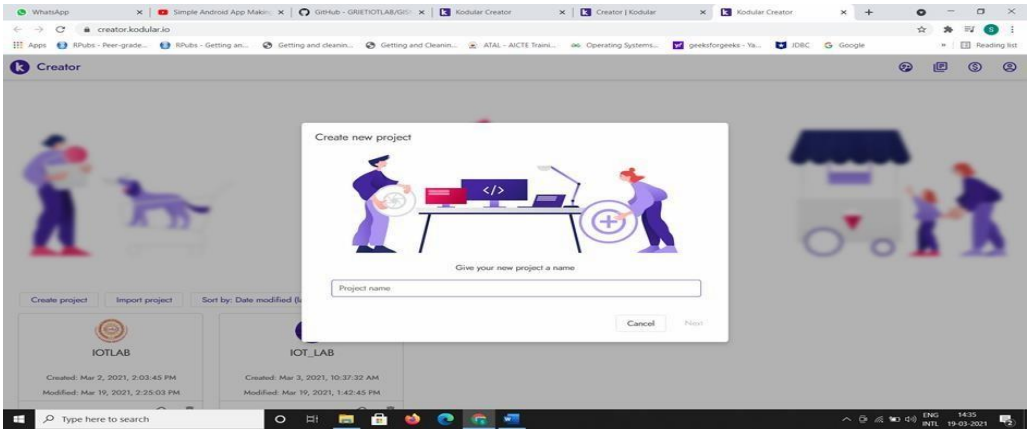
Step1: Go to Kodular Creator website by clicking below link you can go the website.
https://www.kodular.io/creator

Step2: Click on get Started button

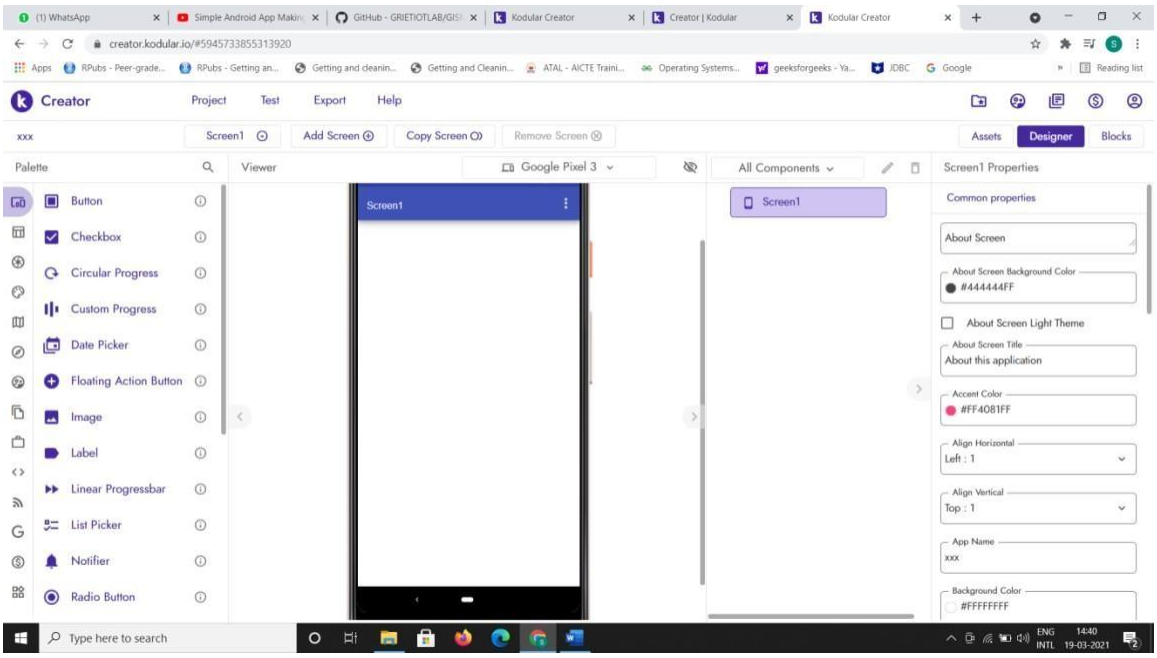
Step3: Then you will get as like this



Step4: Click on create project and give project name.



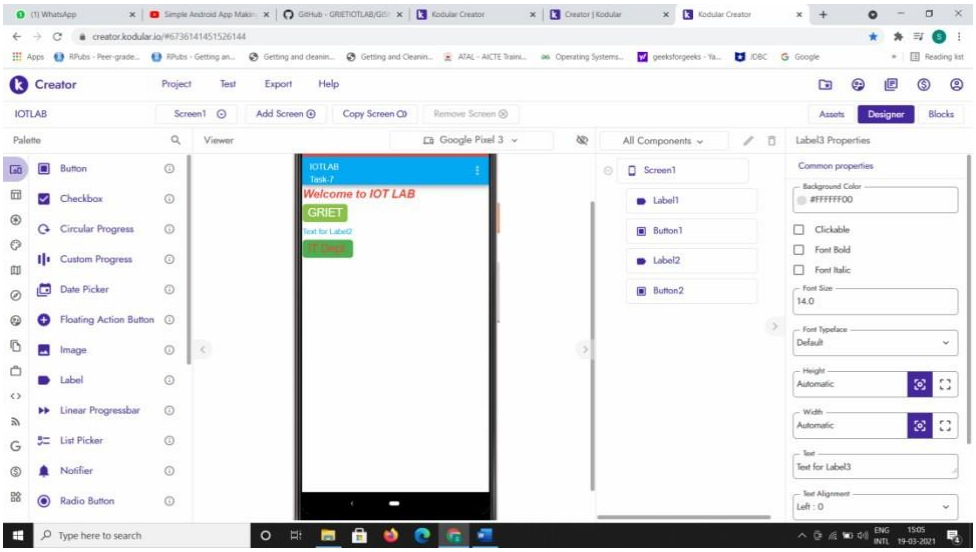
Step 5: Then click on next and finish. Then your project will be created and window will be opened like this.



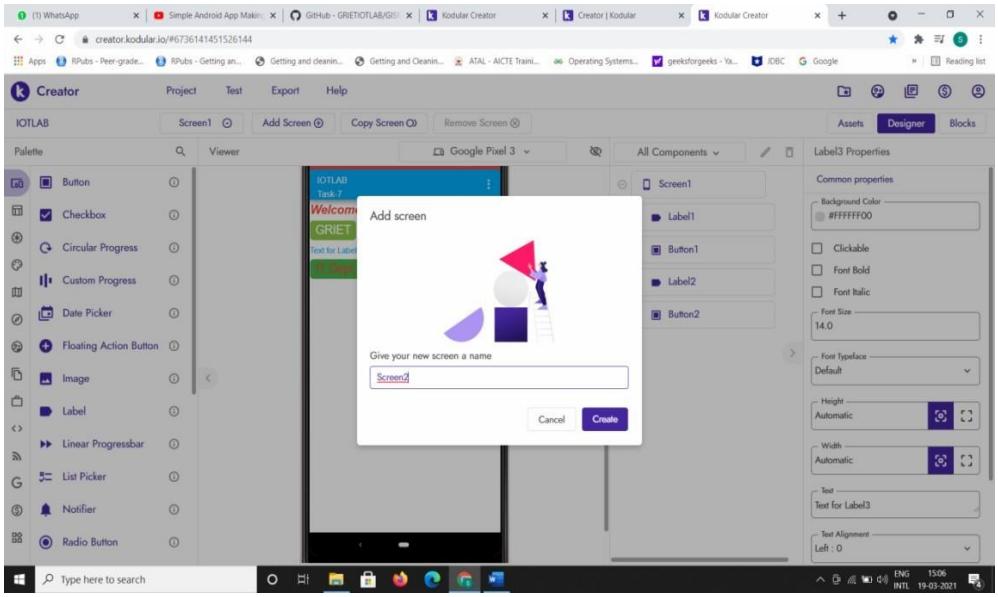
Step 6: Then go to the left side pallet and click on User Interface then you will find all GUI items like
buttons, lables, etc..as shown in the above figure.

Step 7: Drag and drop the controls which you need and those controls will be visible in your app.
Here as a sample mobile app, drag and drop on label and add the text as “Welcome to IOTLAB”
and you can apply color, font etc…

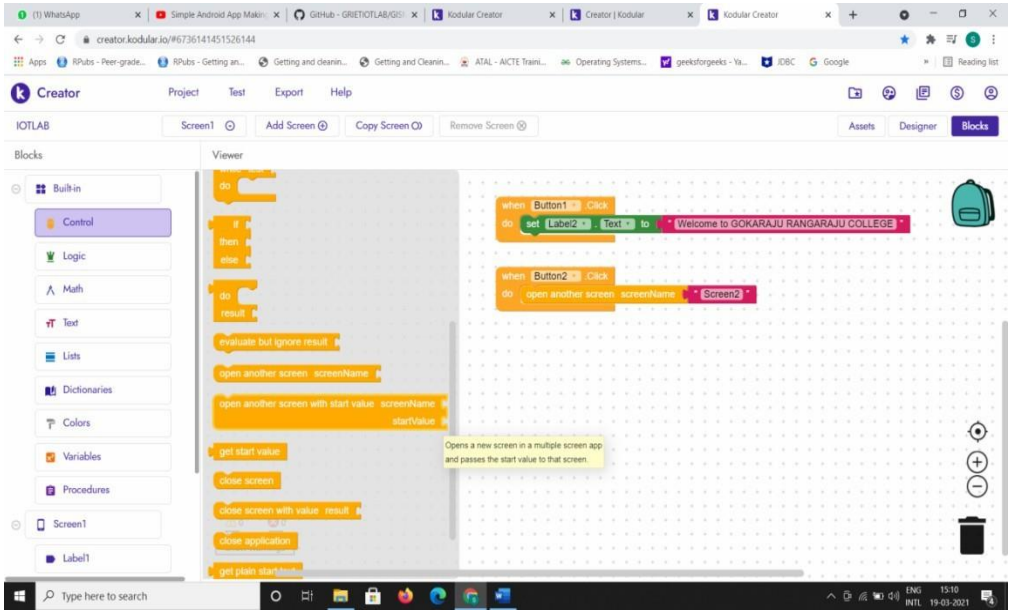
Step 8: And take 2nd button one as GRIET and the other as IT Dept and took two labels one will be printing the text when GRIET button is clicked in the same screen and other will print the text when IT Dept. Button is clicked in another screen.



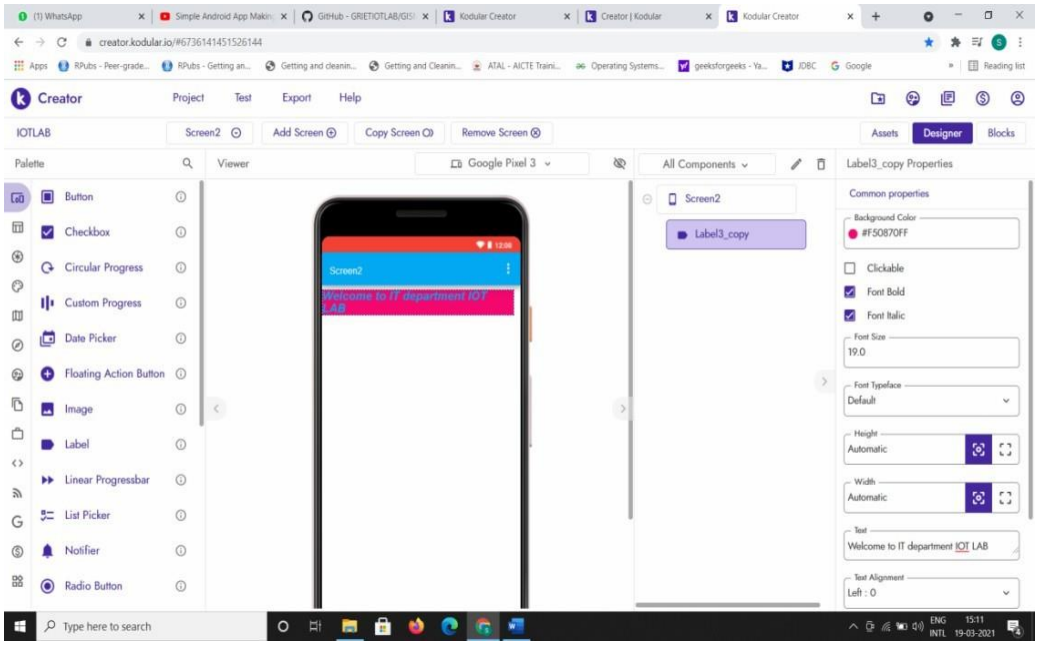
Step 9: Click on Add Screen and click on create button the new screen will be created.



Step 10: Go to screen 1 and go to Blocks tab on the right to write the code to get the text when you click the buttons accordingly.

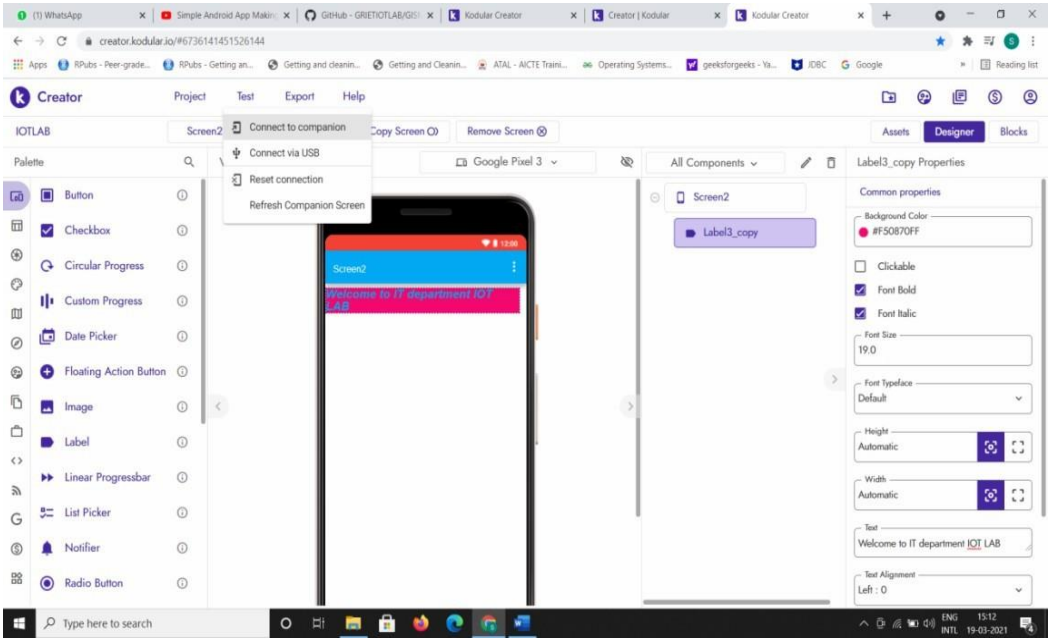


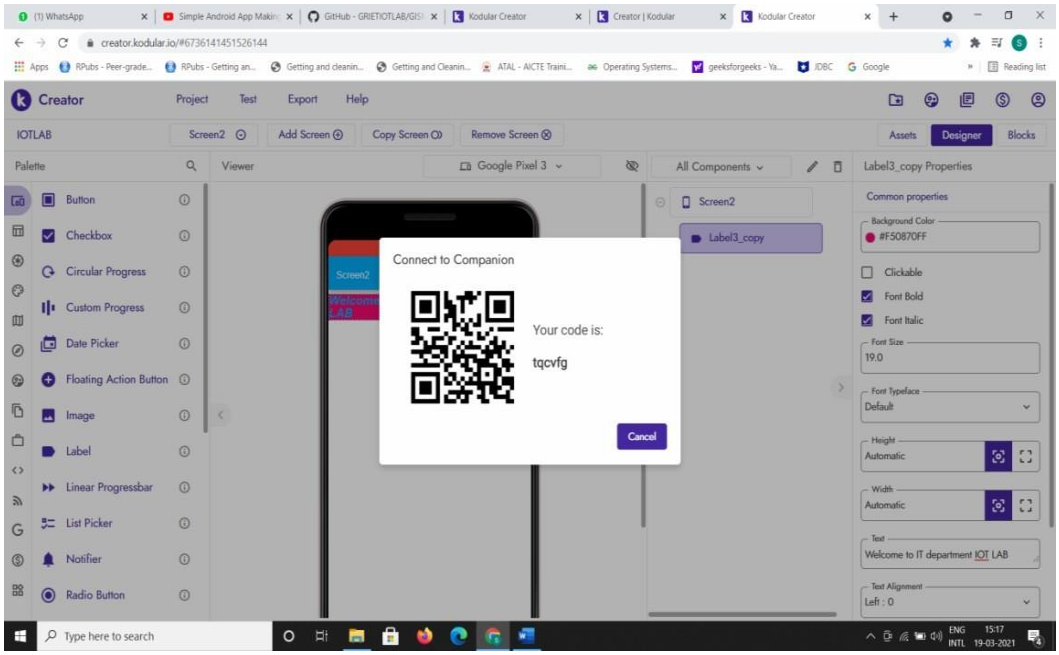
Step 11: Go to screen 2 and put a label and write the text that should be displayed when you click on IT Dept button in screen.



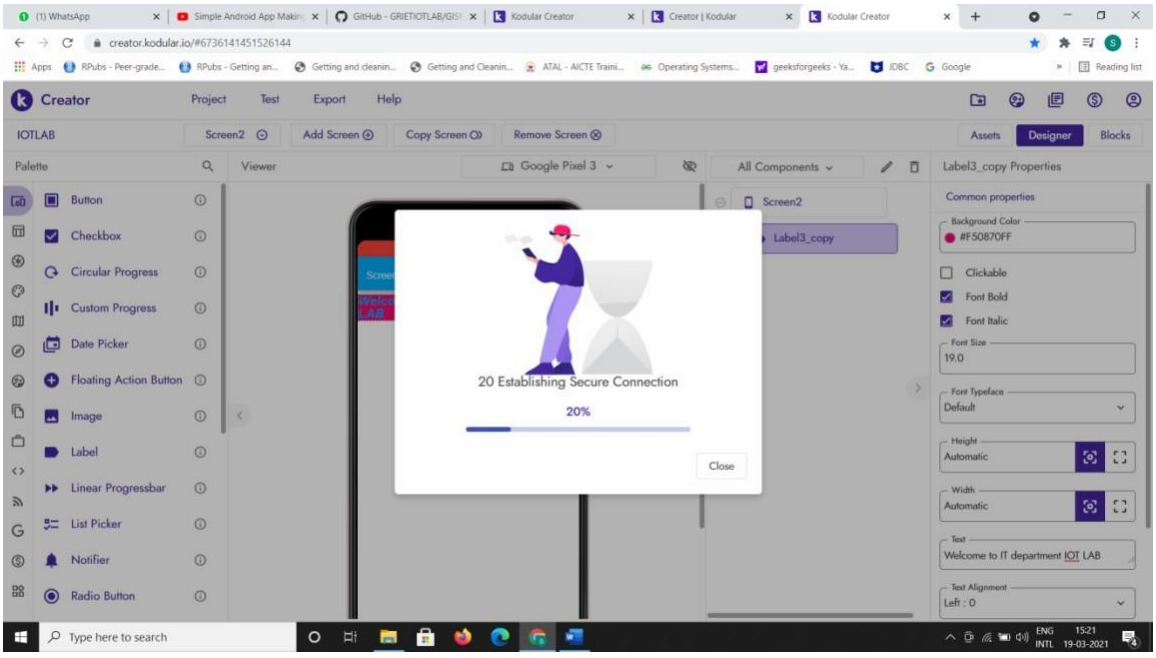
Step 12: Now test the mobile app, buy clicking test and connect to companion. Before this testing open google play store in your mobile and install the app named “Kodular Companion”.

Step 13: Then open that app and scan the QR code which we get when we click on connect to companion in Kodular Creator.

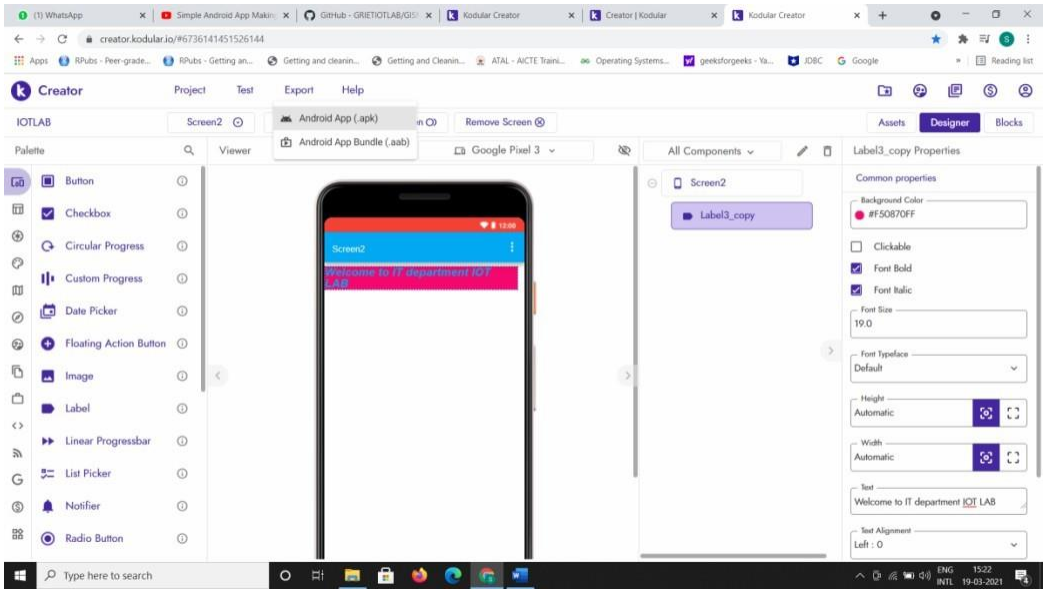


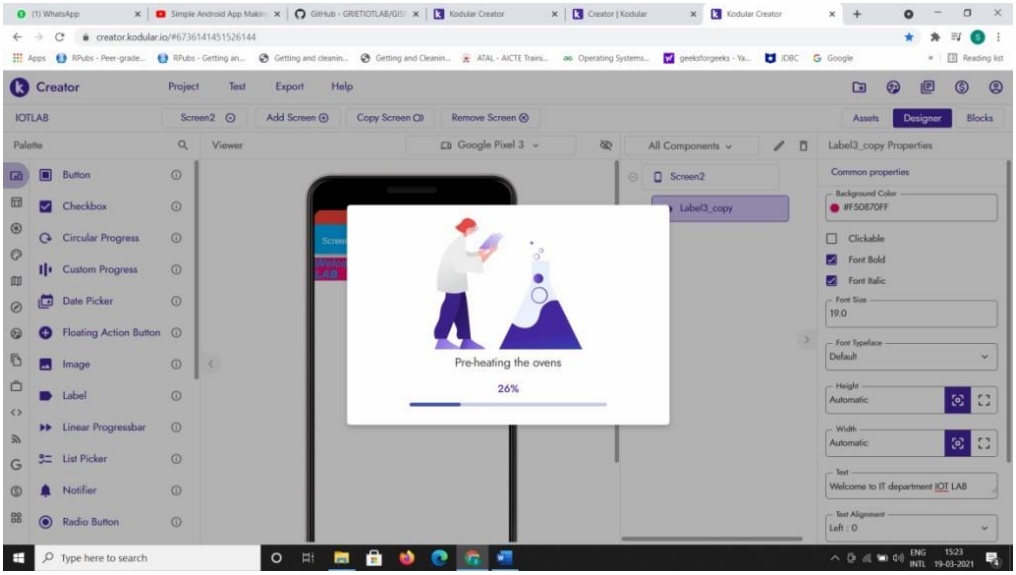


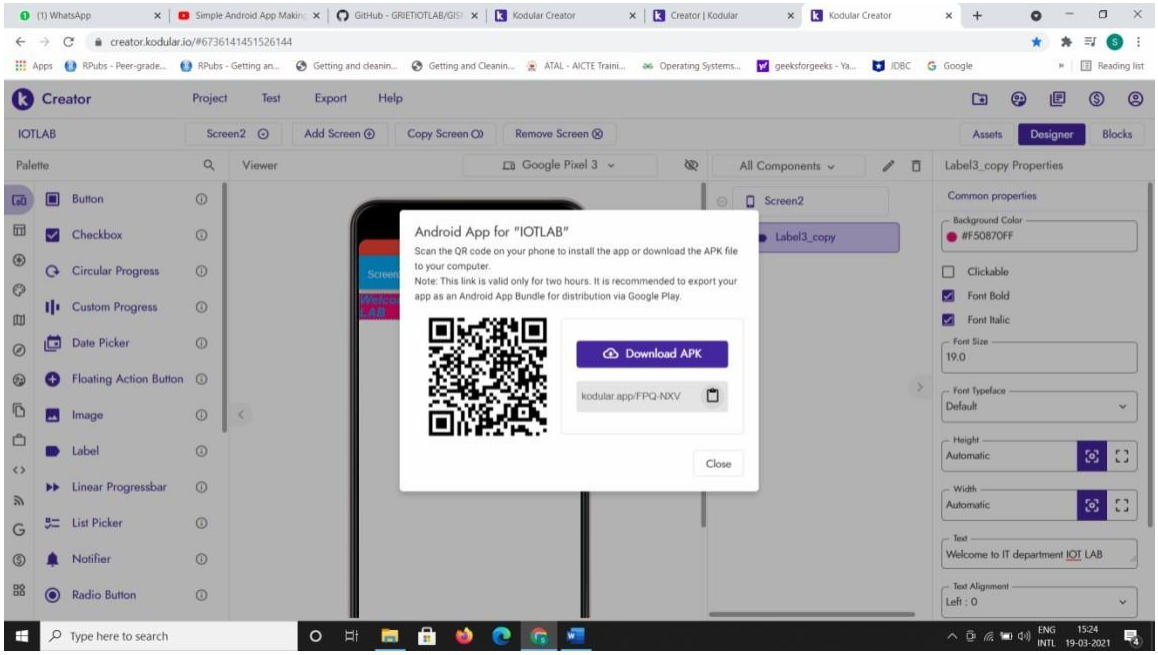
Step 14: If you scan the QR code in your mobile, the app will be opened in your mobile. Now test the app whether it is working correctly or not.

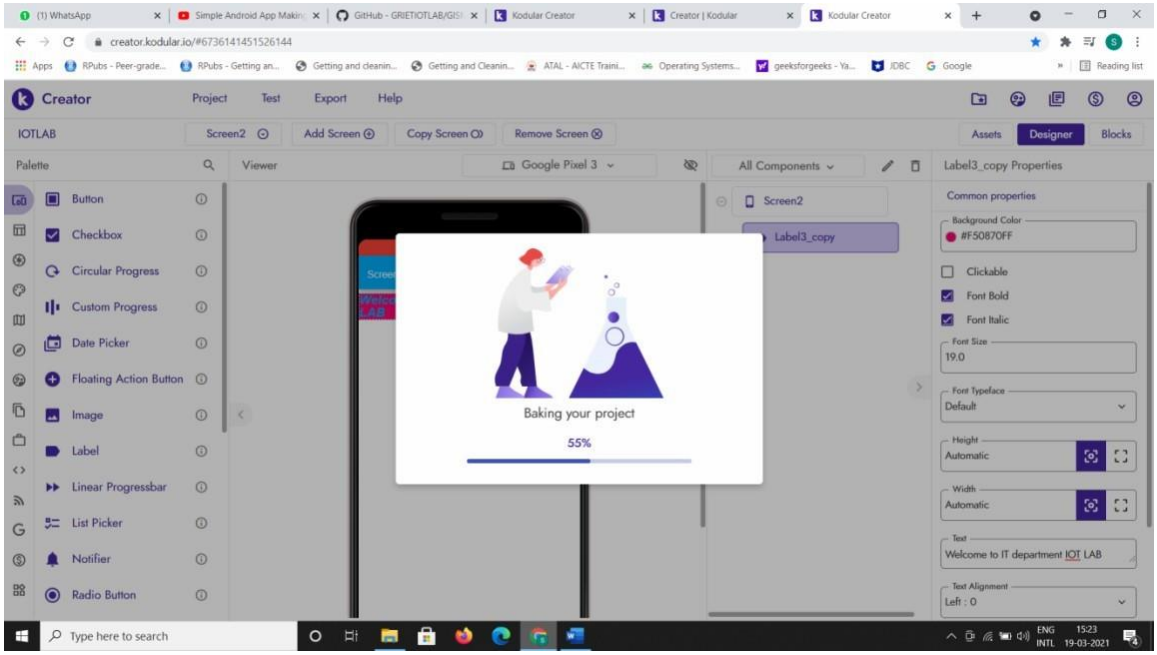


Step 15: Then to create permanent mobile app, go to Export option and click android app.


Creator
Project
Test
Export
Help
IOTLAB
Screen2
Android App (.apk)
Android App Bundle (.aab)
Remove Screen
Assets
Designer
Blocks
Palette
Viewer
Google Pixel 3
All Components
Label3_copy Properties
Button
Checkbox
Circular Progress
Custom Progress
Date Picker
Floating Action Button
Image
Label
Linear Progressbar
List Picker
Notifier
Radio Button
Screen2
Label3_copy
Common properties
Background Color
#F50870FF
Clickable
Font Bold
Font Italic
Font Size
19.0
Font Typeface
Default
Height
Automatic
Width
Automatic
Text
Welcome to IT department IOT LAB
Text Alignment
Left : 0
Type here to search


Creator
Project
Test
Export
Help
IOTLAB
Screen2
Add Screen
Copy Screen
Remove Screen
Assets
Designer
Blocks
Palette
Viewer
Google Pixel 3
All Components
Label3_copy Properties
Pre-heating the ovens
26%
Type here to search





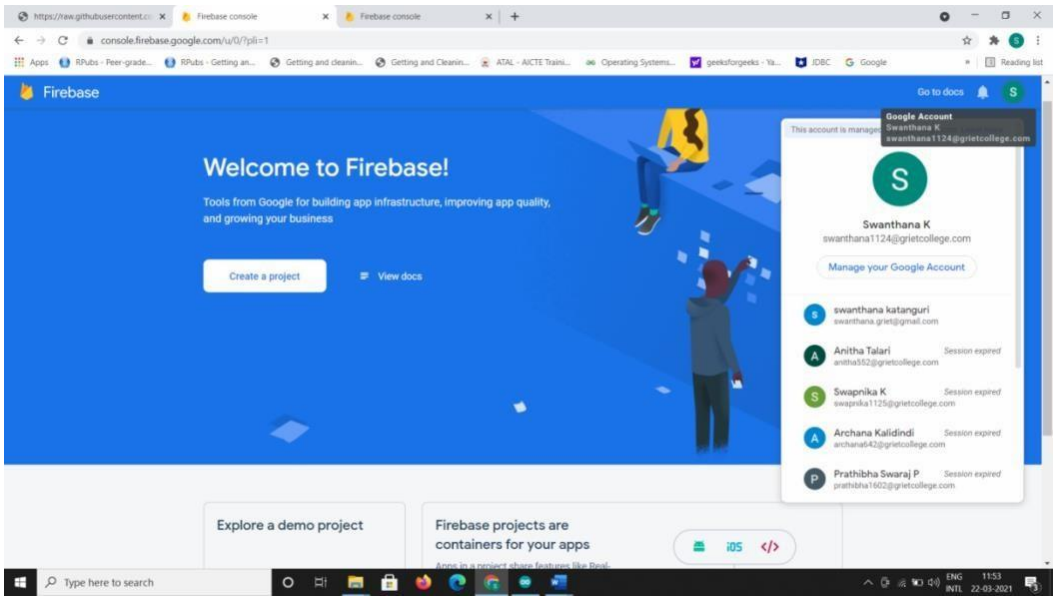
Step 16: Now scan the QR code and also click on download APK. Then download the app in mobile. Make sure that mobile and laptop should be connect to same wifi. Then you will get app in mobile.

Design a Mobile app to work with data of a cloud

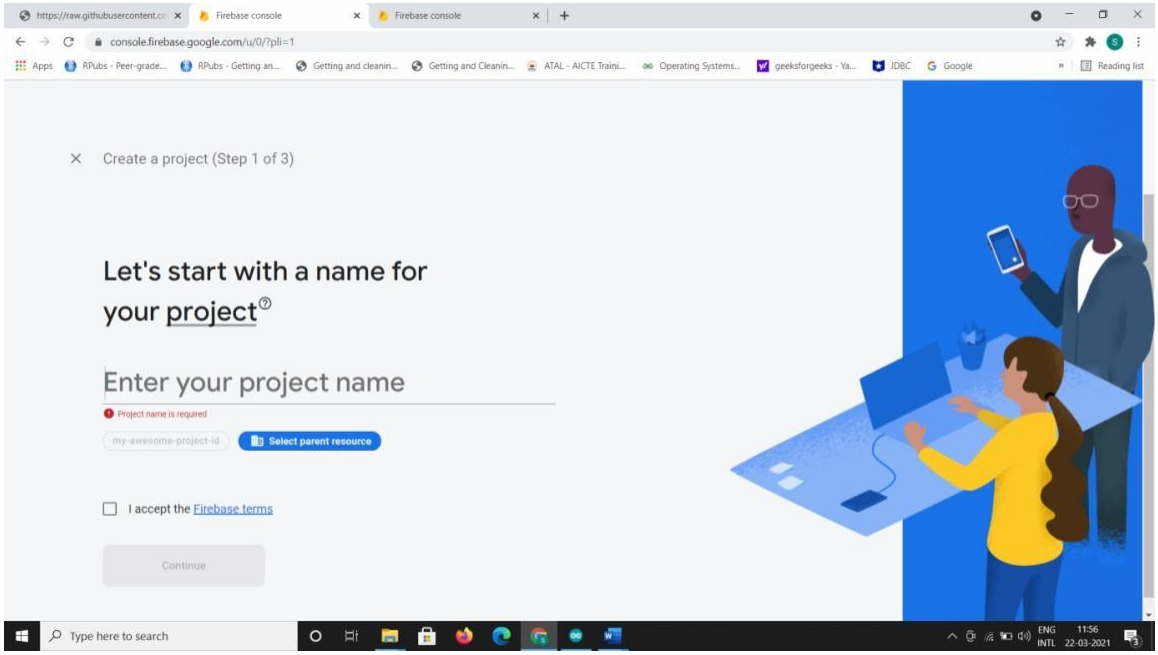
Program:

Follow the below steps to create a firebase (cloud) project.

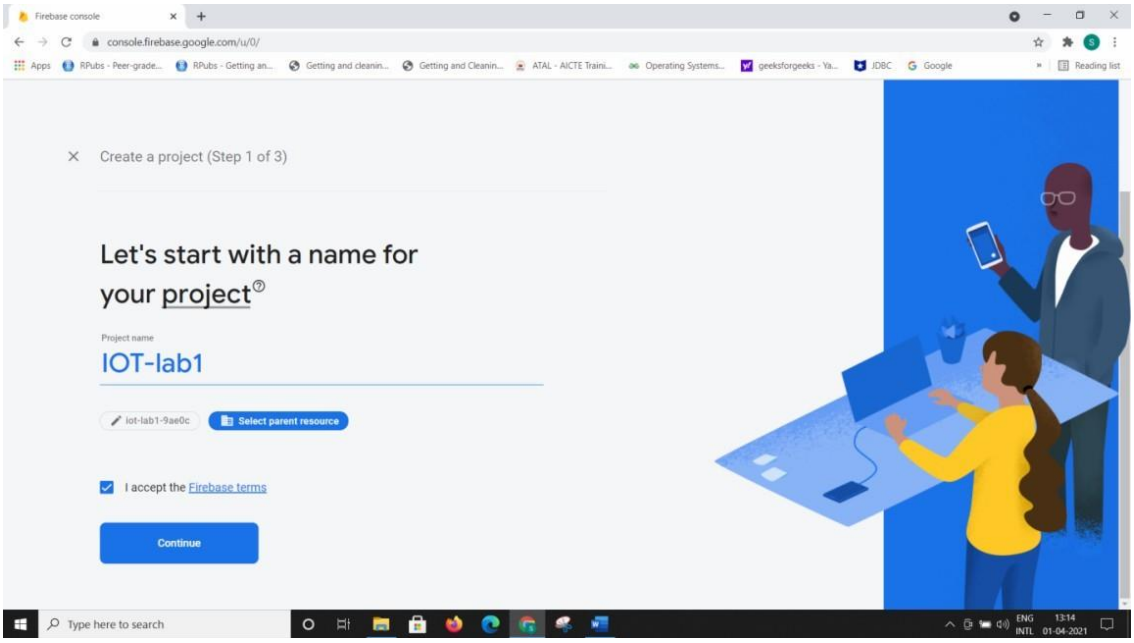
Step 1:



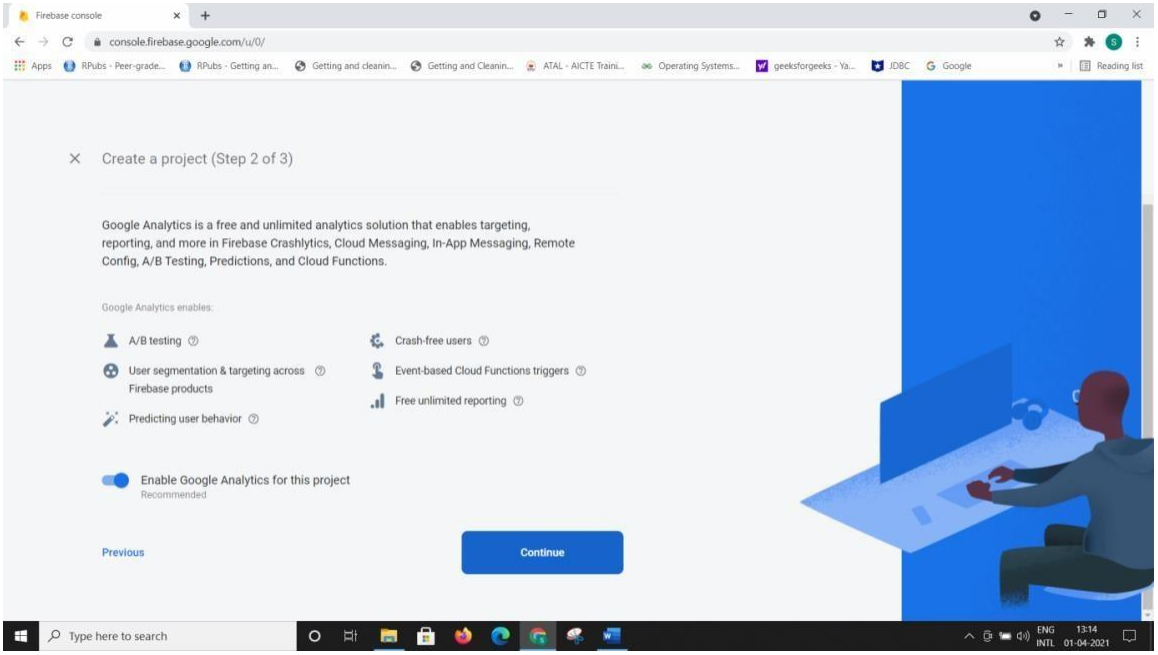
Step 2: click on create project.



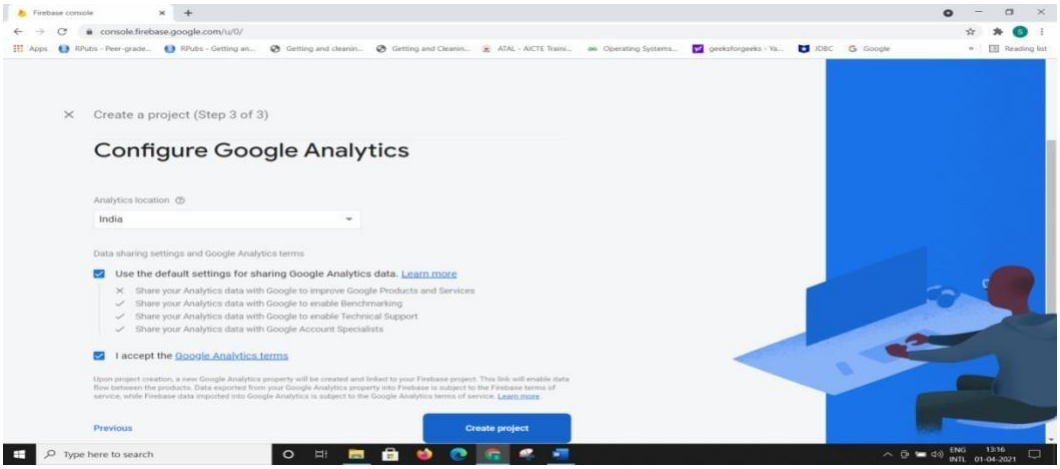
Step 3: Here type your project name and tick I accept and the click on continue.

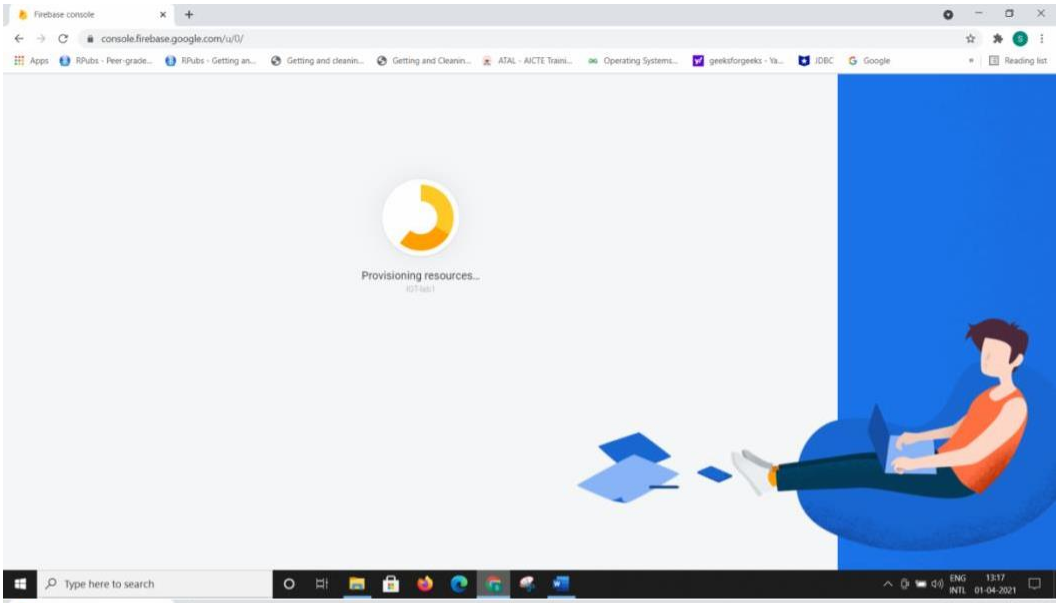


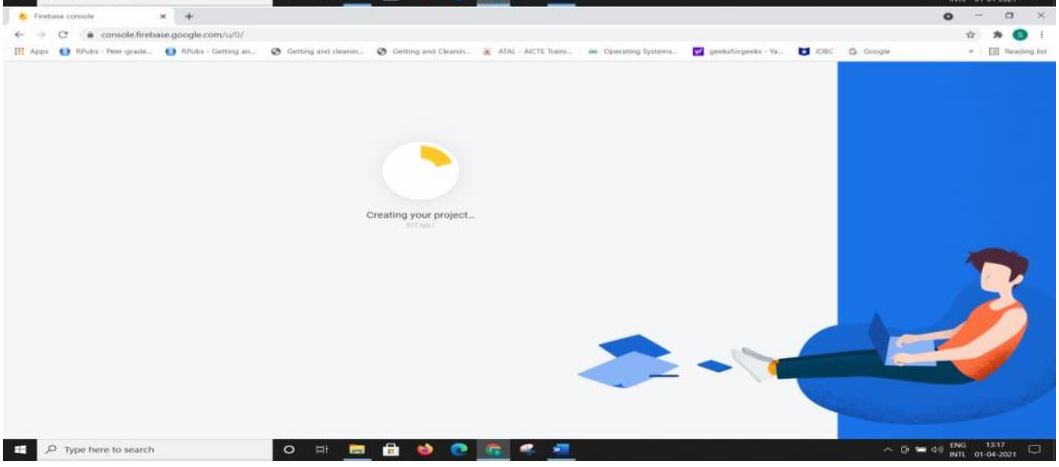
Step 4: Click on continue

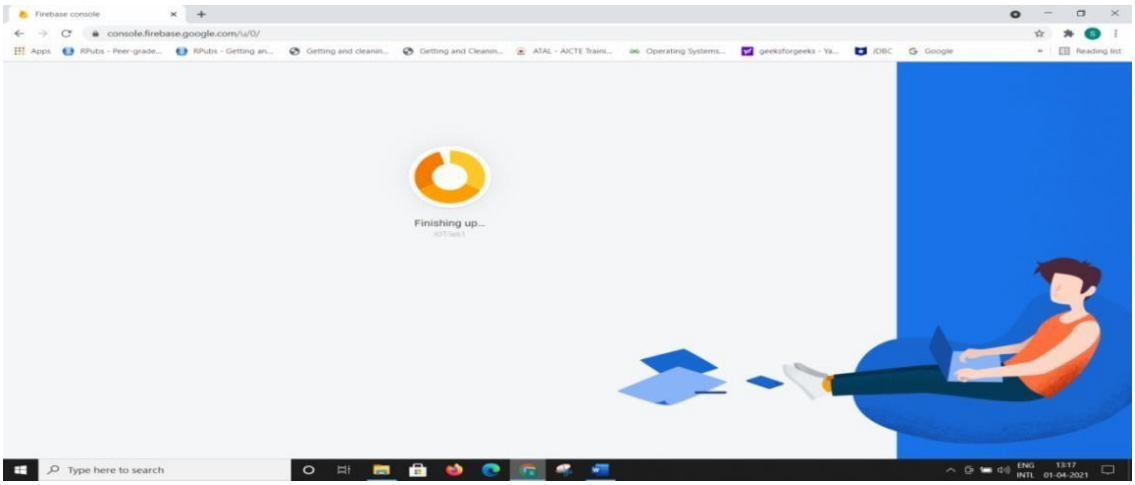


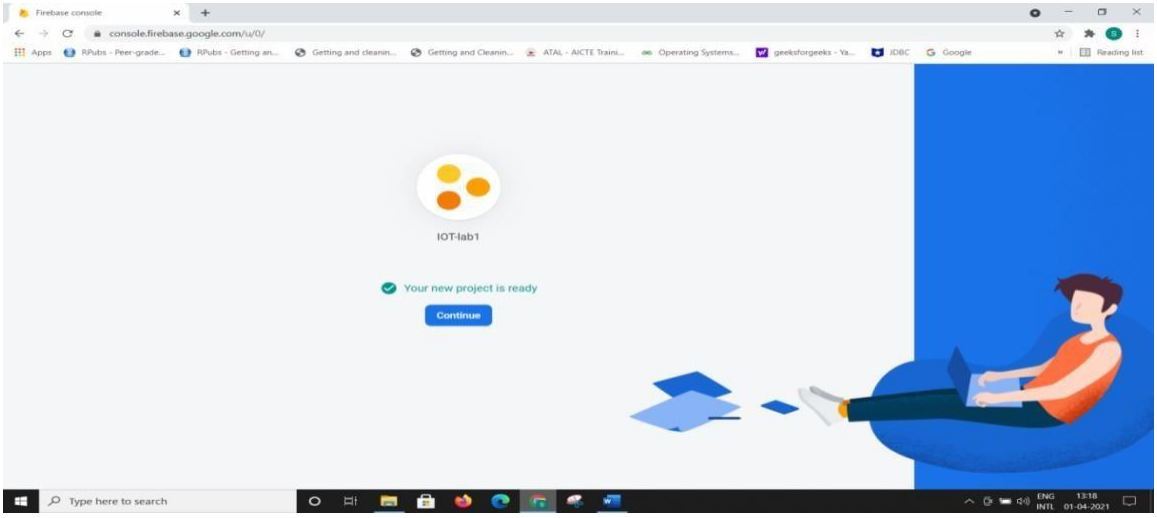
Step 5: Select the location as INDIA and tick the check box of I accept the terms and finally click on Create Project button.


Create a project (Step 3 of 3)
Configure Google Analytics
Analytics location
India
Data sharing settings and Google Analytics terms
Use the default settings for sharing Google Analytics data. Learn more
Share your Analytics data with Google to improve Google Products and Services
Share your Analytics data with Google to enable Benchmarking
Share your Analytics data with Google to enable Technical Support
Share your Analytics data with Google Account Specialists
I accept the Google Analytics terms
Previous
Create project

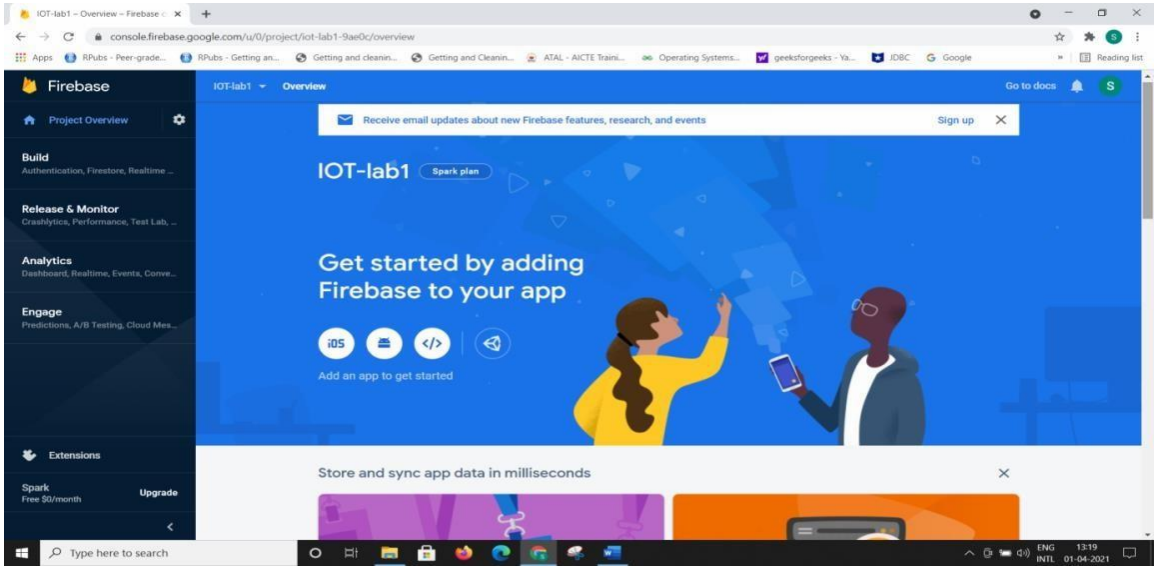

Provisioning resources...


Creating your project...

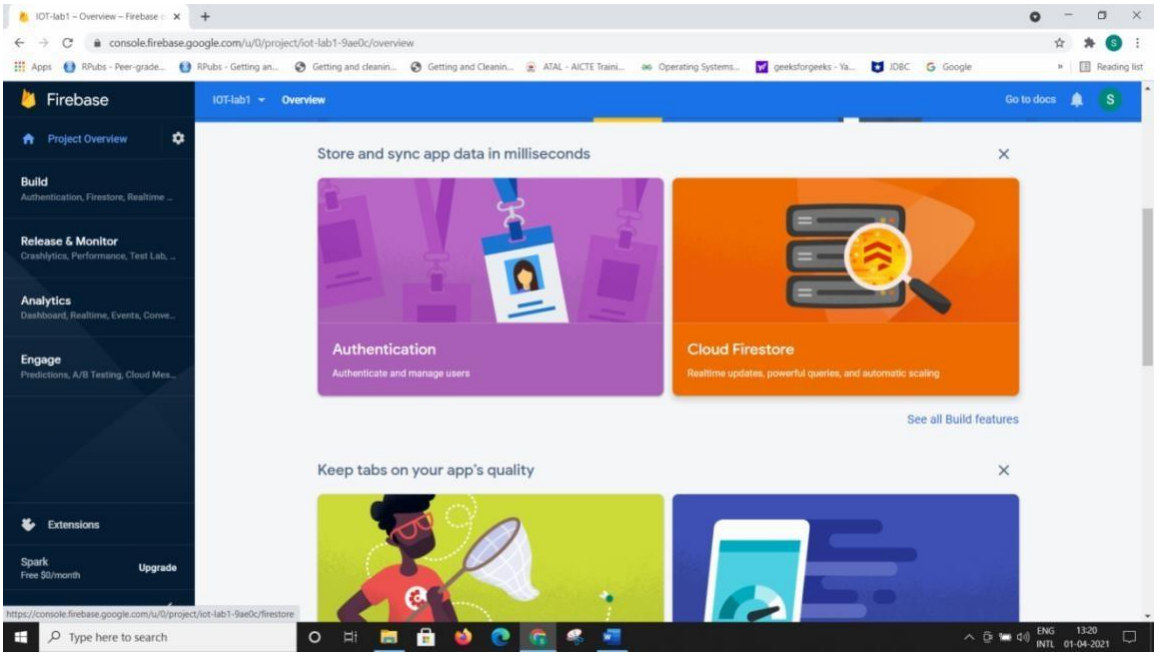


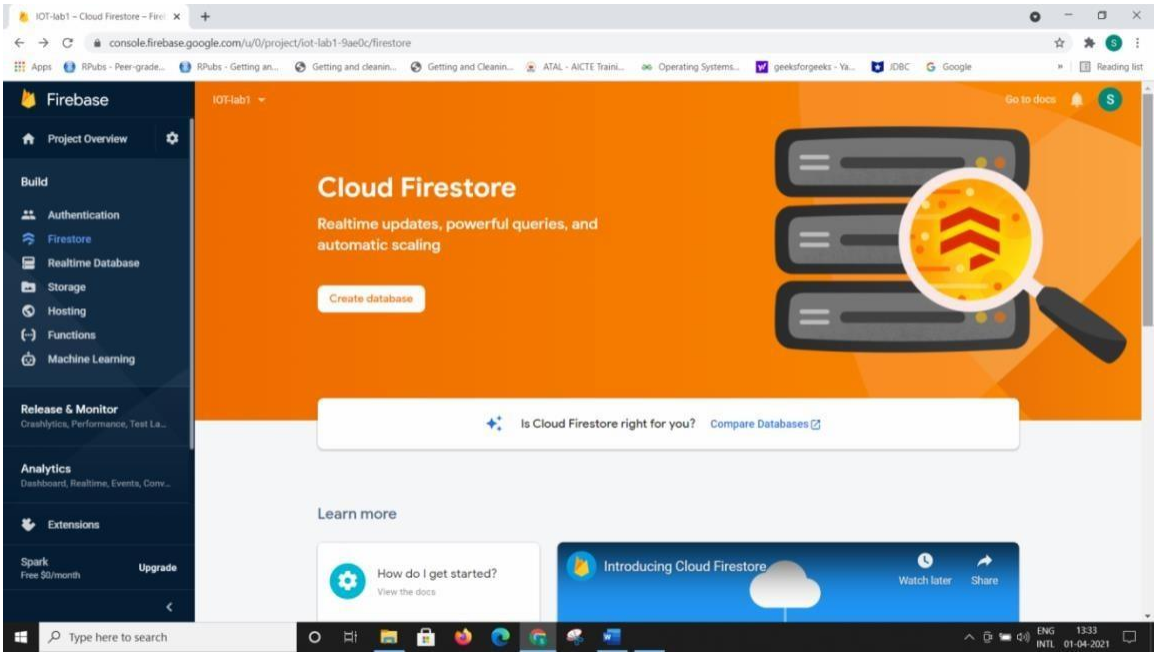


Step 6: Now project is created then click on continue.

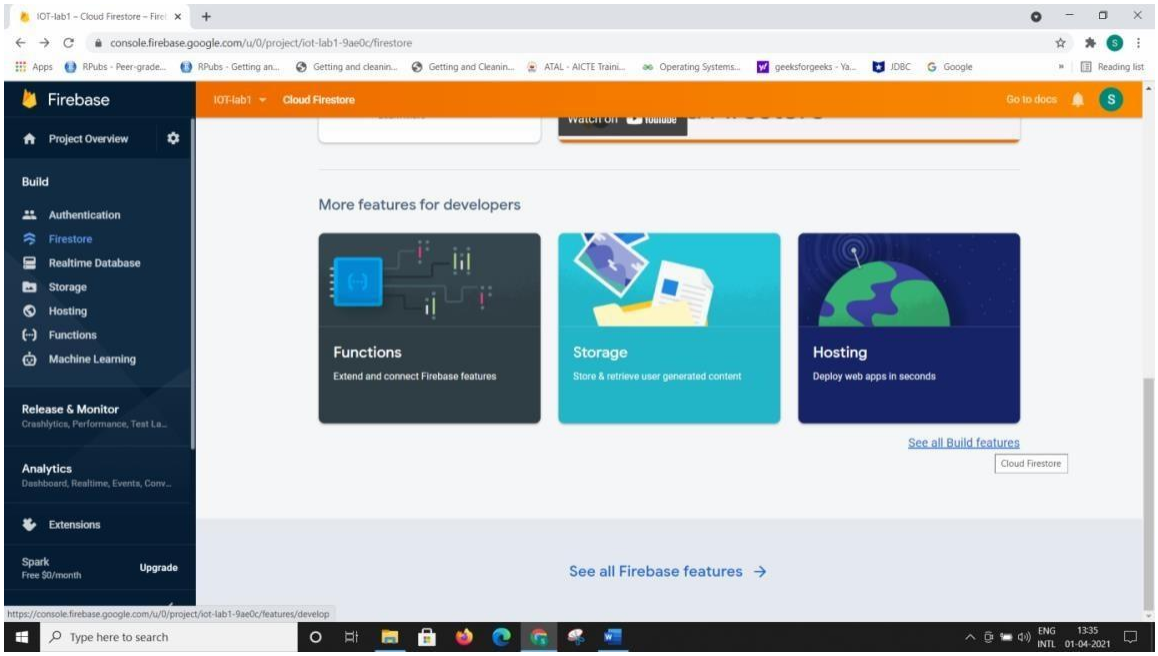


Step 7: Now move down the scroll bar and select cloud fire store.

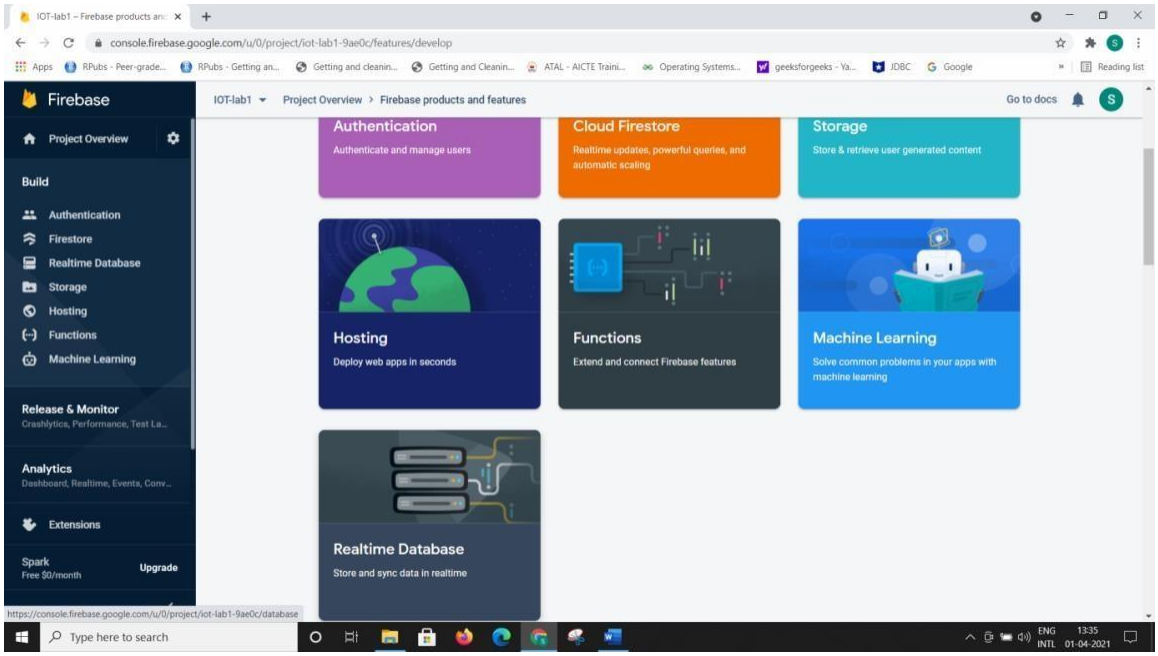




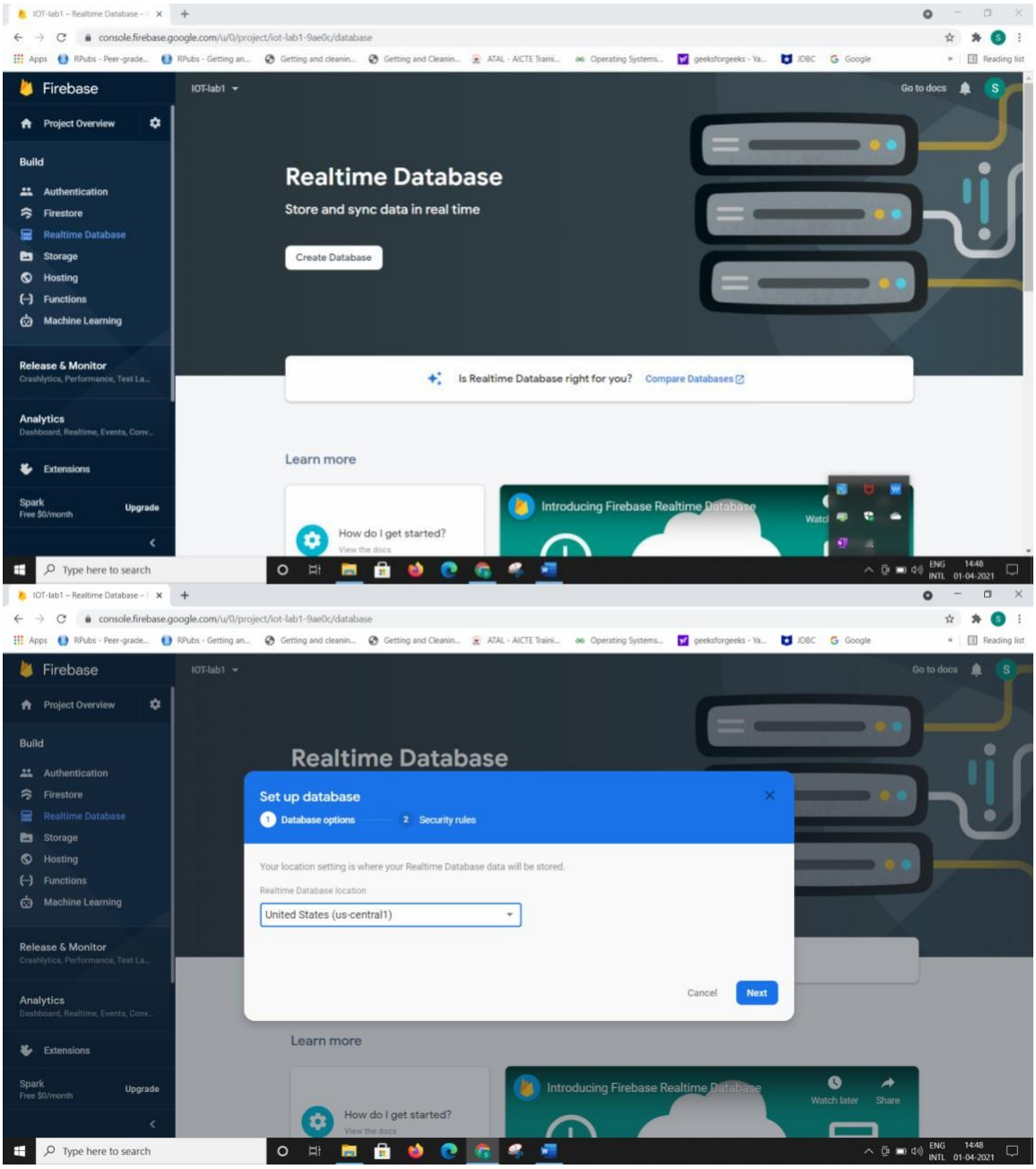
Now scroll down and select see all build features.



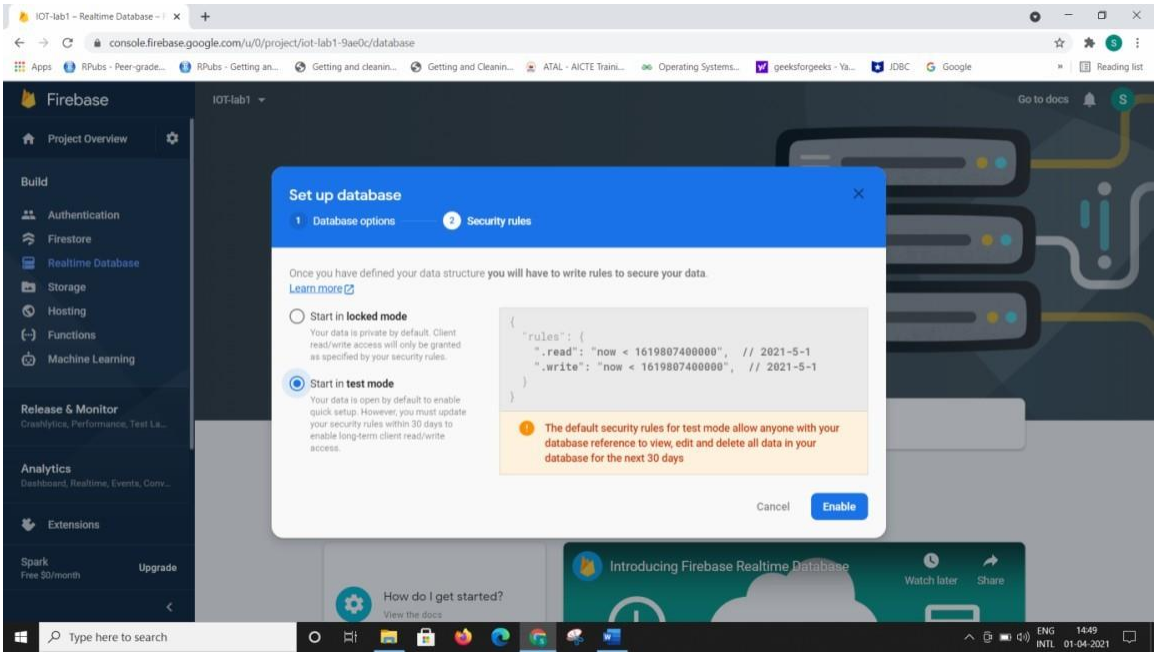
Now select real time database



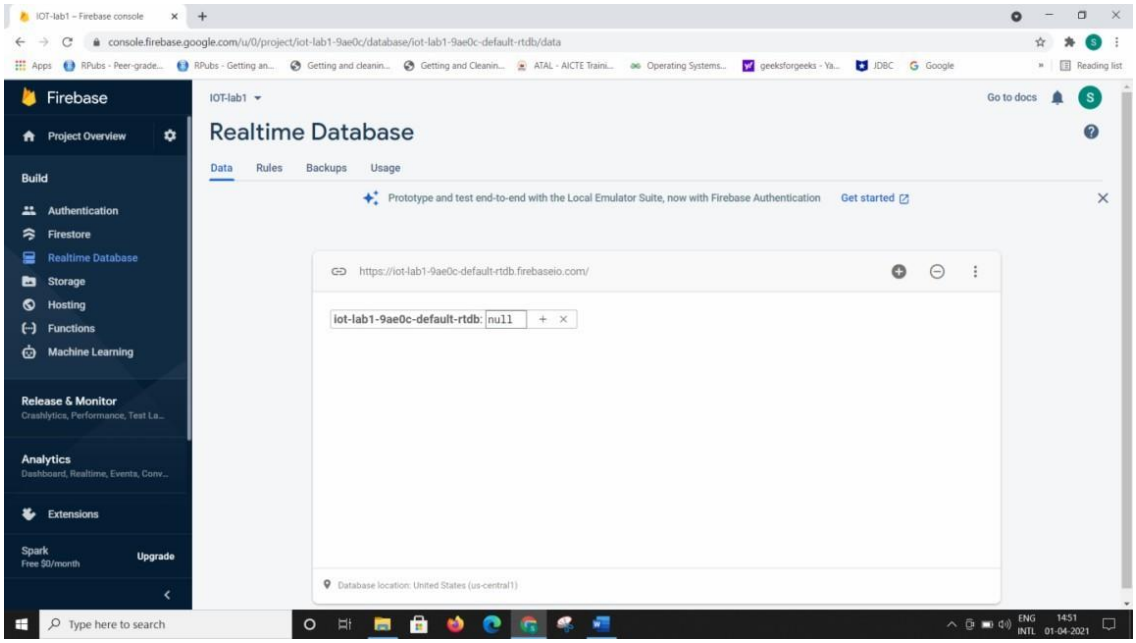
Now click on create database



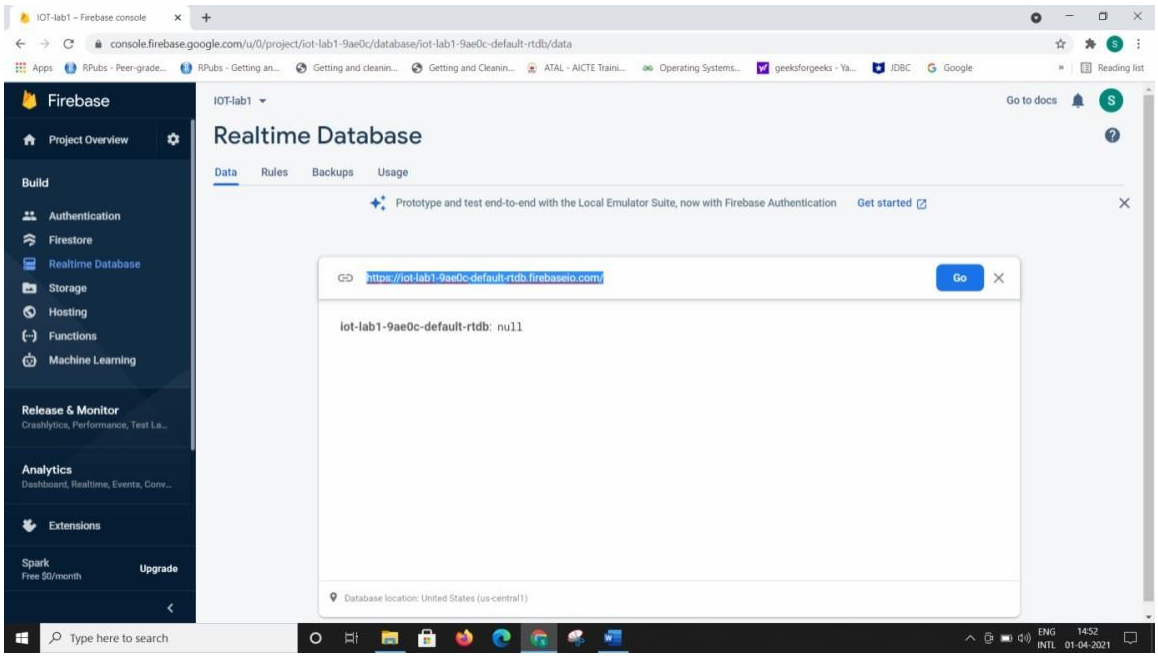
Now select start in test mode radio button and then click on Enable button.



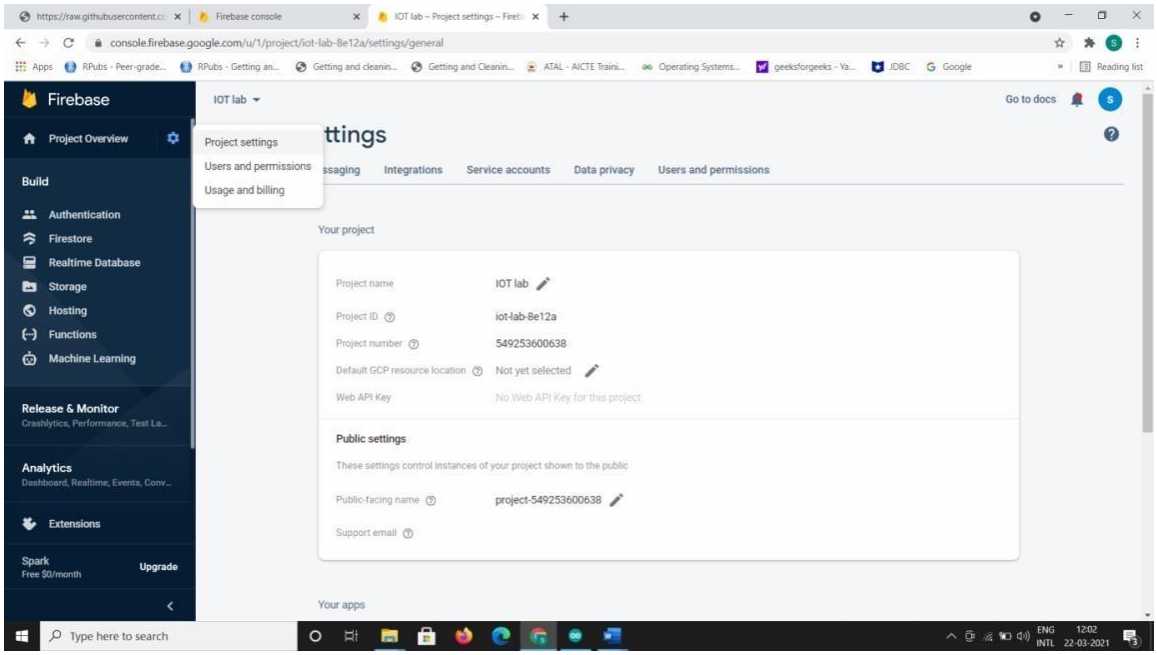
Now we will get the following window



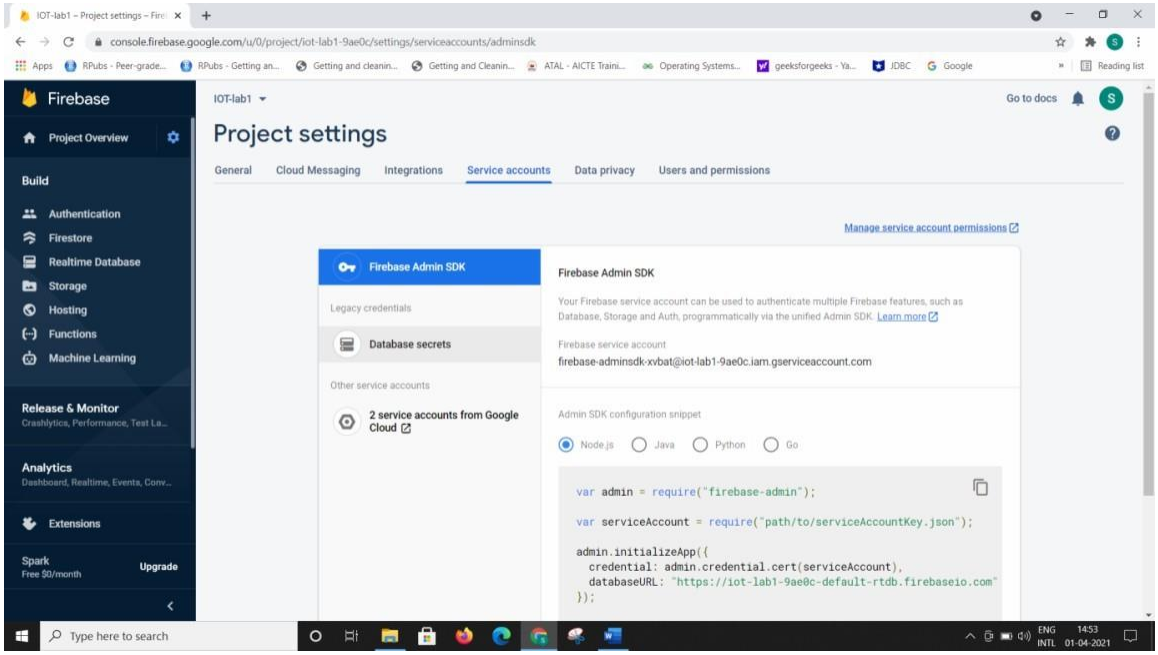
Now copy the fire base URL into a note pad.



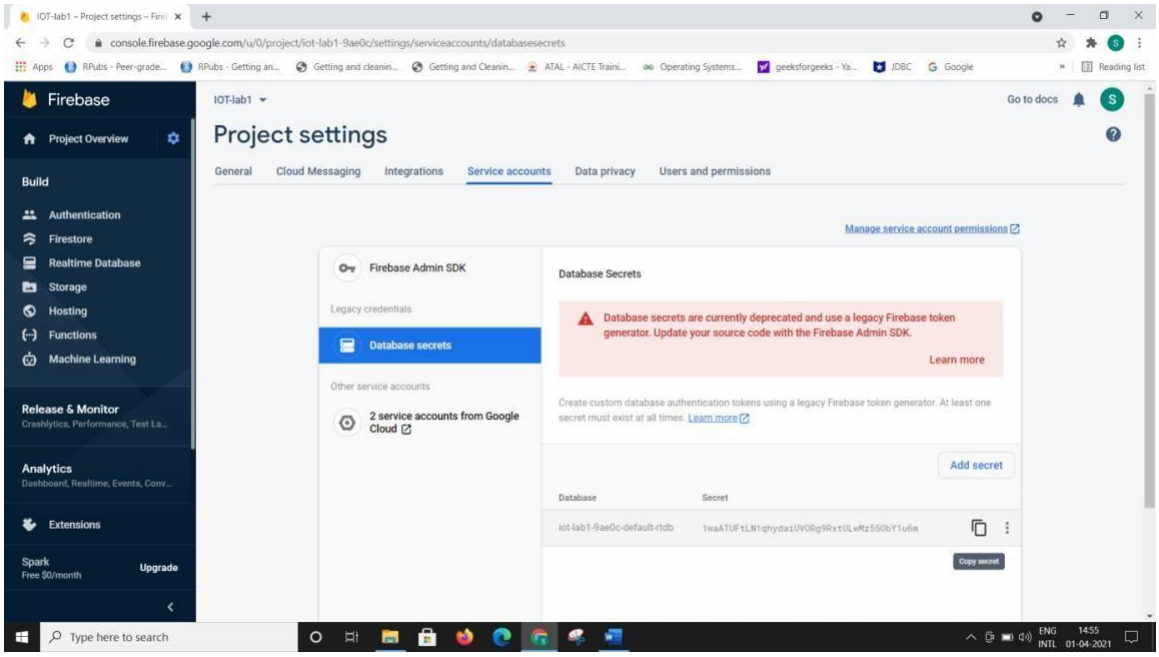
Step 8: Go to left side pallet and click on settings icon and select project settings.



Now select service accounts.



Click on database secrets which is in the left panel and there we find the secret key copy that key into the notepad (whenever we need it we will use it).

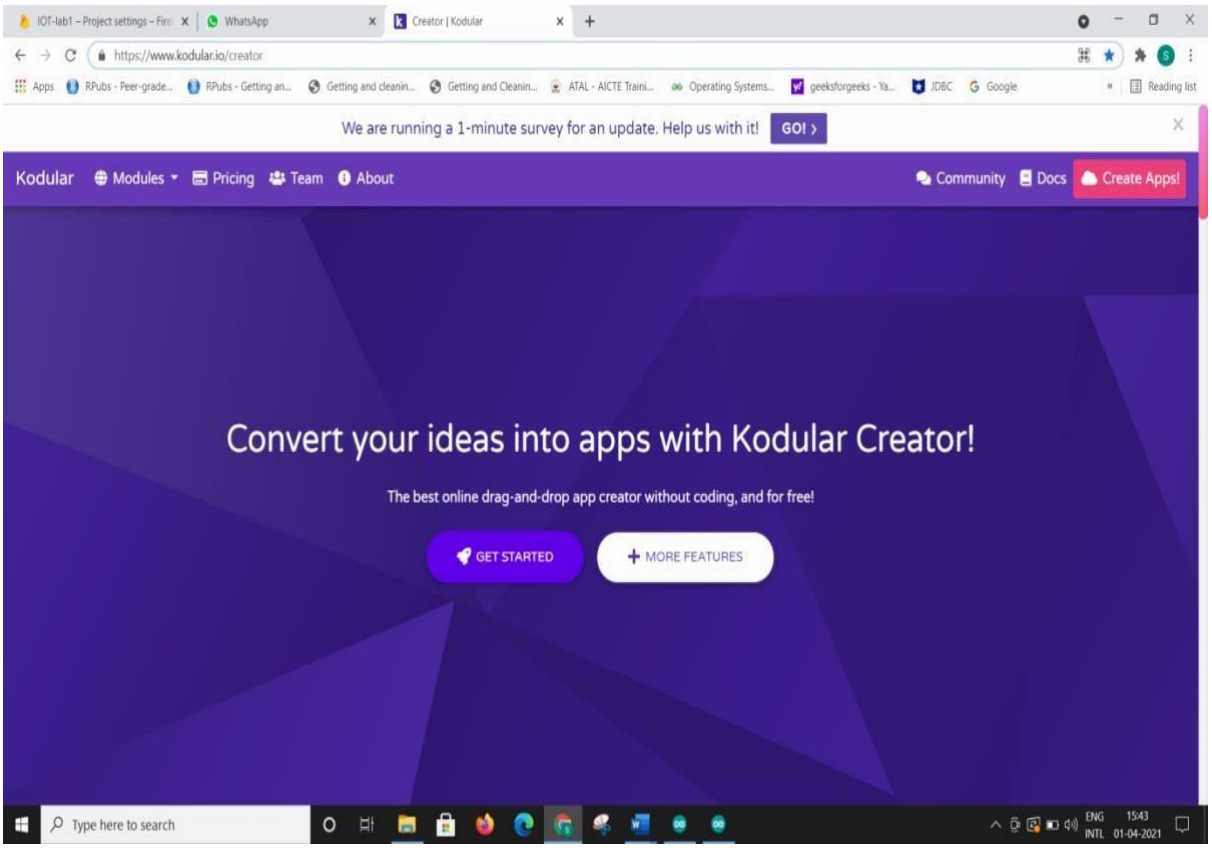


Now we are done with creation of a database in cloud (firebase) to store the data which comes from the board. For example, we will take an Arduino program to sense temperature and humidity using DHT11 sensor on the board.

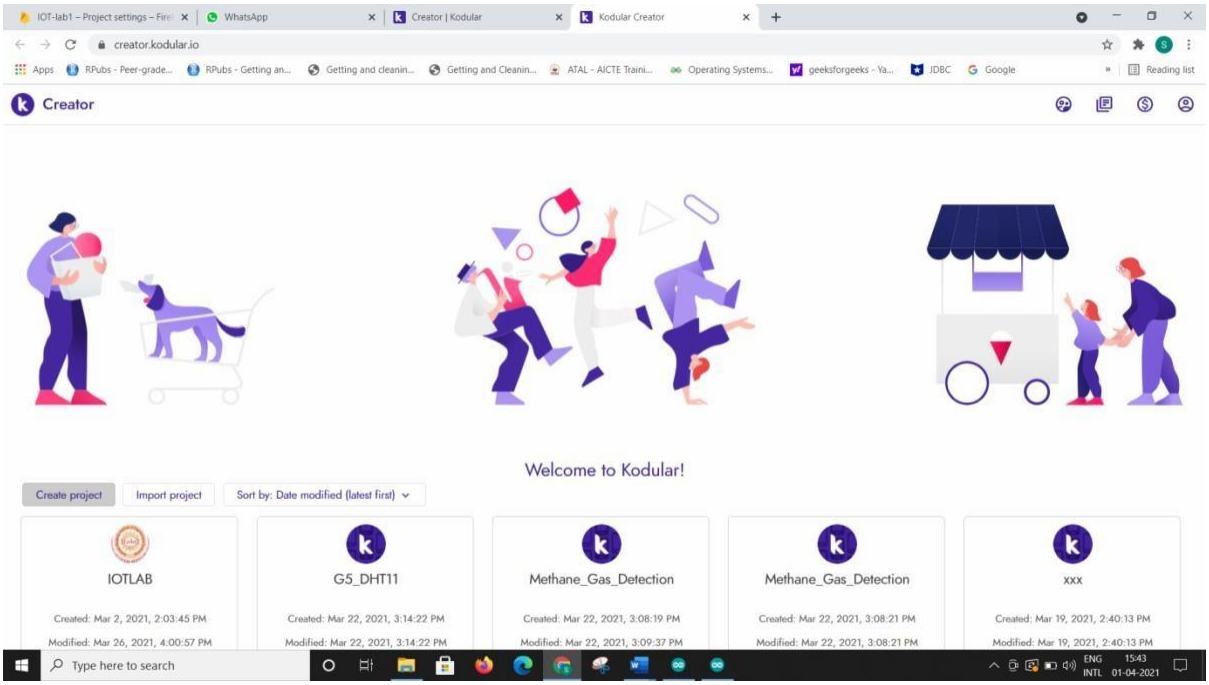
Now let us create a mobile app. We go with kodular creator.

The following steps explain how to develop a mobile app and display the data present in cloud.

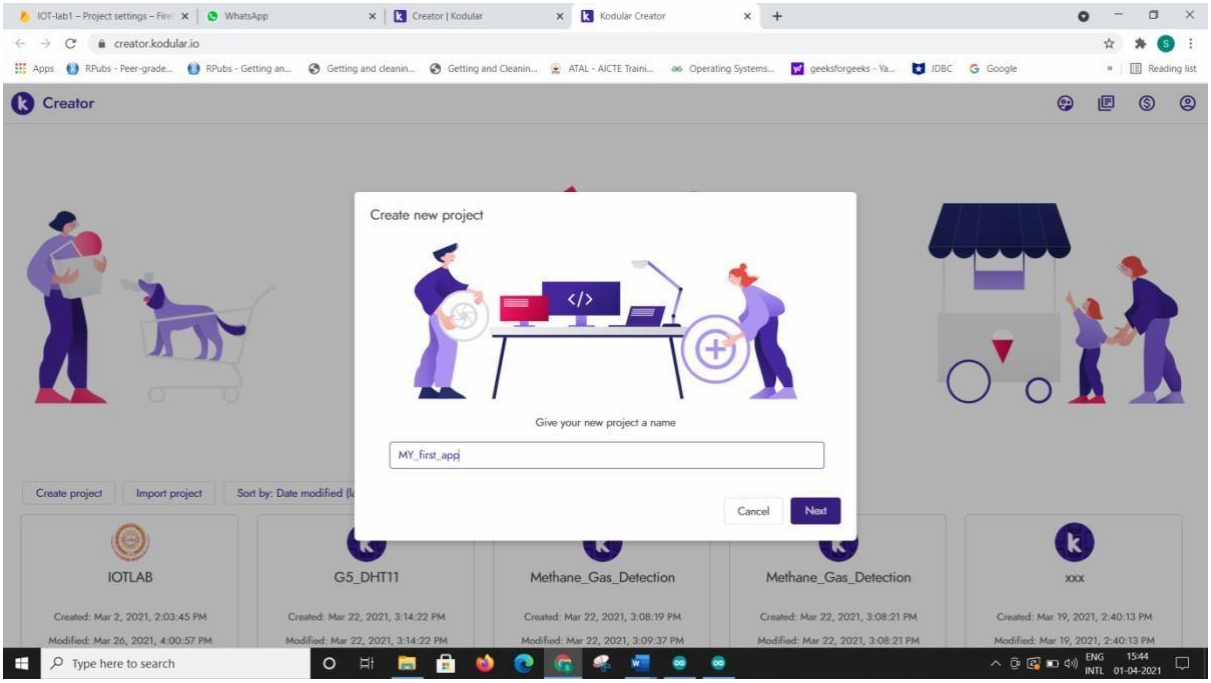
Step 1: Open kodular creator by typing in Google or go to this link https://www.kodular.io/creator



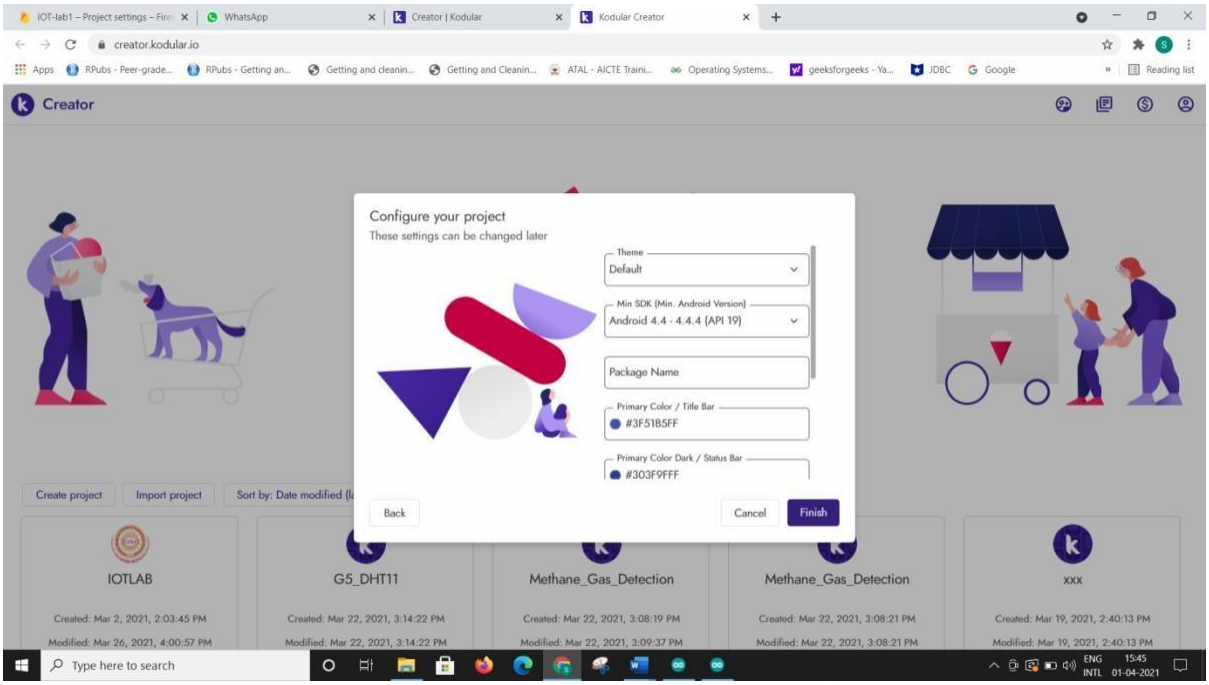
Now click on get started button



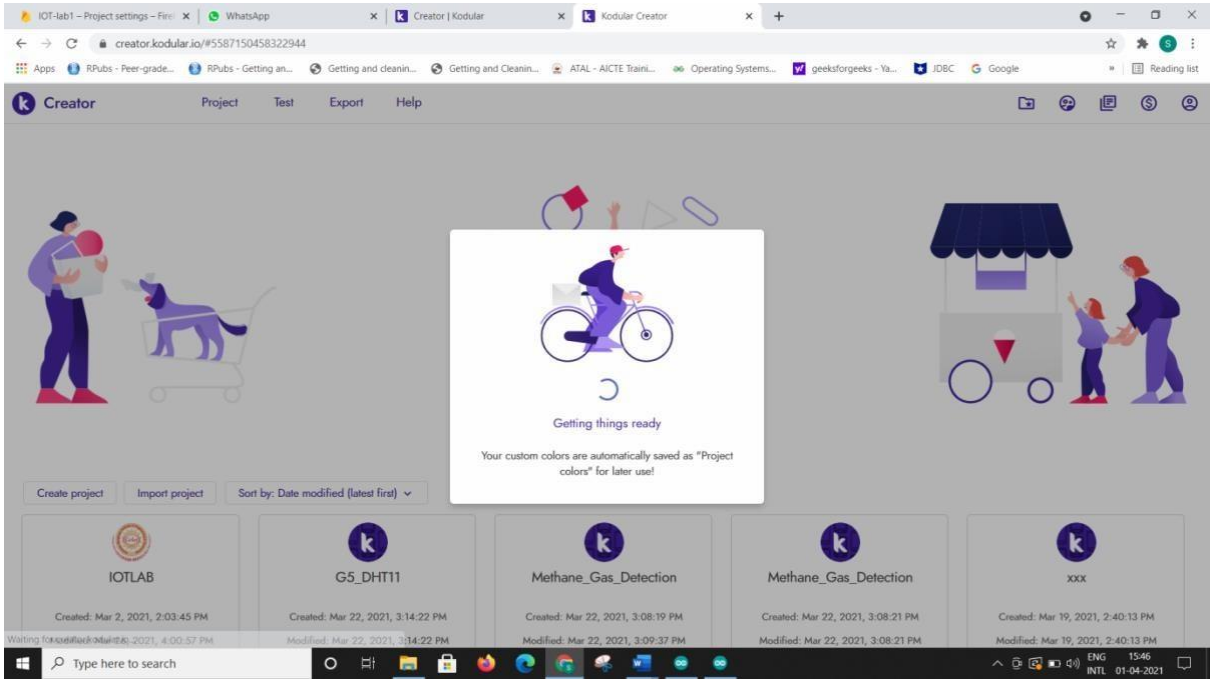
Here click on create project and give the name of the project and click on next



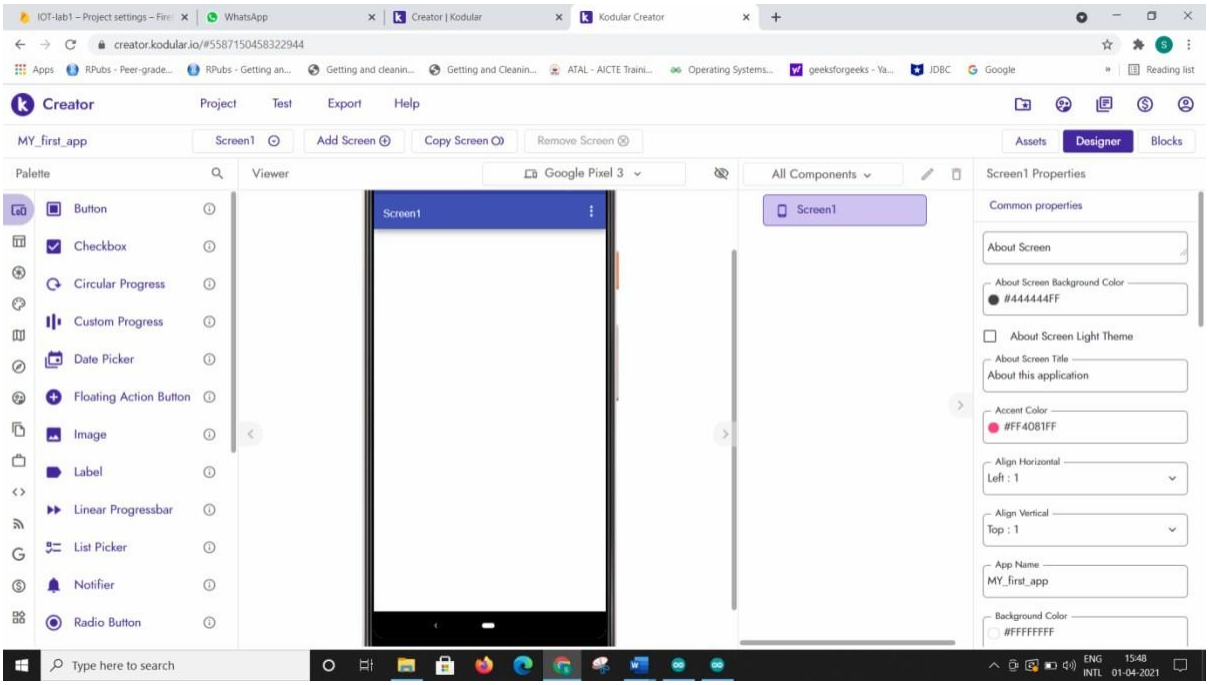
Then we will get this following window then just click on finish.



Then we see this following window.



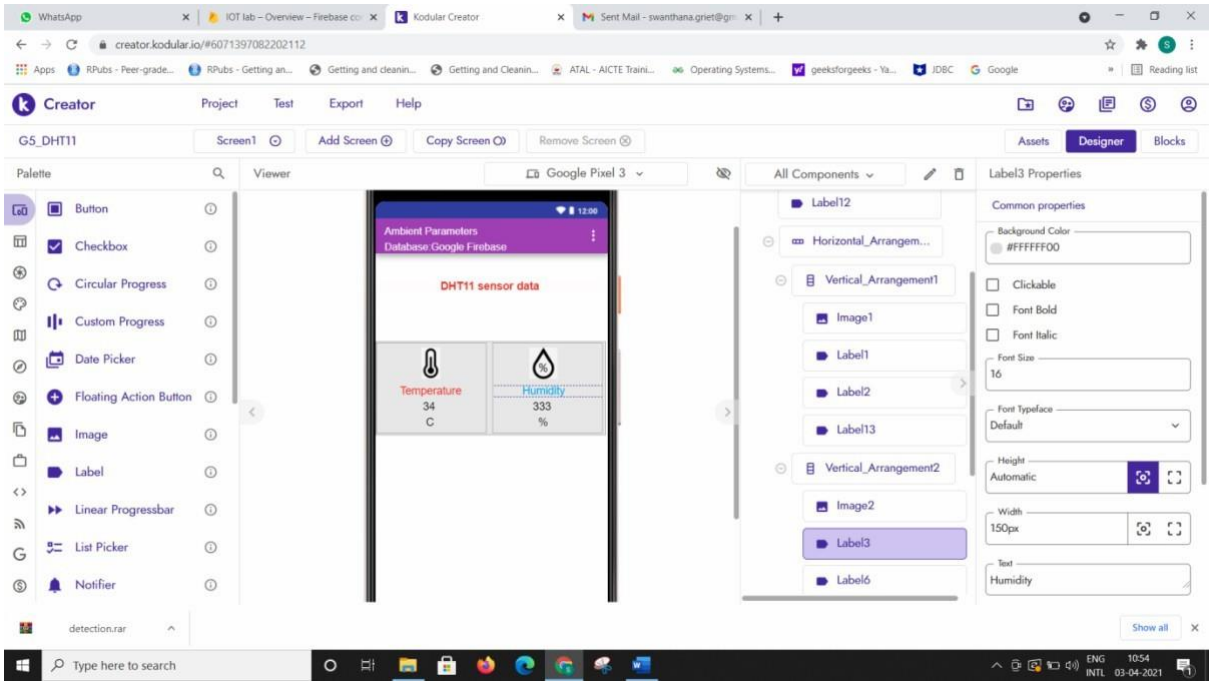
We get this following page to create your mobile app.

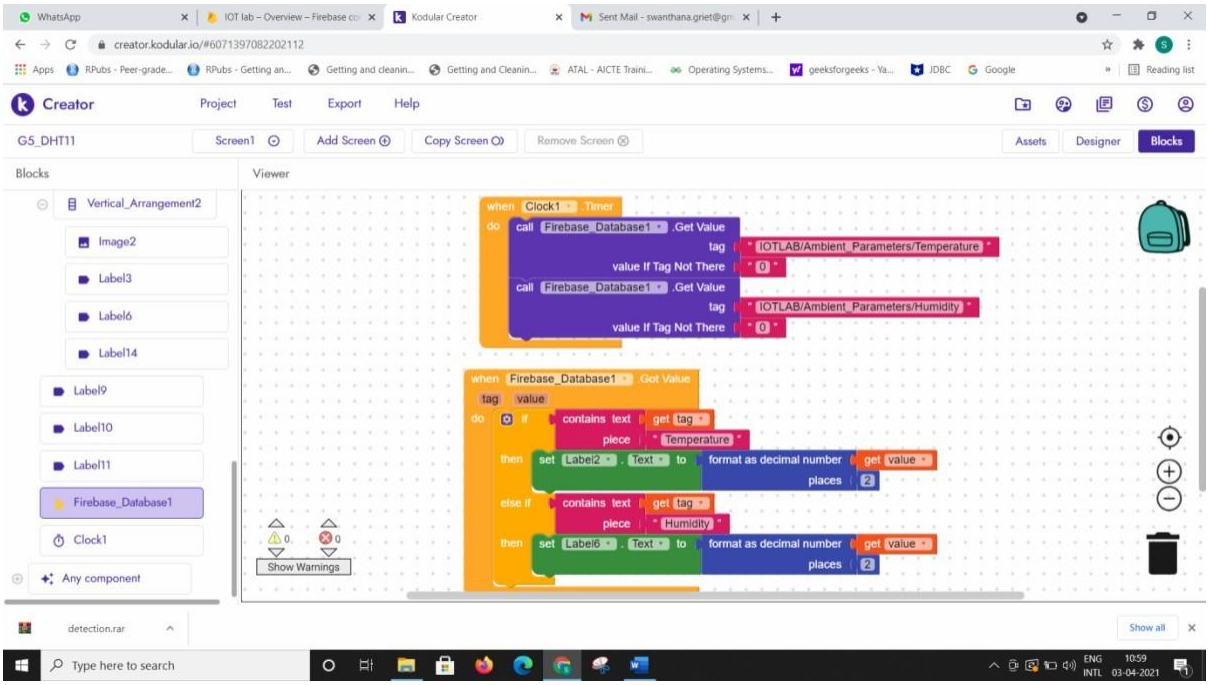


Here on the left side, we will see the Palette where we can find all the GUI widgets, we should just drag and drop the necessary widgets on to the mobile screen visible beside the palette.

Drag and drop the labels and name the text of the labels as Temperature and Humidity and the take more labels to display the values of temperature and humidity by getting the values from the firebase (database). And also we can use alignments also like vertical or horizontal accordingly to your requirements, you should also drag and drop the firebase and clock from the Palette.

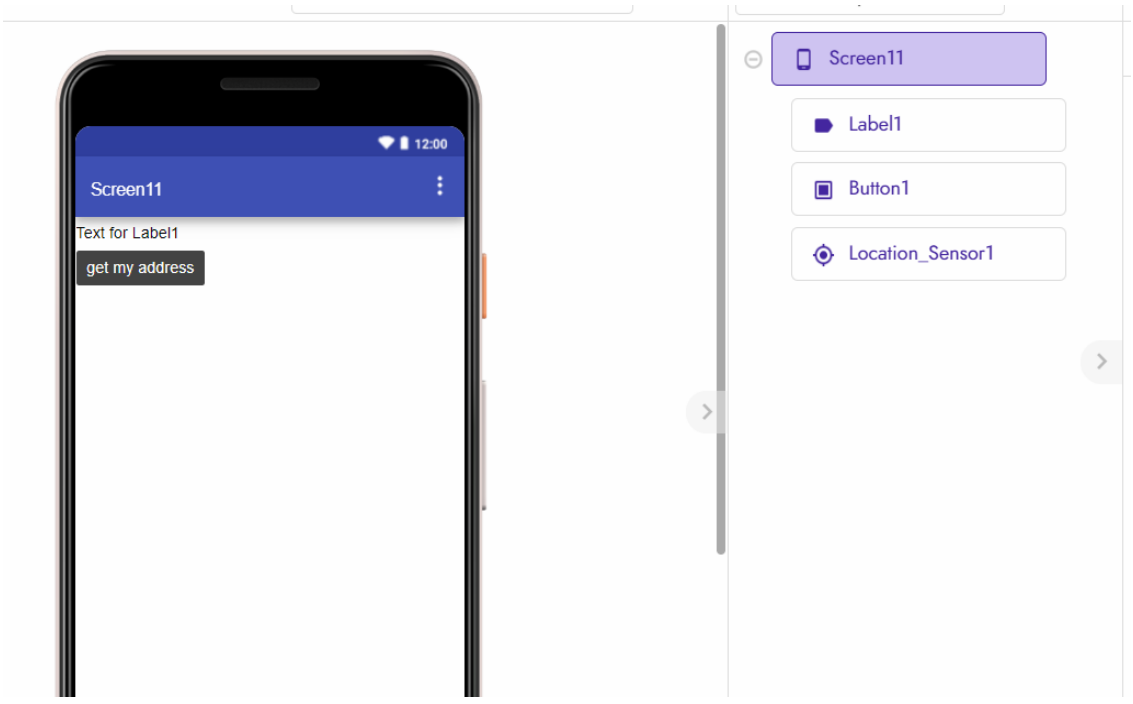
Now design part of the mobile app is done. Now move to Blocks to write the code by clicking Blocks on the right-side top of the window.





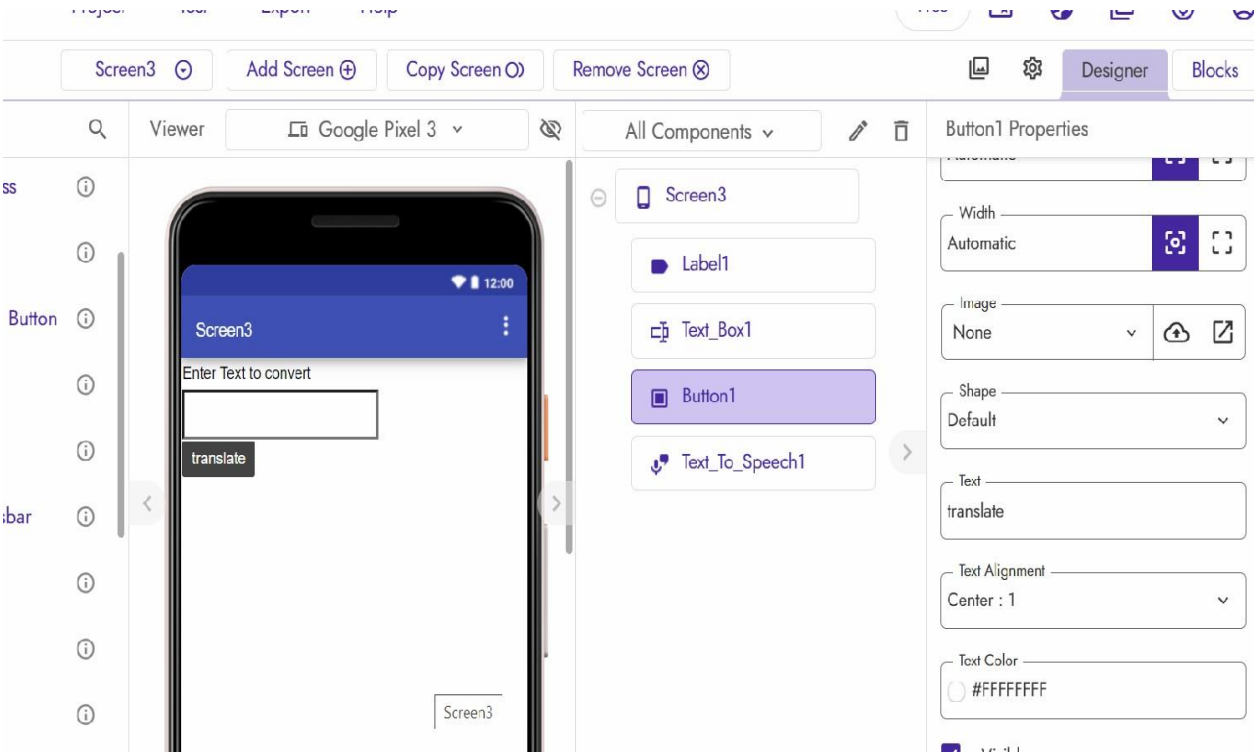
Now your mobile app is ready which is connected to firebase to get the ambient parameters that are updated for every clock second from the board.

7.A) Location Sensor



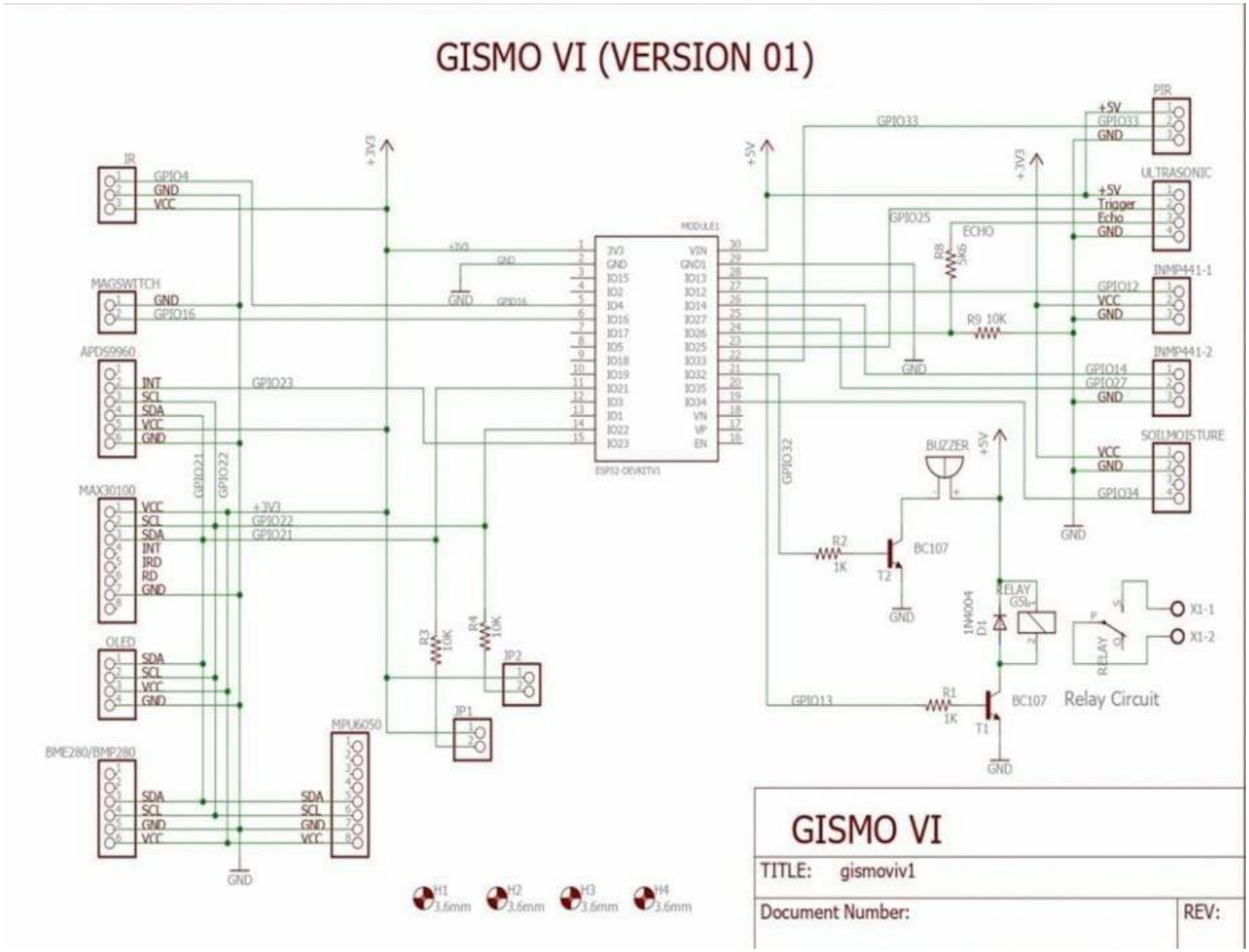


7.B) Text to speech Converter





GISMOVI Schematic Diagram :



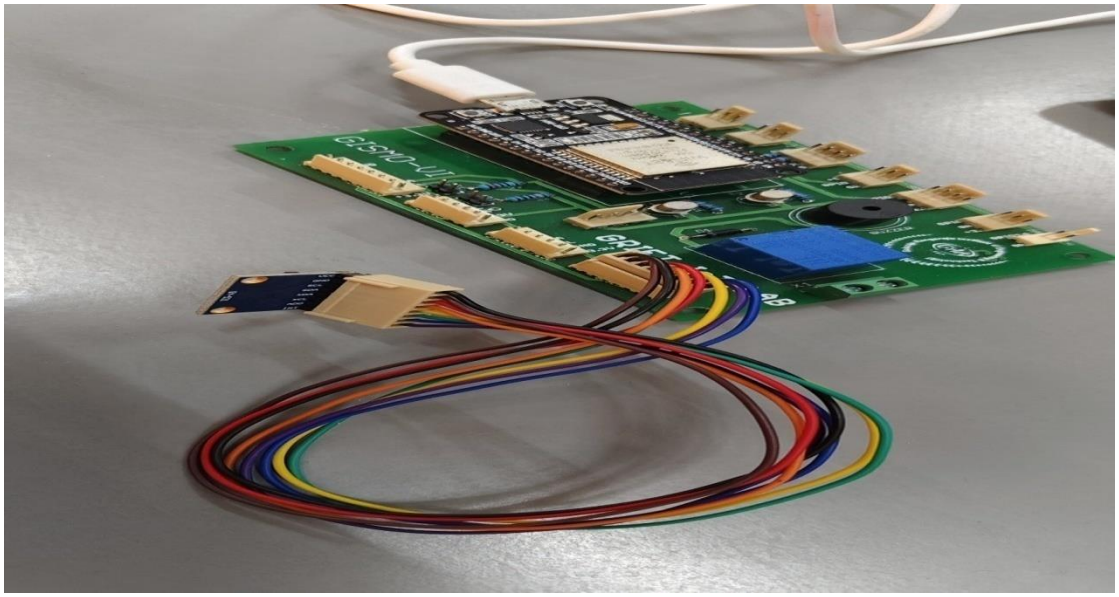
TASK 8

MPU6050

Aim: To Write an Arduino program for Sensor/Activator Interfacing using ESP32.

Components Required: GISMOVI Board, MPU6050, Cable.

Circuit:



Program:

```
#define FIREBASE_HOST "soil-test-bd5b2-default-rtdb.asia-
southeast1.firebasedatabase.app/" #define FIREBASE_AUTH
"ywbGPCbl9w2G56X8URbTyZUsFDjWVUVNAkfF2IS9" #define WIFI_SSID "CSE
MC IOT"
#define WIFI_PASSWORD "griet7788"
#include<Wire.h>
#include <WiFi.h>
#include "FirebaseESP32.h"
// #include "MPU6050.h"  // Commented out MPU6050

FirebaseData firebaseData;
// MPU6050 mpu;  // Commented out MPU6050

// Defines variables
// int16_t ax, ay, az;
// int16_t gx, gy, gz;
const int reedSwitchPin = 5;  // Reed switch connected to GPIO5 bool reedSwitchState;
void WiFiInit();
```

```
void FirebaseInit();

void FirebaseWrite();

// String tagAccel = "IOTLAB/MPU6050/Accelerometer";
// String tagGyro = "IOTLAB/MPU6050/Gyroscope";
String tagMagSwitch = "IOTLAB/Magnetic_Switch";  // Firebase path for reed switch
void setup() {
Serial.begin(115200);
Wire.begin();
WiFiInit();
FirebaseInit();

pinMode (reedSwitchPin, INPUT_PULLUP); // Reed switch uses internal pull-up

// Initialize MPU6050 (Commented out)
// mpu.initialize();
// if (!mpu.testConnection()) {
//     Serial.println("MPU6050 connection failed!");
//     while (1);
// }
}

void loop() {

// Read accelerometer and gyroscope data (Commented out)
// mpu.getAcceleration(&ax, &ay, &az);
// mpu.getRotation(&gx, &gy, &gz);
// Read reed switch state

reedSwitchState = digitalRead(reedSwitchPin); // HIGH = Open, LOW = Closed

// Print values (MPU6050 data removed) Serial.print("Reed Switch: "); Serial.println(reedSwitchState ?
"Open" : "Closed");

FirebaseWrite(); delay(1000);
}
void WiFiInit(){

WiFi.mode(WIFI_STA);
WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
Serial.print("Connecting to Wi-Fi");
int attempts = 0;
while (WiFi.status() != WL_CONNECTED && attempts < 20) { // Retry 20 times Serial.print(".");
delay(1000); attempts++;
}
if(WiFi.status() == WL_CONNECTED){
Serial.println("\nConnected!");
Serial.print("IP Address: ");
 Serial.println(WiFi.localIP());
```

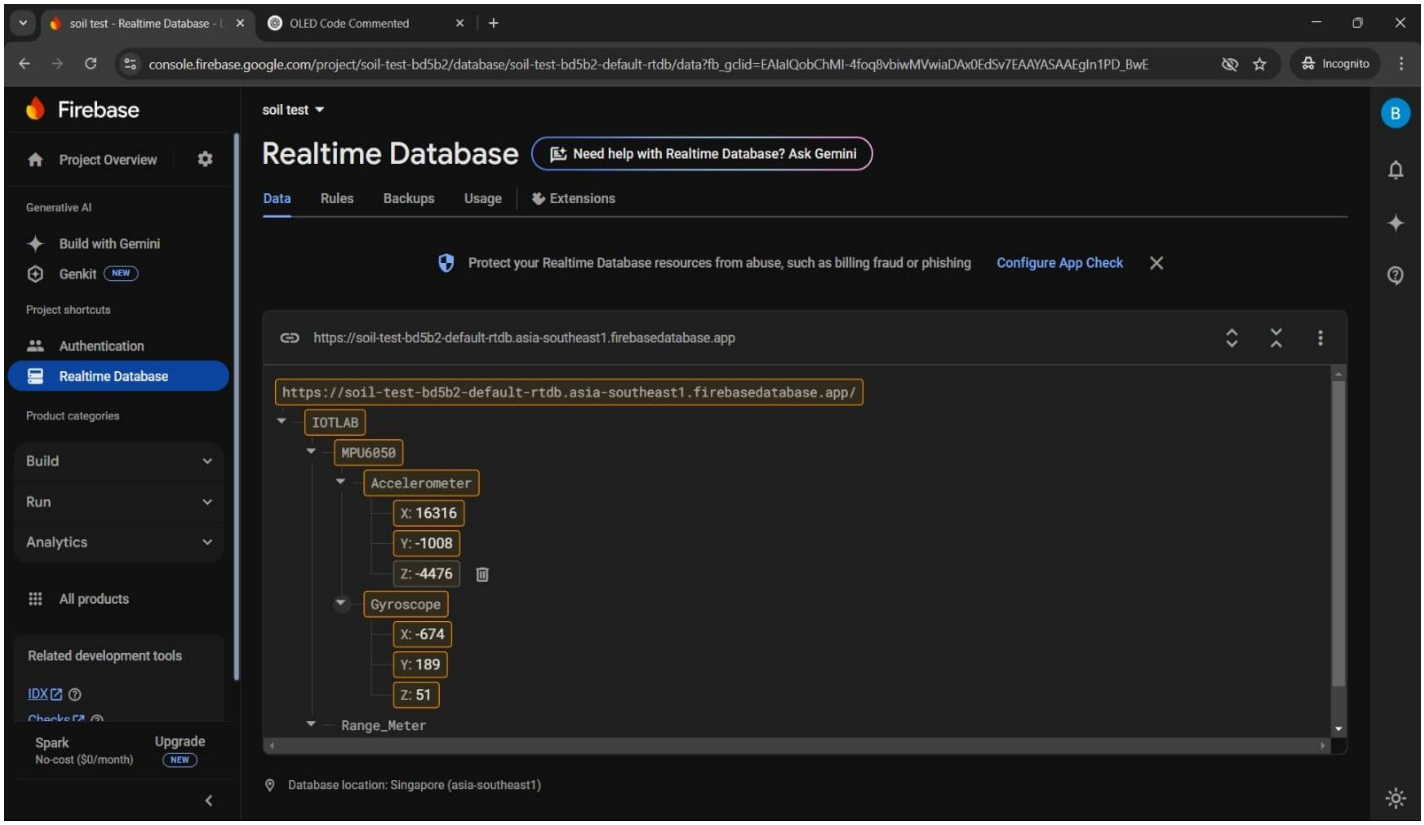
```
} else {
Serial.println("\nFailed to connect to Wi-Fi. Restarting ESP..."); delay(5000);
ESP.restart(); // Restart ESP32
}
}
FirebaseConfig firebaseConfig;
 FirebaseAuth firebaseAuth;
void FirebaseInit(){
firebaseConfig.host = FIREBASE_HOST;
 firebaseConfig.signer.tokens.legacy_token = FIREBASE_AUTH;

Firebase.begin(&firebaseConfig, &firebaseAuth); Firebase.reconnectWiFi(true);
      }

void FirebaseWrite(){
// MPU6050 data removed
Firebase.setBool(firebaseData, tagMagSwitch, reedSwitchState);  // Upload reed switch state

}
```

Output: Gravitational Force along XYZ Axis.



Task 9

Remote Range Indicator

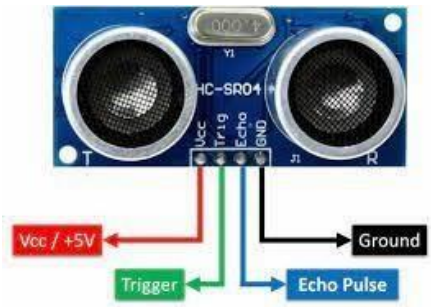
Aim: To implement Internet enabled remote range indicator.

Components Required: GISMO VI Board, Ultrasonic Sensor, USB cable

PIN-OUT

Echo: Echo output
Trig: Trigger input

Ultrasonic Sensor signal	ESP32pins
Trigger	GPIO22
Echo	GPIO23
Vcc	5V
GND	GND



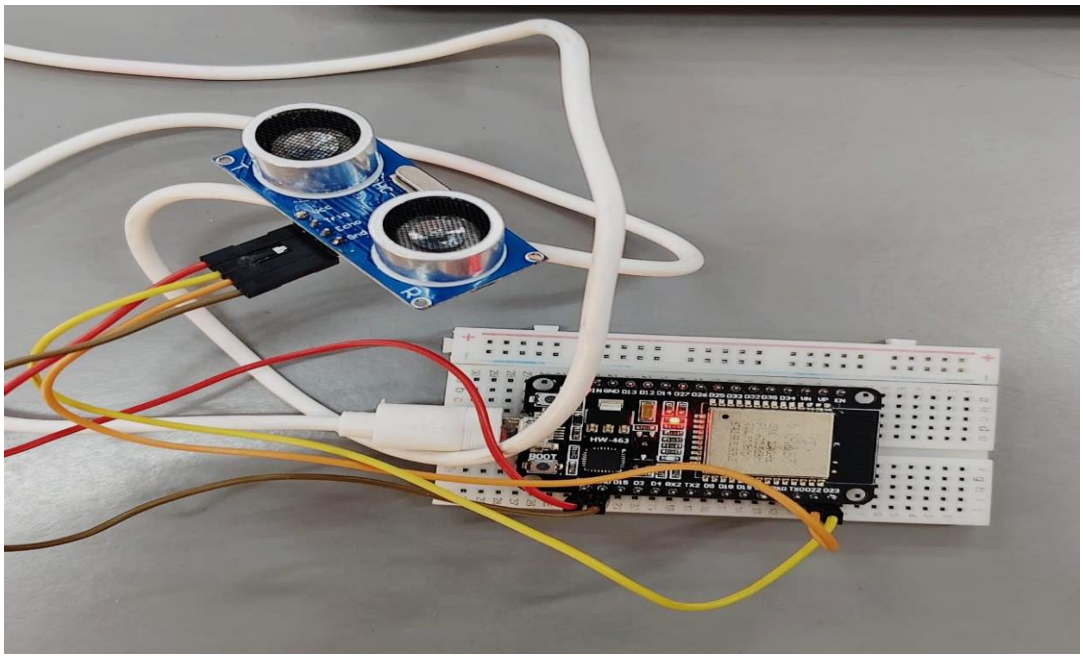
Ultrasonic Sensor (HC-SR04)

The working principle of this module is simple. It sends an ultrasonic pulse out at 40 kHz which travels through the air and if there is an obstacle or object, it will bounce back to the sensor. By calculating the travel time and the speed of sound, the distance can be calculated.

Ultrasonic sensors are a great solution for the detection of clear objects. The sensor head emits an ultrasonic wave and receives the wave reflected back from the target. Ultrasonic /level sensors measure the distance to the target by measuring the time between the emission and reception.

For presence detection, ultrasonic sensors detect objects regardless of the color, surface, or material (unless the material is very soft like wool, as it would absorb sound.)

To detect transparent and other items where optical technologies may fail, ultrasonic sensors are a reliable choice. Sound travels at approximately 340 meters per second. To measure the distance the sound has travelled we use the formula: **Distance = (Time x Speed of Sound) / 2**.The "2"is in the formula because the sound has to travel back and forth. Every 1 Second equals **1000000 Microsecond.**



Program:

```
#define FIREBASE_HOST "soil-test-bd5b2-default-rtdb.asia-southeast1.firebasedatabase.app/"
#define FIREBASE_AUTH "ywbGPCbl9w2G56X8URbTyZUsFDjWVUVNAkfF2IS9"
#define WIFI_SSID "CSE MC IOT"
#define WIFI_PASSWORD "griet7788"
#include <Wire.h>
#include <WiFi.h>
 #include "FirebaseESP32.h"
FirebaseData firebaseData;

// defines pins numbers

const int trigPin = 23; // GPIO5 const int echoPin = 22; // GPIO4
// defines variables long duration;
int distance;

void WiFiInit();
void FirebaseInit();
void FirebaseWrite();
String tag = "IOTLAB/Range_Meter/Range";

void setup()
{
pinMode(trigPin, OUTPUT); // Sets the trigPin as an Output
pinMode(echoPin, INPUT); // Sets the echoPin as an Input
Serial.begin(115200); // Starts the serial communication
WiFiInit();
```

```
FirebaseInit();
}
void loop()
{
// Clears the trigPin
digitalWrite(trigPin, LOW);
delayMicroseconds(2);

// Sets the trigPin on HIGH state for 10 micro seconds

digitalWrite(trigPin, HIGH);
delayMicroseconds(10);
digitalWrite(trigPin, LOW);

// Reads the echo Pin, returns the sound wave travel time in microseconds

duration = pulseIn(echoPin, HIGH);

// Calculating the distance

 distance = duration * 0.034 / 2;

// Prints the distance on the Serial Monitor

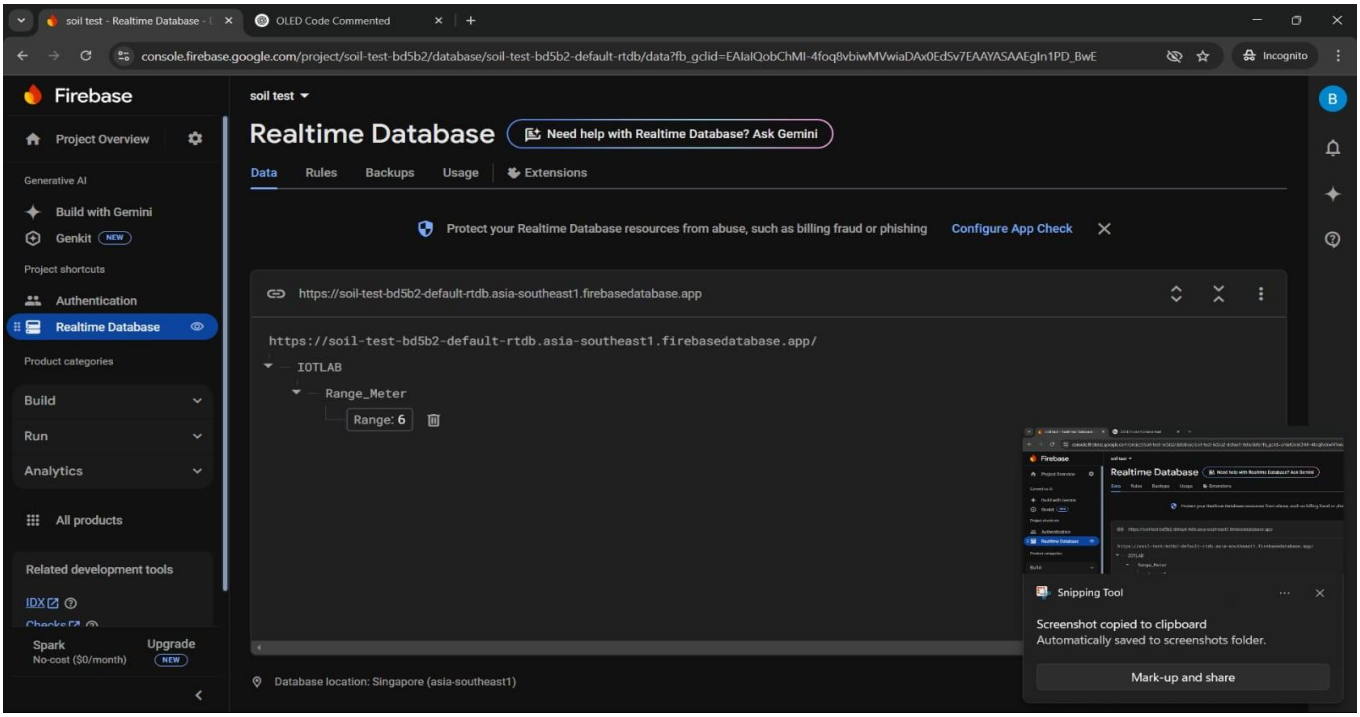
Serial.print("Distance: ");
Serial.println(distance);
FirebaseWrite();
delay(1000);

}
void WiFiInit(){
WiFi.mode(WIFI_STA);
WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
Serial.print("Connecting to Wi-Fi");
int attempts = 0;
while (WiFi.status() != WL_CONNECTED && attempts < 20)
{
// Retry 20 times
 Serial.print(".");
delay(1000);
attempts++;
}
if(WiFi.status() == WL_CONNECTED){
Serial.println("\nConnected!");
Serial.print("IP Address: ");
Serial.println(WiFi.localIP());
}
else
{
Serial.println("\nFailed to connect to Wi-Fi. Restarting ESP...");
```

```
delay(5000);
ESP.restart(); // Restart ESP32
}
}
FirebaseConfig firebaseConfig;
FirebaseAuth firebaseAuth;
void FirebaseInit()
{
firebaseConfig.host = FIREBASE_HOST;
firebaseConfig.signer.tokens.legacy_token = FIREBASE_AUTH;
Firebase.begin(&firebaseConfig, &firebaseAuth);
Firebase.reconnectWiFi(true);
}

void FirebaseWrite()
{
Firebase.setInt(firebaseData, tag, distance);
}
```

Output:



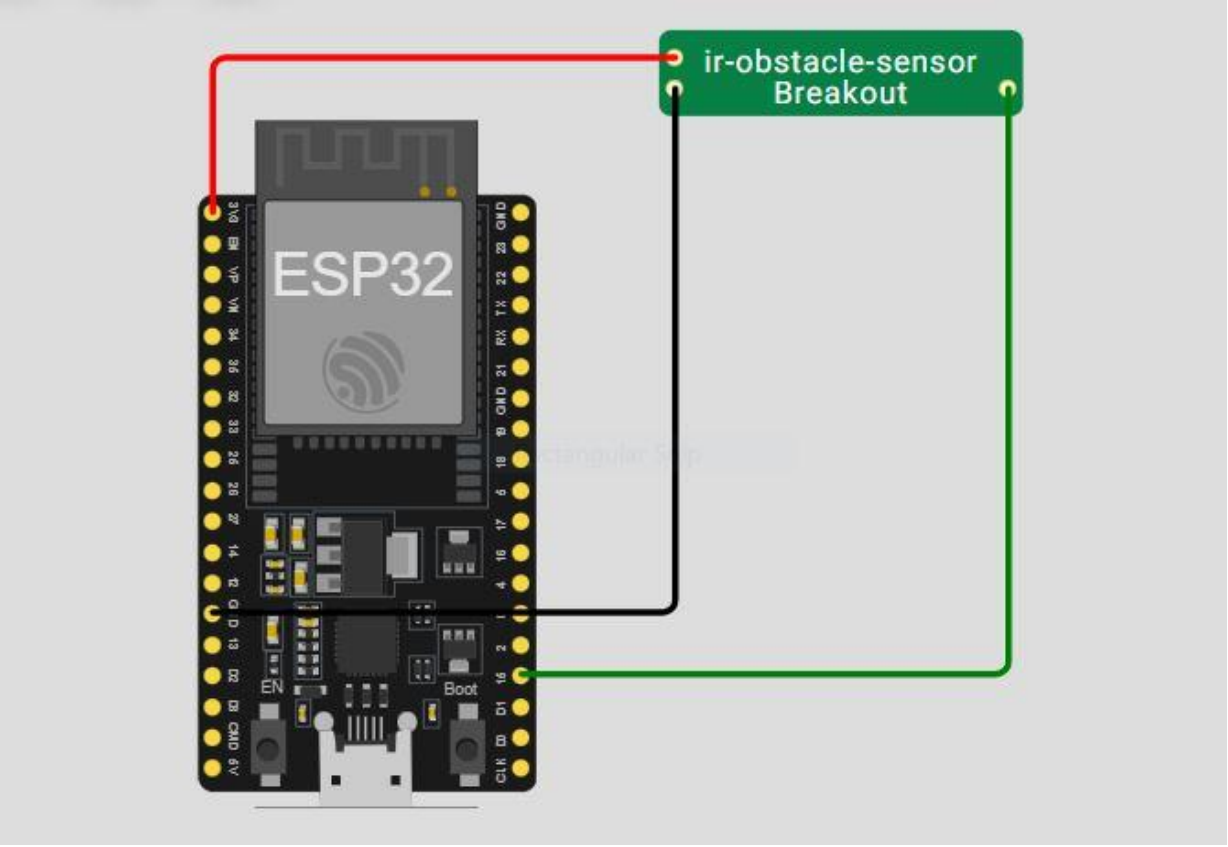
Task 10

IR Sensor

Aim: Implement Internet Enabled IR Sensor using ESP32.

Components Required: GISMOVI Board, IR Sensor, Cable (Wokwi Online Simulator).

Circuit:



Program:

```
#include <WiFi.h>
#include <FirebaseESP32.h>

// --- WiFi Credentials ---
#define WIFI_SSID "Wokwi-GUEST"
#define WIFI_PASSWORD ""

// --- Firebase Credentials ---
#define FIREBASE_HOST "project url"
#define FIREBASE_AUTH "secret key"

// --- Hardware Pins ---
#define IR_SENSOR_PIN 15

// Firebase objects
FirebaseData firebaseData;
FirebaseAuth auth;
FirebaseConfig config;

// State variable
int lastSensorState = -1;

void setup() {
  Serial.begin(115200);

  pinMode(IR_SENSOR_PIN, INPUT_PULLUP); // Use pullup for stability

  Serial.print("Connecting to WiFi");
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  while (WiFi.status() != WL_CONNECTED) {
    Serial.print(".");
    delay(500);
  }
  Serial.println("\nConnected to WiFi!");

  // Initialize Firebase
  config.database_url = FIREBASE_HOST;
  config.signer.tokens.legacy_token = FIREBASE_AUTH;
  Firebase.begin(&config, &auth);
  Firebase.reconnectWiFi(true);

  Serial.println("Firebase Initialized. Monitoring Custom IR Sensor...");
}

void loop() {
  // Read your custom chip's output
  int currentState = digitalRead(IR_SENSOR_PIN);
```

```
  // If the state changes, push to Firebase
  if (currentState != lastSensorState) {

    String sensorStatus = "";

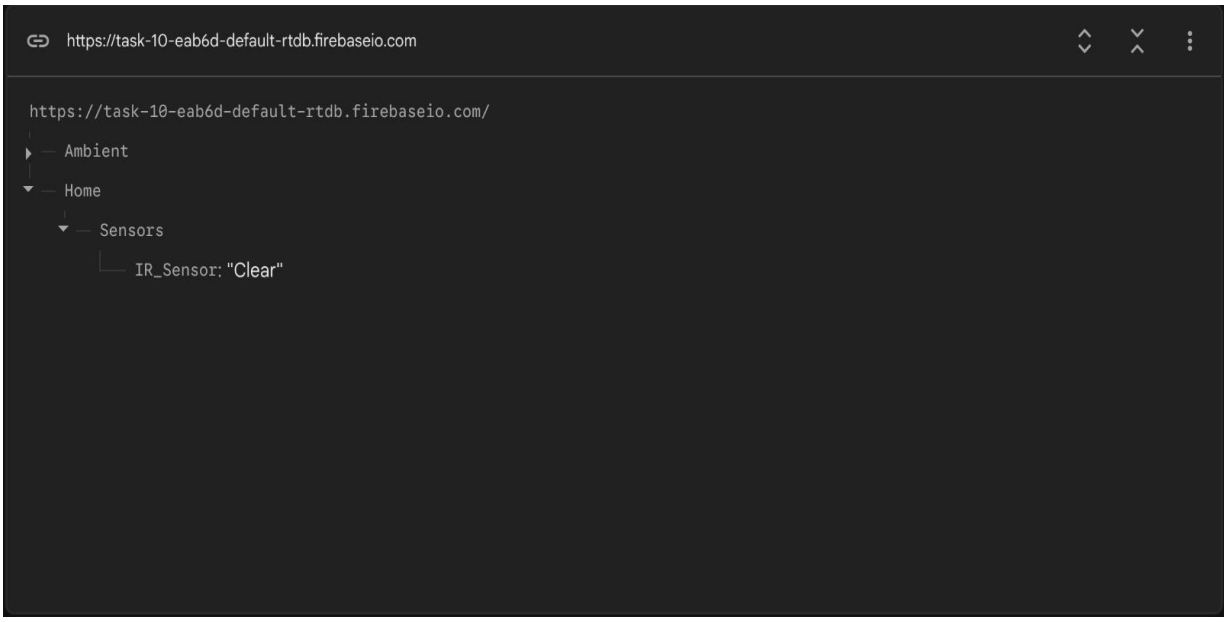
    // IR is Active LOW (0 = Obstacle, 1 = Clear)
    if (currentState == LOW) {
      Serial.println("IR Sensor: Obstacle Detected! (Pushed to Firebase)");
      sensorStatus = "Obstacle Detected";
    } else {
      Serial.println("IR Sensor: Clear! (Pushed to Firebase)");
      sensorStatus = "Clear";
    }

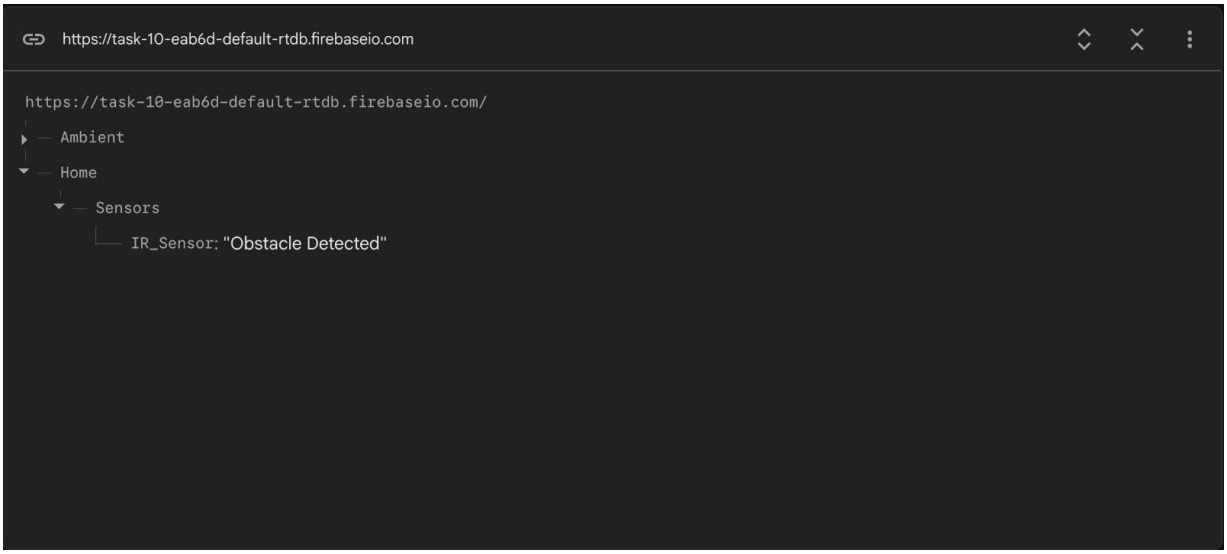
    // Push to Database
    Firebase.setString(firebaseData, "/Home/Sensors/IR_Sensor", sensorStatus);

    lastSensorState = currentState;
  }

  delay(10);
}
```

Output:





https://wokwi.com/projects/458349113673828353

Task 11

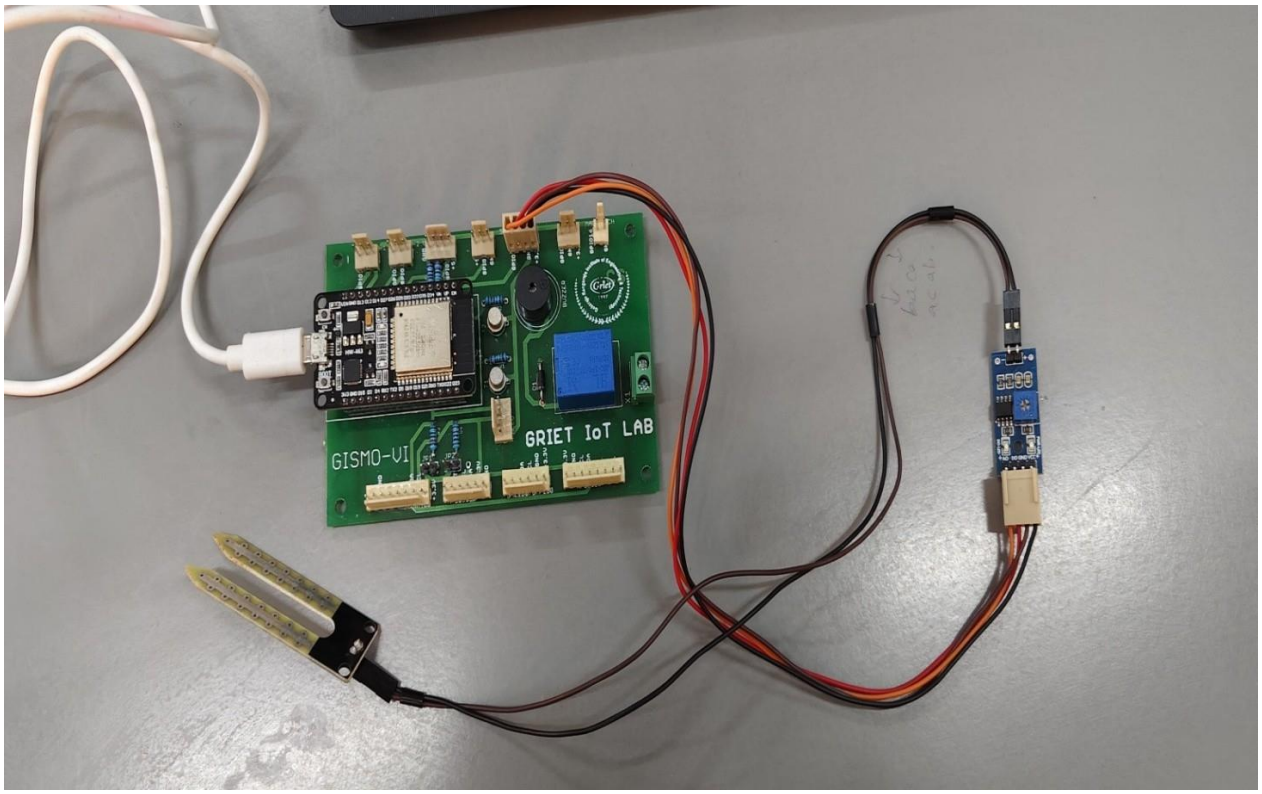
Smart Garden Maintenance

Aim: Internet enabled Smart Garden Maintenance.

Components Required: Soil Moisture Sensor, connecting wires, ESP32.

PIN-OUT:

Soil Moisture sensor	ESP32pins
AO	GPIO34
DO	NC
Vcc	3.3V
GND	GND



Program:

```
/*Soil Moisture sensor pin: GPIO34
*Relay pin  : GPIO13
#include"FirebaseESP32.h"
#defineFIREBASE_HOST"xxxxxxxxxxxxxxxxxxxxxxxxxxxxx"//DonotincludehttpsinFIREBASE_HOST
#define FIREBASE_AUTH "xxxxxxxxxxxxxxxxxxxxxxxxxxxx"
#defineWIFI_SSID"XXXXXX"
#defineWIFI_PASSWORD "XXXXXXXX"
//DefineFirebaseDataobject FirebaseData firebaseData;
#definesmPin34
#definerelayPin13
int smValue;
 int smLimit = 3000;
int delayTime=2000;
String smValueCloud;

void setup()
{
//putyoursetupcodehere,torunonce:

pinMode(relayPin,OUTPUT);
Serial.begin(115200);

//Connecting to Wi-Fi network

WiFi.begin(WIFI_SSID,WIFI_PASSWORD);
Serial.print("ConnectingtoWi-Fi");

while(WiFi.status()!= WL_CONNECTED)
{
Serial.print("."); delay(300);
}
Serial.println();
 Serial.print("ConnectedwithIP:");
Serial.println(WiFi.localIP());
Serial.println();
Firebase.begin(FIREBASE_HOST,FIREBASE_AUTH);
Firebase.reconnectWiFi(true);
}

void loop(){

//put your main code here, to run repeatedly:

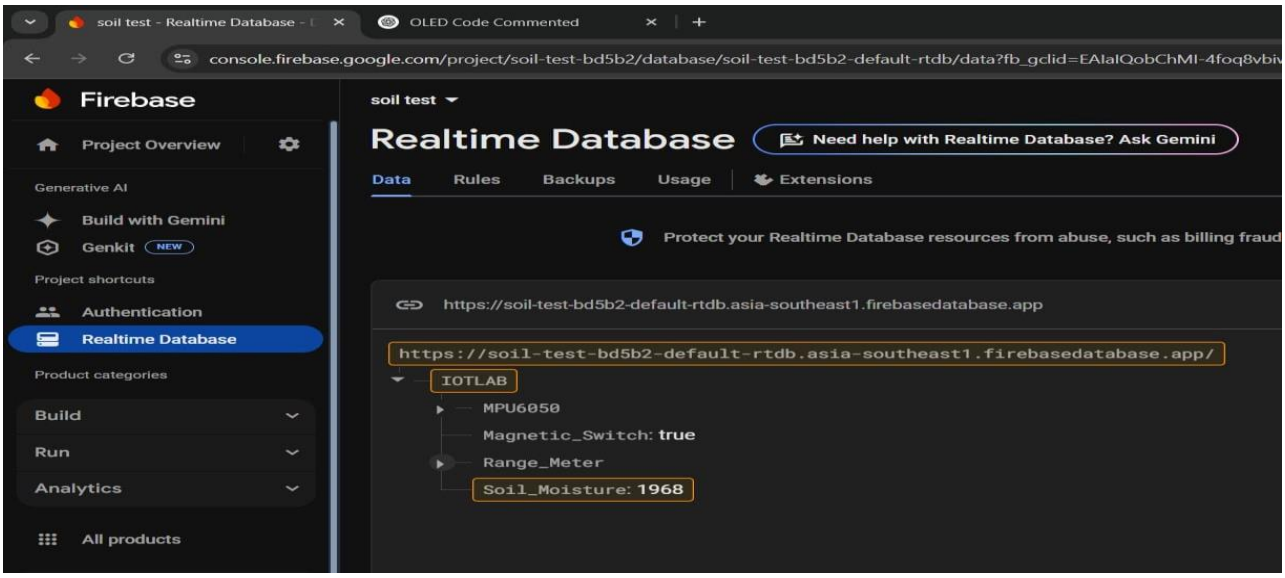
String smCloudFull;
String smCloud;

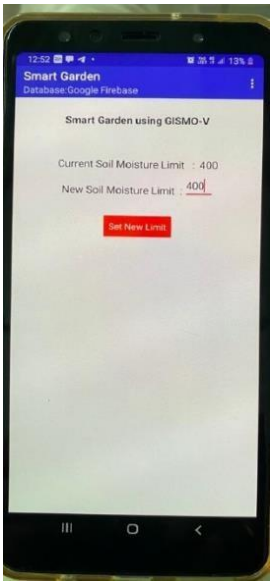
if(Firebase.getString(firebaseData,"xxx/IOTLAB/Smart_Garden/SM_Threshold",smCloudFull))
```

```
{
smCloud = smCloudFull.substring(2,smCloudFull.length()-2);
 smLimit=smCloud.toInt();
Serial.println(smLimit);
}

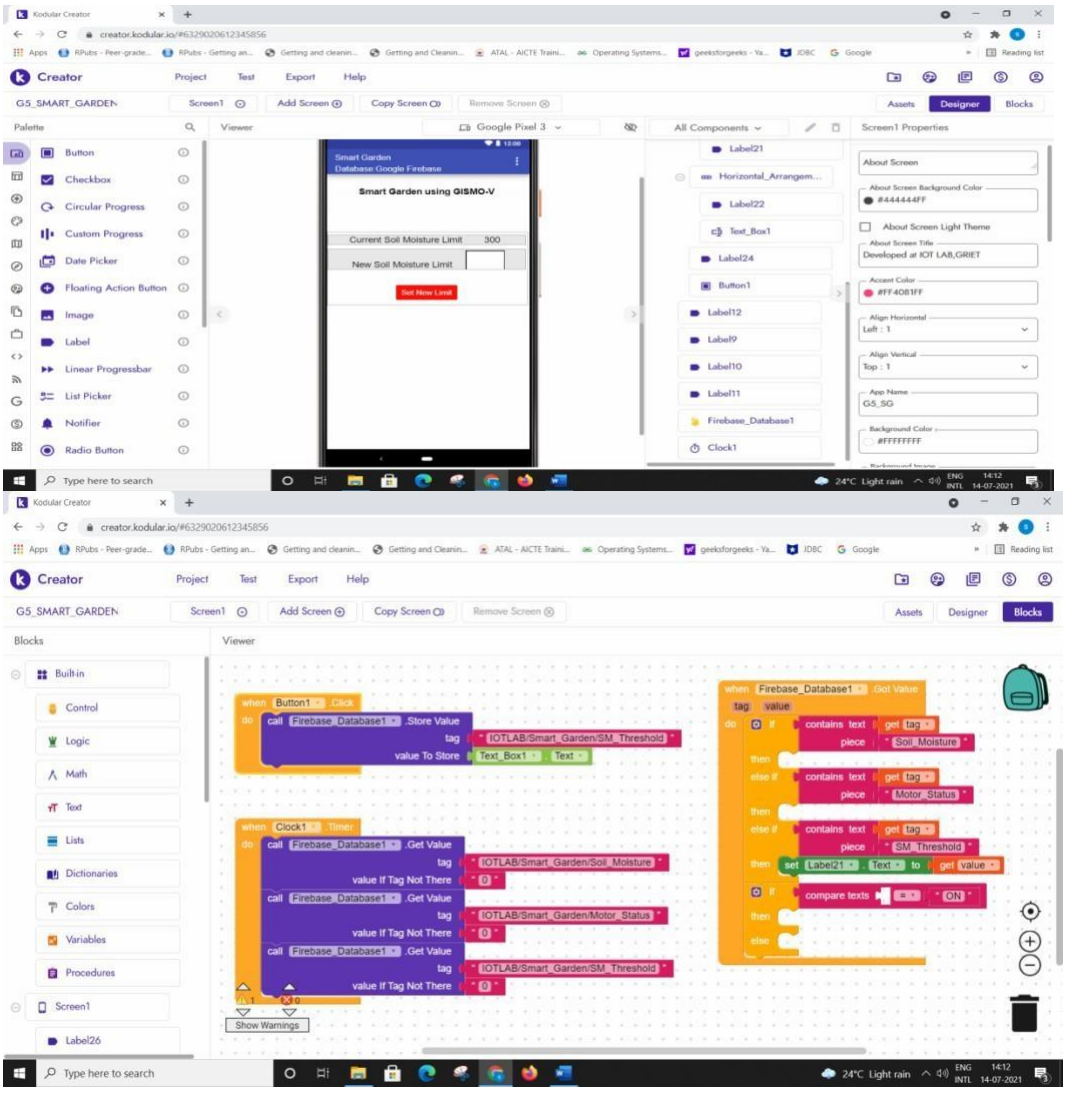
smValue=4095-analogRead(smPin);
Serial.print("Soil Moisture = ");
Serial.println(smValue);

Firebase.setInt(firebaseData,"IOTLAB/Smart_Garden/Soil_Moisture",smValue);
if (smValue < smLimit)
{
digitalWrite(relayPin,HIGH);
Serial.println("Motor turned ON");
Firebase.setString(firebaseData,"IOTLAB/Smart_Garden/Motor_Status","ON");
}
else{
digitalWrite(relayPin,LOW);
Serial.println("Motor turned OFF");
Firebase.setString(firebaseData,"IOTLAB/Smart_Garden/Motor_Status","OFF");
}
delay(delayTime);
}
```


soil test - Realtime Database
OLED Code Commented
console.firebase.google.com/project/soil-test-bd5b2/database/soil-test-bd5b2-default-rtdb/data?fb_gclid=EAIaIQobChMI-4foq8vbiv
Firebase
Project Overview
Generative AI
Build with Gemini
Genkit NEW
Project shortcuts
Authentication
Realtime Database
Product categories
Build
Run
Analytics
All products
soil test
Realtime Database
Need help with Realtime Database? Ask Gemini
Data
Rules
Backups
Usage
Extensions
Protect your Realtime Database resources from abuse, such as billing fraud
https://soil-test-bd5b2-default-rtdb.asia-southeast1.firebasedatabase.app
https://soil-test-bd5b2-default-rtdb.asia-southeast1.firebasedatabase.app/
IOTLAB
MPU6050
Magnetic_Switch: true
Range_Meter
Soil_Moisture: 1968


Smart Garden
Database:Google Firebase
Smart Garden using GISMO-V
Current Soil Moisture Limit : 400
New Soil Moisture Limit : 400
Set New Limit

Here using a mobile app we can set the Soil Moisture limit, depending up on the seasons we can change the limit of the Soil Moisture.



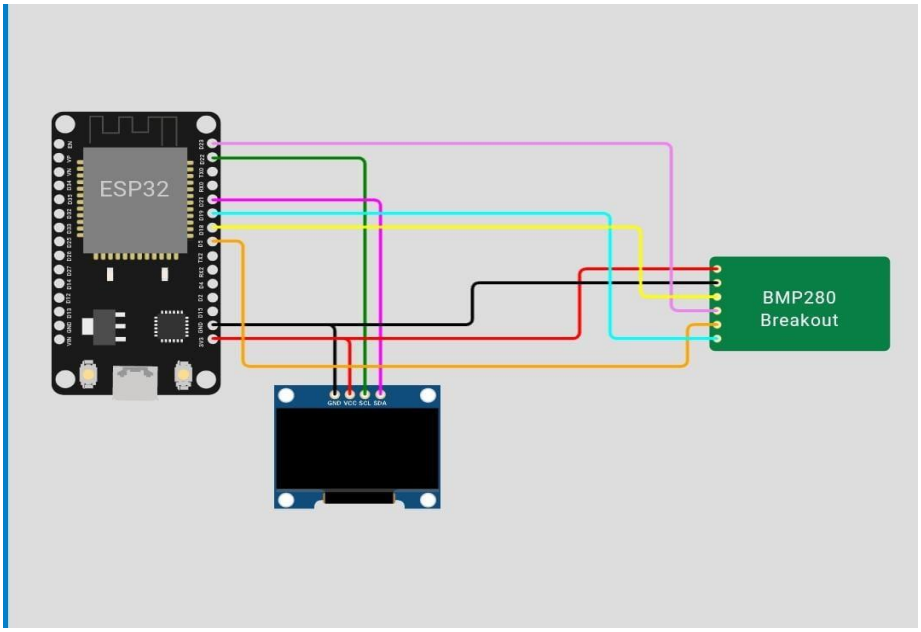
Task 12

BMP 280

Aim: Internet enabled Display of Ambient Parameter (BMP).

Components Required: GISMOVI, BMP Sensor, USB Cable Pin-Out

Circuit:



Program:

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <Adafruit_Sensor.h>
#include <SPI.h>
#include <Adafruit_BMP280.h>

// --- ADDED FOR FIREBASE ---
#include <WiFi.h>
#include <FirebaseESP32.h>

#define FIREBASE_HOST "project url"
```

```
#define FIREBASE_AUTH "secret key"
#define WIFI_SSID "Wokwi-GUEST" // Change if using on physical hardware
#define WIFI_PASSWORD ""

FirebaseData firebaseData;
FirebaseAuth auth;
FirebaseConfig config;
// -------------------------

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64

#define BMP_SCK  (18)
#define BMP_MISO (19)
#define BMP_MOSI (23)
#define BMP_CS   (5)

Adafruit_BMP280 bmp(BMP_CS, BMP_MOSI, BMP_MISO,  BMP_SCK);

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);


void setup() {
  Serial.begin(115200);
  Serial.println(F("BMP280 test"));
  while ( !Serial )
    delay(100);

  unsigned status;
  status = bmp.begin();
  if (!status) {
    Serial.println(F("Could not find a valid BMP280 sensor, check wiring or "
                     "try a different address!"));
    while (1)
      delay(10);
  }

  bmp.setSampling(Adafruit_BMP280::MODE_NORMAL,
                  Adafruit_BMP280::SAMPLING_X2,
                  Adafruit_BMP280::SAMPLING_X16,
                  Adafruit_BMP280::FILTER_X16,
                  Adafruit_BMP280::STANDBY_MS_500);

  if(!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println(F("SSD1306 allocation failed"));
    for(;;);
  }

  // --- ADDED: CONNECT TO WI-FI ---
```

```
  Serial.print("Connecting to Wi-Fi");
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  while (WiFi.status() != WL_CONNECTED) {
    Serial.print(".");
    delay(300);
  }
  Serial.println();
  Serial.print("Connected with IP: ");
  Serial.println(WiFi.localIP());

  // --- ADDED: INITIALIZE FIREBASE ---
  config.host = FIREBASE_HOST;
  config.signer.tokens.legacy_token = FIREBASE_AUTH;
  Firebase.begin(&config, &auth);
  Firebase.reconnectWiFi(true);
  // --------------------------------

  delay(2000);
  display.clearDisplay();
  display.setTextColor(WHITE);
}

void loop() {

  float h = 12;
  float t = 11;

  if (isnan(h) || isnan(t)) {
    Serial.println("DHT Sensor could not be read correctly!");
  }
  delay(1000);
  display.clearDisplay();

  // Storing values in variables to send to Firebase and Display
  float currentTemp = bmp.readTemperature();
  float currentPressure = bmp.readPressure();

  Serial.print(F("Temperature = "));
  Serial.print(currentTemp);
  Serial.println(" *C");

  display.setTextSize(1);
  display.setCursor(0,0);
  display.print("Temperature: ");
  display.setTextSize(2);
  display.setCursor(0,10);
  display.print(currentTemp);
  display.print(" ");
  display.setTextSize(1);
```

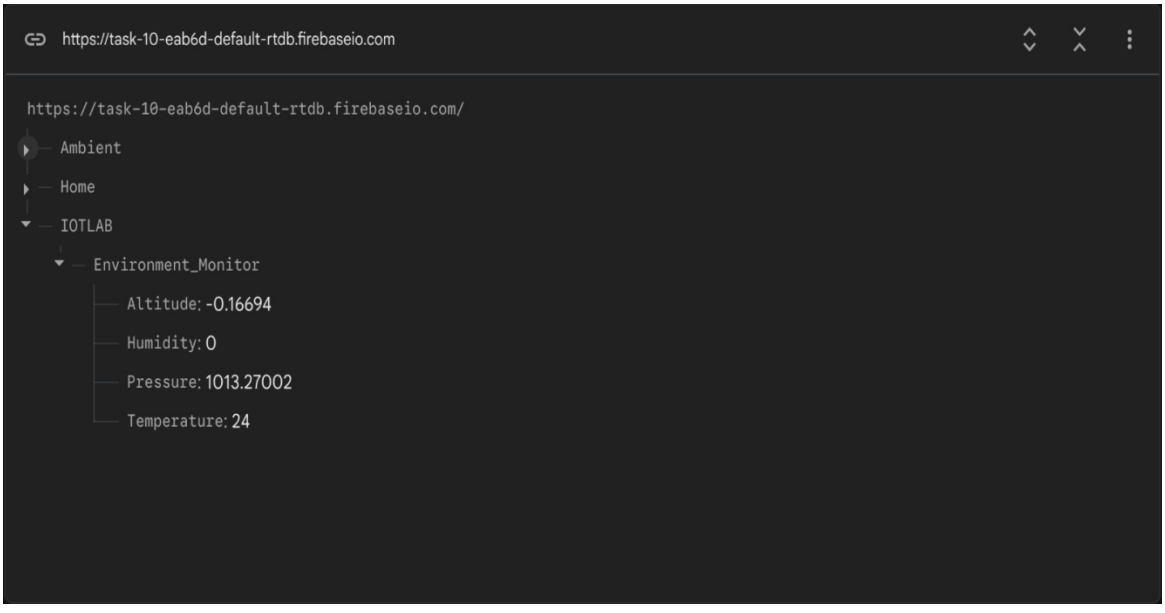
```
  display.cp437(true);
  display.write(167);
  display.setTextSize(2);
  display.print("C");

  display.setTextSize(1);
  display.setCursor(0, 35);
  display.print("Pressure: ");
  display.setTextSize(2);
  display.setCursor(0, 45);
  display.print(currentPressure);
  display.print(" Pa");

  display.display();

  // --- ADDED: SEND DATA TO FIREBASE ---
  if(Firebase.setFloat(firebaseData, "IOTLAB/Environment_Monitor/Temperature", currentTemp)) {
    Serial.println("Data sent to Firebase!");
  } else {
    Serial.println("Firebase Error: " + firebaseData.errorReason());
  }
  Firebase.setFloat(firebaseData, "IOTLAB/Environment_Monitor/Pressure", currentPressure);
  // ------------------------------------
}
```

Output:



Task 13

Home Safety and Security System

Aim: Internet Enabled Home Safety and security system.

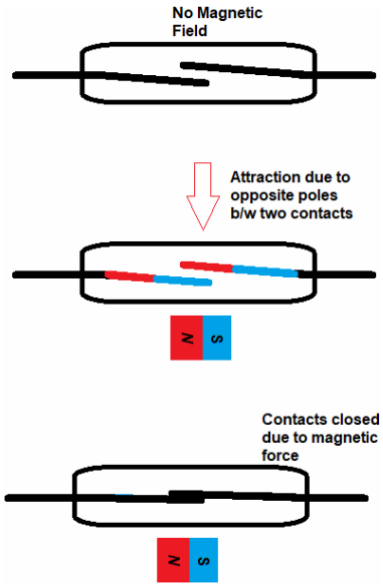
Components Required: GISMOVI, Magnetic switch, USB Cable.

PIN-OUT:

Magnetic Sensor	ESP32pins
Sensor o/p contact1	GPIO16
Sensor o/p contac2	GND
Vcc	3.3V

Magnetic sensors **detect moving ferrous metal**. The simplest magnetic sensor consists of a wire coiled around a permanent magnet. A ferrous object approaching the sensor changes magnetic flux through the coil, generating a voltage at the coil terminals Magnetic sensors measure speeds up to 600,000 rpm.

Magnetic Switch: The switching mechanism is comprised of **two Ferro magnetic blades, separated by only a few microns**. When a magnet approaches these blades, the two blades pull toward one another. Once touching, the blades close the normally open (NO) contacts, allowing electricity to flow.
The use of INPUT_PULLUP with pinMode(). It monitors the state of a switch by establishing serial communication between your Arduino and your computer over USB.



The INPUT_PULL UP also does **the same function as INPUT** however only differing in the aspect of base.

Voltage, where the INPUT argument pulls down the voltage of that port to 0V every time there is no voltage is detected across the port while the INPUT_PULLUP argument pulls up the voltage across the port to the maximum.

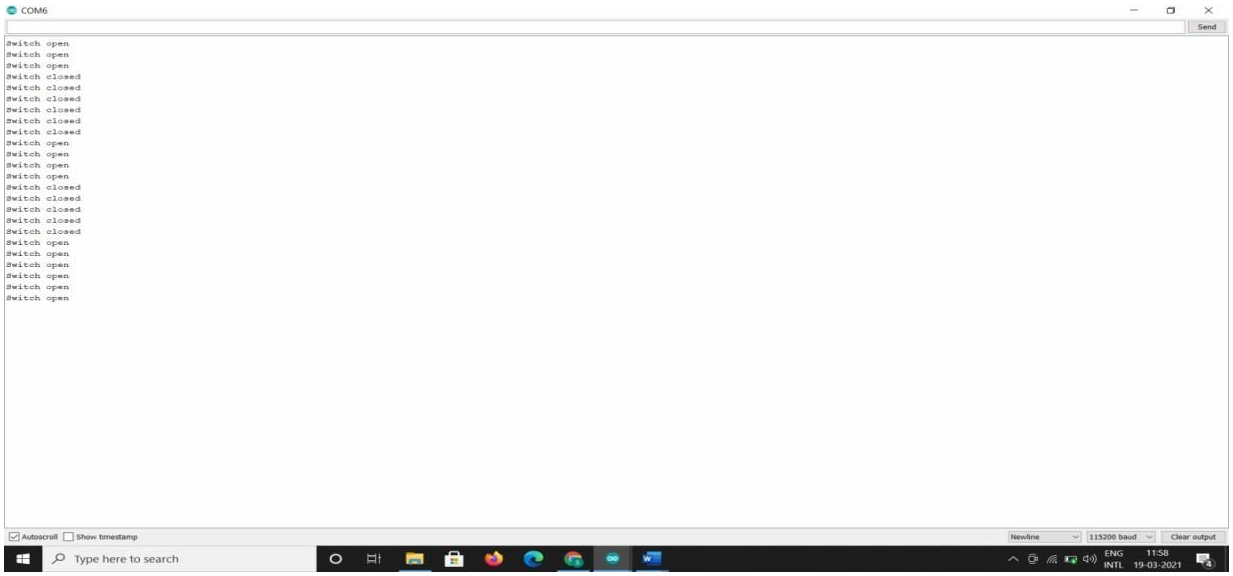
Program:

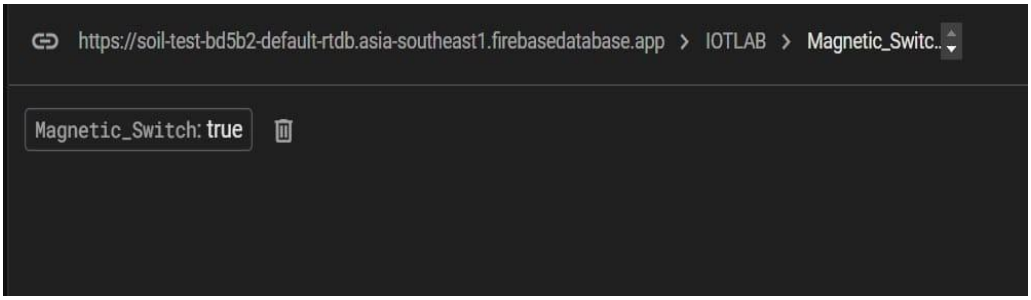
```
#define magSW 16

void setup()
{
pinMode(magSW, INPUT_PULLUP);
Serial.begin(115200);
}

void loop()
{
int sw_status;
sw_status  = digitalRead(magSW);
 if (sw_status)
  Serial.println("Switch open");
  else
Serial.println("Switchclosed");
delay(1000);
}
```

Output:


COM6
Send
Switch open
Switch open
Switch open
Switch closed
Switch closed
Switch closed
Switch closed
Switch closed
Switch closed
Switch open
Switch open
Switch open
Switch open
Switch closed
Switch closed
Switch closed
Switch closed
Switch closed
Switch open
Switch open
Switch open
Switch open
Switch open
Autoscroll
Show timestamp
Newline
115200 baud
Clear output
Type here to search
ENG
INTL
11:58
19-03-2021

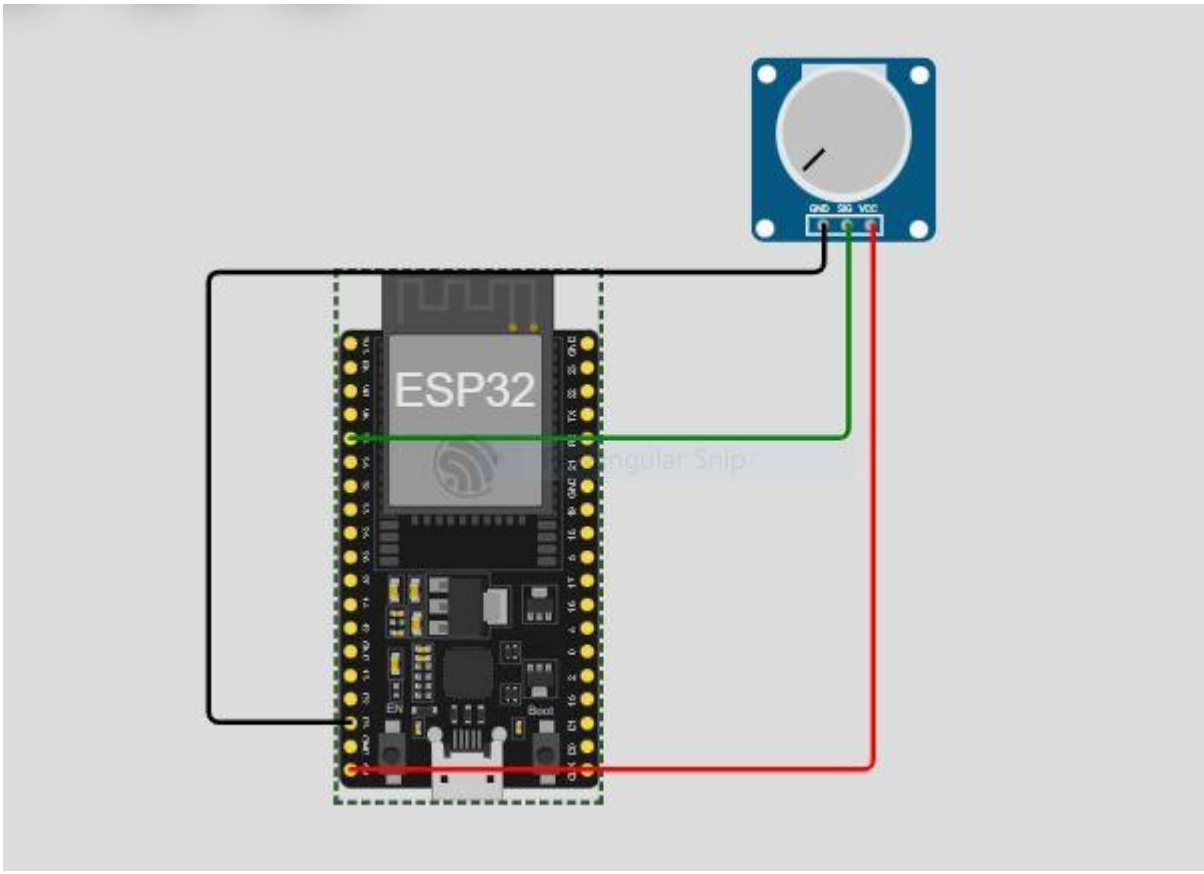

https://soil-test-bd5b2-default-rtdb.asia-southeast1.firebasedatabase.app
IOTLAB
Magnetic_Switc..
Magnetic_Switch: true

Task 14
APDS-9960

Aim: Implement Internet Enabled Ambient light Proximity Digital and Gesture Sensor (APDS-9960) using ESP32.

Components Required: GISMOVI Board, APDS-9960 Sensor, Cable (Wokwi Online Simulator).

Circuit:



Program :

```
#include <WiFi.h>
#include <FirebaseESP32.h>

// --- WiFi Credentials ---
#define WIFI_SSID "Wokwi-GUEST"
#define WIFI_PASSWORD ""

// --- Firebase Credentials ---
#define FIREBASE_HOST "PROJECT URL" // e.g., task-10-eab6d-default-rtdb.firebaseio.com
#define FIREBASE_AUTH "SECRET KEY"

// --- Hardware Mock Pins ---
#define POT_PIN 34   // Simulates Proximity/Light
#define BTN_UP 12    // Simulates UP gesture
#define BTN_DOWN 14  // Simulates DOWN gesture
#define BTN_LEFT 27  // Simulates LEFT gesture
#define BTN_RIGHT 26 // Simulates RIGHT gesture

// Firebase objects
FirebaseData firebaseData;
FirebaseAuth auth;
FirebaseConfig config;

// Timers and State variables
unsigned long lastAmbientRead = 0;
int lastUp = HIGH, lastDown = HIGH, lastLeft = HIGH, lastRight = HIGH;

void setup() {
  Serial.begin(115200);

  // Set up pins (Buttons use INPUT_PULLUP so they are HIGH normally, LOW when pressed)
  pinMode(BTN_UP, INPUT_PULLUP);
  pinMode(BTN_DOWN, INPUT_PULLUP);
  pinMode(BTN_LEFT, INPUT_PULLUP);
  pinMode(BTN_RIGHT, INPUT_PULLUP);

  // Connect to WiFi
  Serial.print("Connecting to WiFi");
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  while (WiFi.status() != WL_CONNECTED) {
    Serial.print(".");
    delay(500);
  }
  Serial.println("\nConnected to WiFi!");
```

```
  // Initialize Firebase
  config.database_url = FIREBASE_HOST;
  config.signer.tokens.legacy_token = FIREBASE_AUTH;
  Firebase.begin(&config, &auth);
  Firebase.reconnectWiFi(true);
  Serial.println("Firebase Initialized. Ready for inputs!");
}

void checkGesture(int pin, int &lastState, String gestureName) {
  int currentState = digitalRead(pin);
  // Detect a button press (state goes from HIGH to LOW)
  if (currentState == LOW && lastState == HIGH) {
    Serial.print("Simulated Gesture: ");
    Serial.println(gestureName);

    // Instantly push the gesture to Firebase
    Firebase.setString(firebaseData, "/Ambient/Last_Gesture", gestureName);
    delay(50); // Simple debounce
  }
  lastState = currentState;
}

void loop() {
  // 1. INSTANT GESTURE DETECTION (Checking all 4 buttons)
  checkGesture(BTN_UP, lastUp, "UP");
  checkGesture(BTN_DOWN, lastDown, "DOWN");
  checkGesture(BTN_LEFT, lastLeft, "LEFT");
  checkGesture(BTN_RIGHT, lastRight, "RIGHT");

  // 2. PERIODIC AMBIENT & PROXIMITY READING (Every 5 Secs)
  if (millis() - lastAmbientRead > 5000) {

    // Read the potentiometer (0 to 4095)
    int potValue = analogRead(POT_PIN);

    // Map the dial to fake Proximity (0-255) and fake Light (0-10000)
    int fakeProximity = map(potValue, 0, 4095, 0, 255);
    int fakeLight = map(potValue, 0, 4095, 0, 10000);

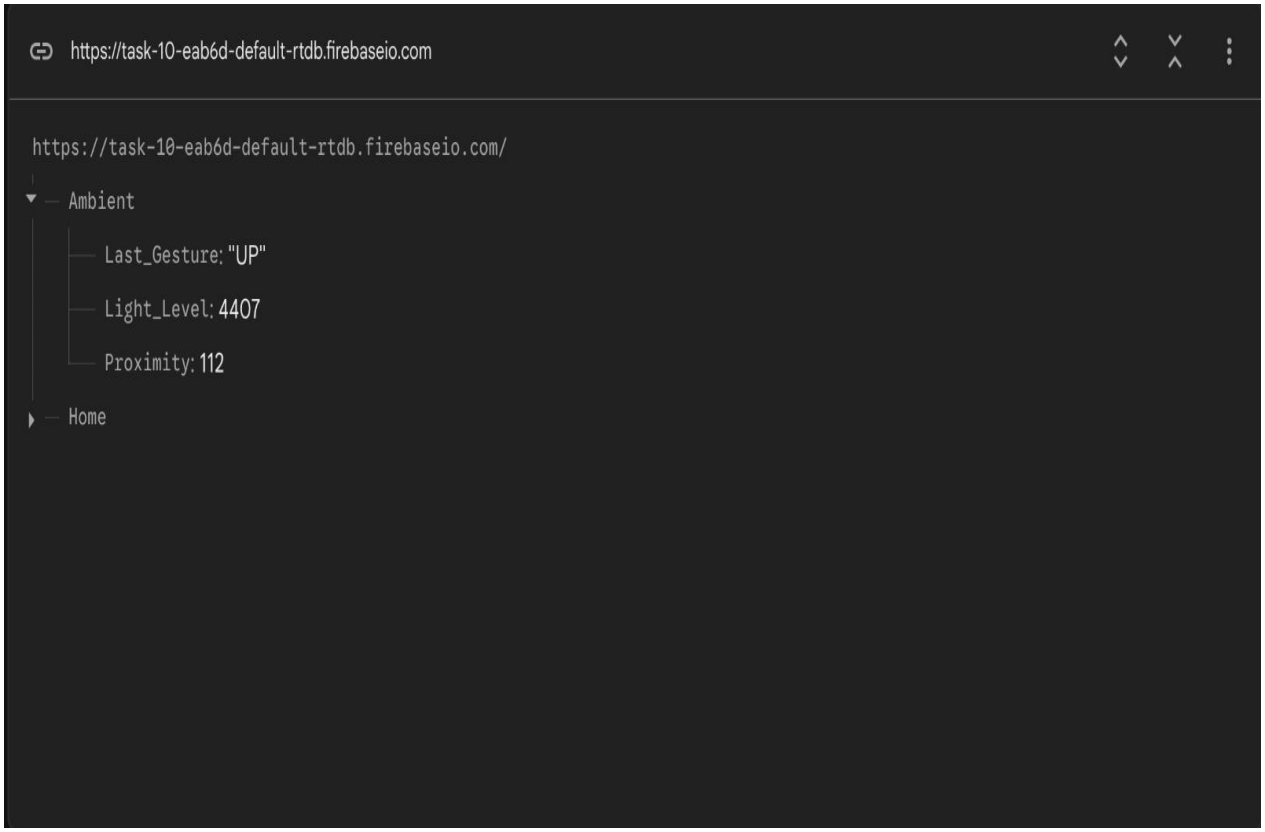
    Serial.print("Simulated Proximity: ");
    Serial.print(fakeProximity);
    Serial.print(" | Simulated Light: ");
    Serial.println(fakeLight);

    // Push to Firebase
    Firebase.setInt(firebaseData, "/Ambient/Proximity", fakeProximity);
    Firebase.setInt(firebaseData, "/Ambient/Light_Level", fakeLight);
```

```
    lastAmbientRead = millis();
  }

  delay(10); // Keep simulator running smoothly
}
```

Output:



https://wokwi.com/projects/458179839392048129

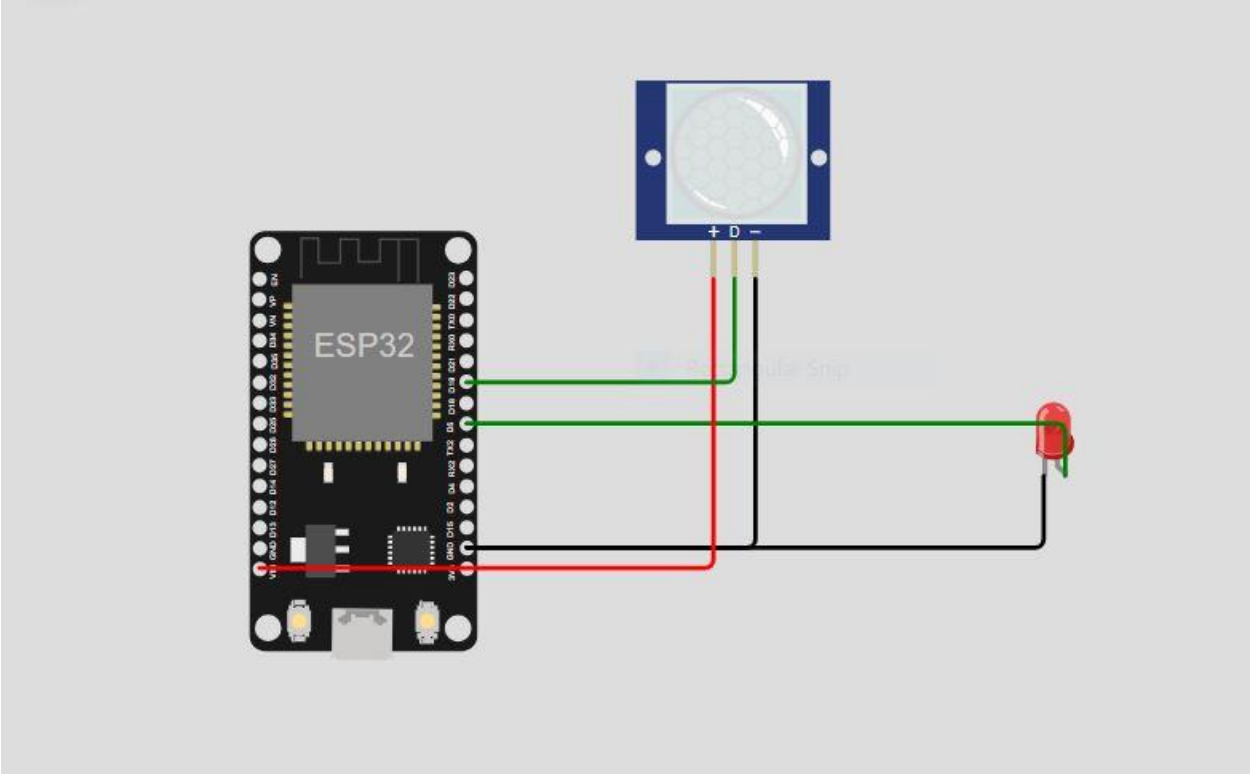
Task 15

PIR Sensor

Aim: Implement Internet Enabled PIR Sensor using ESP32.

Components Required: GISMOVI Board, PIR Sensor, Cable (Wokwi Online Simulator).

Circuit:



https://wokwi.com/projects/458174275852346369

Program:

```
#include <WiFi.h>
#include <FirebaseESP32.h>

// --- WiFi Credentials ---
#define WIFI_SSID "Wokwi-GUEST"
#define WIFI_PASSWORD ""

// --- Firebase Credentials ---
#define FIREBASE_HOST "project url"
```

```
#define FIREBASE_AUTH "secret key"

// --- Hardware Pins ---
#define SENSOR_PIN 15  // PIR Sensor Pin
#define LED_PIN 5      // LED Pin

// Define Firebase objects
FirebaseData firebaseData;
FirebaseAuth auth;
FirebaseConfig config;

// --- Dynamic State Variable ---
int lastSensorState = -1;

void setup() {
  Serial.begin(115200);

  // Initialize Pins
  pinMode(SENSOR_PIN, INPUT);
  pinMode(LED_PIN, OUTPUT);

  // Connect to WiFi
  Serial.print("Connecting to WiFi");
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  while (WiFi.status() != WL_CONNECTED) {
    Serial.print(".");
    delay(500);
  }
  Serial.println("\nConnected to WiFi!");

  // Configure and Initialize Firebase
  config.database_url = FIREBASE_HOST;
  config.signer.tokens.legacy_token = FIREBASE_AUTH;
  Firebase.begin(&config, &auth);
  Firebase.reconnectWiFi(true);

  Serial.println("Firebase Initialized.");
  Serial.println("Waiting for motion...");
}

void loop() {
  // Read the current sensor value instantly
  int currentState = digitalRead(SENSOR_PIN);

  // ONLY trigger actions if the state has ACTUALLY changed
  if (currentState != lastSensorState) {

    String sensorStatus = "";
```

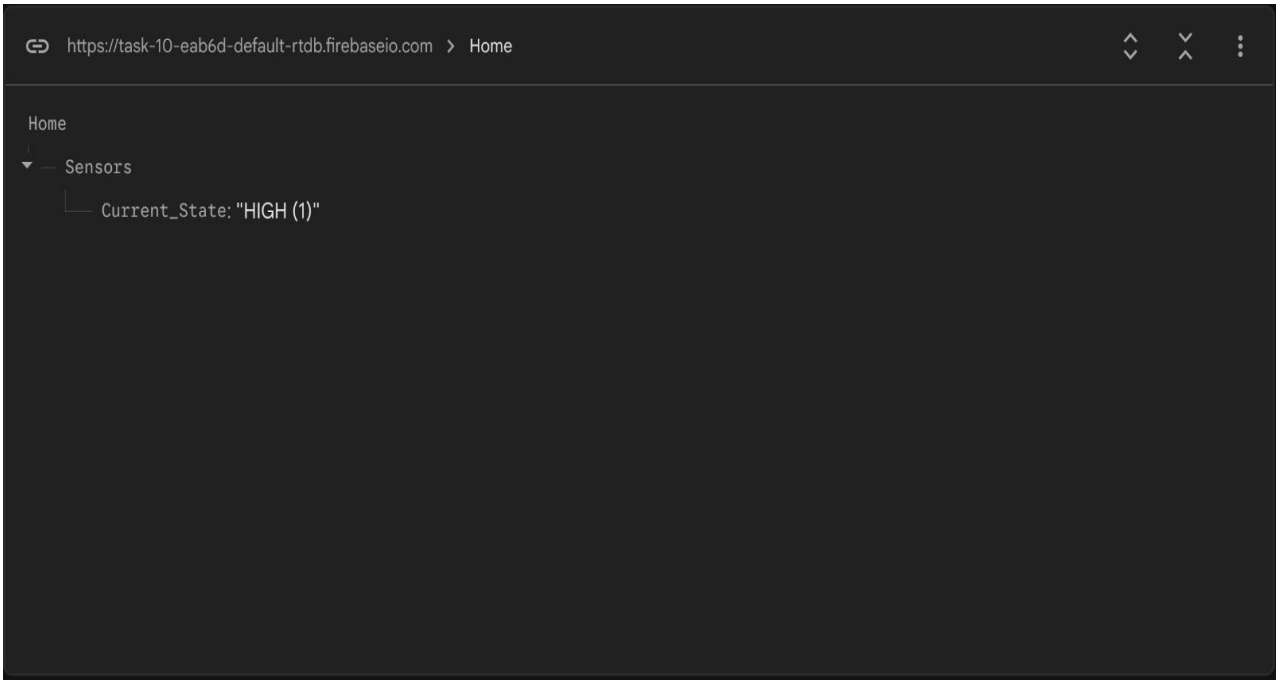
```
  // If motion is detected (HIGH / 1)
  if (currentState == HIGH) {
    digitalWrite(LED_PIN, HIGH); // Turn LED ON
    Serial.println("Motion detected! LED ON (1)");
    sensorStatus = "Motion Detected";
  }
  // If motion ended (LOW / 0)
  else {
    digitalWrite(LED_PIN, LOW);  // Turn LED OFF
    Serial.println("Motion ended! LED OFF (0)");
    sensorStatus = "Clear";
  }

  // Push the new status to Firebase Realtime Database
  if (Firebase.setString(firebaseData, "/Home/Sensors/Current_State", sensorStatus)) {
    Serial.println(" -> Real-time status instantly updated in Firebase.");
  } else {
    Serial.print(" -> Firebase Error: ");
    Serial.println(firebaseData.errorReason());
  }

  // Update the memory variable
  lastSensorState = currentState;
 }

 // A tiny 10ms delay keeps the simulation running smoothly
 delay(10);
}
```

Output:


https://task-10-eab6d-default-rtdb.firebaseio.com › Home
Home
Sensors
Current_State: "HIGH (1)"